

# Avinash Group App — Documentation

Technical Reference + User Guide · site avinas1 · generated from docs/

# avinashgroup\_app — Documentation Index

Complete documentation for the Avinash Group custom app (site **avinas1**, Frappe/ERPNext v15). Two sets:

- **Technical reference** ( `technical/` ) — for developers. Verified against the code on 2026-07-05; where older docs in this folder disagree, the technical chapters win.
- **User guide** ( `user_guide/` ) — task-oriented, for accountants, HR, purchase/sales staff and administrators.

## Technical reference

| #  | Chapter                                                   | Covers                                                                                                                                                                                        |
|----|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <a href="#">Architecture Overview</a>                     | repo layout, hooks.py wiring map, conventions, patches, fixtures, dev workflow                                                                                                                |
| 2  | <a href="#">Dynamic Approval System</a>                   | the * -hooked approval engine, doctypes, generated workflow, admin bypass, JS                                                                                                                 |
| 3  | <a href="#">Biometric Attendance &amp; K40 Bridge</a>     | iclock/ADMS, bridge architecture, checkin pipeline, Attendance Fix, allowance engine                                                                                                          |
| 4  | <a href="#">VAT/Excise/TDS, Overrides &amp; Numbering</a> | the Apply-On tax scheme, 9 class overrides, warehouses, voucher numbering, GL patches, dead-code inventory                                                                                    |
| 5  | <a href="#">CBMS Integration</a>                          | IRD e-billing sync, payloads, retry/reconcile schedulers, status model                                                                                                                        |
| 6  | <a href="#">Access Control &amp; Audit</a>                | fiscal-year access, company filtering, audit trail, daily entry summary                                                                                                                       |
| 7  | <a href="#">Document Generator</a>                        | template engine, data sources, PDF/email pipeline, security, authoring                                                                                                                        |
| 8  | <a href="#">Reports &amp; Portal Pages</a>                | all 20+ financial/sales reports, print/PDF infrastructure, portals                                                                                                                            |
| 9  | <a href="#">Doctypes Reference</a>                        | all 26 custom doctypes + exported custom-field bundles (quick map)                                                                                                                            |
| 10 | <a href="#">Complete File Index</a>                       | <b>every source file</b> — location, purpose, why, and wired/console/dead status                                                                                                              |
| 11 | <a href="#">Field Reference</a>                           | <b>every custom field</b> on core doctypes (182) + <b>every field</b> of all <b>41</b> module doctypes (28 file-based + 13 DB-only)                                                           |
| 12 | <a href="#">Reports Appendix</a>                          | full per-report detail: sources, all filters, columns, logic, why                                                                                                                             |
| 13 | <a href="#">Per-Doctype Reference</a>                     | <b>by doctype</b> — for each doctype, everything the app did to it (fields, hooks, JS, overrides, reports, why)                                                                               |
| 14 | <a href="#">Site Customization Inventory</a>              | <b>full avinas1 audit — code + database:</b> every doctype's custom fields, property setters, client scripts, print formats, notifications, permissions (incl. DB-only items not in the repo) |

Two ways to navigate:

- **By feature** — chapters 2-8 explain each mechanism (approvals, VAT, CBMS...).
- **By doctype** — [chapter 13](#) lists, per doctype, everything done to it. [Chapter 10 \(File Index\)](#) does the same for source files ("where is X and why").

## User guide

| # | Chapter                                   | Audience                  |
|---|-------------------------------------------|---------------------------|
| 1 | <a href="#">Overview</a>                  | everyone                  |
| 2 | <a href="#">Approvals</a>                 | requesters & approvers    |
| 3 | <a href="#">Attendance &amp; HR</a>       | HR                        |
| 4 | <a href="#">Buying, Selling &amp; VAT</a> | accounts, purchase, sales |
| 5 | <a href="#">CBMS / IRD e-Billing</a>      | accounts                  |
| 6 | <a href="#">Document Generator</a>        | accounts/admin            |
| 7 | <a href="#">Reports</a>                   | all report users          |
| 8 | <a href="#">Administrator Setup</a>       | System Managers           |

## Other documents in this folder

Feature/change docs kept for depth and history (the technical chapters link to them where relevant):

- `Documentation of avinash group.md` — earlier developer overview (reports + customizations); still a good quick reference, superseded where it conflicts
- `stock_ledger_guide.md` — how ERPNext's stock ledger engine works (context for Stock Adjustment / revaluation)
- `sales_stock_ledger_fixes.md` — Sales Stock Ledger fix history
- `vehicle_expense_report.md` — Vehicle Expense report spec & status
- `user_daily_entry_summary.md` — full feature doc for the daily entry summary
- `customer_supplier_item_duplicate_checks_and_default_accounts.md` — the master-form client helpers
- `product wise invoice customer portal.md` — the customer portal feature doc
- `migrate_from_biometric_and_nepal_hrms.md` — historical app-consolidation runbook (endpoints therein are legacy)
- `db-master-slave-replication.md` — database replication ops runbook
- `dynamic_approval_guide.md`, `dynamic_approval_workflow.md`, `dynamic_workflow.md` — **stale** in places; read [technical/02-dynamic-approval.md](#) §9 before trusting them

## Bridge (separate component)

- `../k40_bridge/README.md` and `../k40_bridge/SETUP.md` — the Windows K40 Bridge install & configuration runbooks.

# Architecture Overview — avinashgroup\_app

Technical documentation, chapter 1 of 9. Audience: developers.  
The app runs on site **avinash1** (Frappe/ERPNext v15 bench at `/home/dell/frappe-v15`).

`avinashgroup_app` is the single custom app for the Avinash Group of companies (Nepal — LP gas trading, dealerships, and allied businesses). It consolidates what were previously three apps: the original `avinashgroup` customizations, plus `biometric_integration` and `nepal_hrms`, which were merged in (2026-05) and uninstalled — see `docs/migrate_from_biometric_and_nepal_hrms.md`.

## 1. Repository layout

```
apps/avinashgroup_app/
├── avinashgroup_app/
│   ├── hooks.py
│   ├── modules.txt
│   ├── patches.txt
│   └── avinash_group_app/
│       ├── doctype/
│       ├── report/
│       ├── page/document_generator/
│       ├── custom/
│       ├── custom_code/
│       │   ├── common/
│       │   ├── SalesInvoice/
│       │   ├── purchase_invoice/
│       │   ├── payment_entry/
│       │   ├── Override/
│       │   ├── CBMS/
│       │   ├── globalfilter/
│       │   ├── document_generator/
│       │   ├── dynamic_approval.py
│       │   ├── fiscal_year_filter.py
│       │   ├── excise_ledger.py
│       │   └── ... (misc – see chapter 4 & 6)
│       ├── biometric/
│       ├── payroll/
│       ├── utils/
│       ├── scripts/
│       ├── patches/
│       ├── fixtures/
│       ├── public/js, public/css
│       └── templates/pages/
├── k40_bridge/
└── docs/
```

```
# the Python package
# ALL wiring – start here for any feature
# single module: "Avinash Group App"
# migration patches (see §7)
# the Frappe module
# 26 custom doctypes (see chapter 9)
# 26 script reports (see chapter 8)
# Document Generator desk page
# exported custom-field JSON for core doctypes
# server-side business logic, grouped by area
# purchase/selling VAT+TDS handlers & mapper
# SI taxes, credit control
# PI TDS engine
# cheque bounce
# ERPNext class overrides, naming engine
# Nepal IRD CBMS e-billing sync
# company-scoped link filtering
# letter/PDF generation backend
# dynamic approval workflow engine
# per-user fiscal-year access control
# excise GL rewriting on PI
# ZKTeco iclock protocol + attendance pipeline
# attendance allowance
# audit_file_manager, fiscal_year_utils, ...
# one-off maintenance scripts
# migration patch implementations
# Document Template fixtures
# ~24 client scripts + desk CSS
# customer/supplier portal pages
# Windows desktop bridge for ZKTeco K40 devices
# this documentation
```

**One Frappe module** — everything lives under module “Avinash Group App” (`modules.txt`).

## 2. The eight functional domains

| # | Domain                        | Entry points                                                                                               | Chapter    |
|---|-------------------------------|------------------------------------------------------------------------------------------------------------|------------|
| 1 | Dynamic Approval System       | <code>custom_code/dynamic_approval.py</code> , 4 doctypes, <code>approval_workflow_*.js</code>             | 02         |
| 2 | Biometric attendance          | <code>biometric/</code> , <code>k40_bridge/</code> , 6 doctypes                                            | 03         |
| 3 | VAT / TDS / doctype overrides | <code>custom_code/common/</code> , <code>SalesInvoice/</code> , <code>Override/</code>                     | 04         |
| 4 | CBMS (IRD e-billing)          | <code>custom_code/CBMS/</code> , 3 doctypes                                                                | 05         |
| 5 | Access control & audit        | <code>fiscal_year_filter.py</code> , <code>globalfilter/</code> , <code>utils/audit_file_manager.py</code> | 06         |
| 6 | Document Generator            | <code>custom_code/document_generator/</code> , desk page, 5 doctypes                                       | 07         |
| 7 | Reports suite                 | <code>avinash_group_app/report/</code> (26 reports)                                                        | 08         |
| 8 | Portal pages                  | <code>templates/pages/</code>                                                                              | 08 §Portal |

## 3. hooks.py — the wiring map

`hooks.py` is deliberately the single source of truth for how features attach to Frappe. It uses a helper `_add_doc_event(doctype, event, handler)` to merge per-feature event dicts into one `doc_events` structure without clobbering earlier registrations (`hooks.py:133-151`).

### 3.1 Base doc\_events come from the audit engine

```
doc_events = AuditEventManager.get_doc_events() # hooks.py:131
```

`AuditEventManager` (`utils/audit_file_manager.py`) returns handlers that stamp audit fields (`custom_created_by/on`, `custom_modified_by`, `custom_company`, `custom_naming_series` / `custom_name` document numbering) on ~80 doctypes. Every other feature is merged on top via `_add_doc_event`.

### 3.2 Per-doctype server events (merged on top)

| Doctype                                                | Events → handlers                                                                                                                                                                                                    |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purchase Invoice                                       | before_validate/before_save/validate → purchase taxes; validate → vehicle-mandatory; before_submit → excise_ledger.modify_gl_entries; on_submit → stock_revaluation                                                  |
| Purchase Order / Purchase Receipt / Supplier Quotation | before_save + validate → purchase taxes handler                                                                                                                                                                      |
| Material Request / Request for Quotation               | validate → purchase taxes handler (warehouse defaults)                                                                                                                                                               |
| Sales Invoice                                          | before_validate/before_save/validate → selling taxes; on_submit + before_cancel → CBMS sync; before_print → IRD print-copy counter (chapter 4 §5a)                                                                   |
| Quotation / Sales Order / Delivery Note                | before_validate/before_save → selling taxes handler; validate → salesinvoice_taxes validators                                                                                                                        |
| Journal Entry                                          | validate → vehicle-mandatory                                                                                                                                                                                         |
| Attendance                                             | validate → holiday flag, shift deviation, late-arrival half-day (uses validate, not before_save, because device-marked attendance is inserted already-submitted and before_save would never fire — hooks.py:118-129) |
| Employee Checkin                                       | after_insert → reconcile with existing Attendance                                                                                                                                                                    |
| Employee                                               | validate → unique biometric device-ID check                                                                                                                                                                          |
| Company Filter Config / Field                          | on_update / on_trash → clear filter cache                                                                                                                                                                            |
| User                                                   | on_update → clear fiscal-year permission cache                                                                                                                                                                       |
| * (all doctypes)                                       | validate, before_save, on_update, before_workflow_action → dynamic_approval engine                                                                                                                                   |

3.3 Class overrides ( override\_doctype\_class , hooks.py:230)

Nine transactional doctypes get subclassed controllers from custom\_code/Override/overrides.py : Material Request, Purchase Order, Sales Invoice, Sales Order, Delivery Note, Purchase Invoice, Purchase Receipt, Supplier Quotation, Request for Quotation. See chapter 4.

3.4 Whitelisted-method overrides ( override\_whitelisted\_methods , hooks.py:258)

| Core method                                            | Replaced by                          | Why                                               |
|--------------------------------------------------------|--------------------------------------|---------------------------------------------------|
| ...request_for_quotation.create_supplier_quotation     | templates/pages/rfq.py               | supplier portal RFQ with VAT/discount/attachments |
| erpnext.stock.get_item_details.get_item_details        | Override/get_item_details.py         | branch-wise warehouse resolution                  |
| frappe.model.workflow.get_transitions / apply_workflow | workflow_admin_bypass.py             | System Manager bypass of workflow restrictions    |
| frappe.client.get_list                                 | fiscal_year_filter.filtered_get_list | fiscal-year access control on API list calls      |
| frappe.desk.query_report.get_data_for_custom_field     | Override/query_report.py             | report “Add Column” shows Link titles not ids     |

3.5 Other hooks

- page\_renderer = biometric.iclock.IclockRenderer ( hooks.py:11 ) — implements the ZKTeco **iclock/ADMS push protocol** as raw URL endpoints ( /iclock/... ).
- before\_request → auto\_insert\_item\_price.patch\_insert\_item\_price\_set\_company — monkey-patches ERPNext so auto-created Item Prices get company stamped.
- scheduler\_events ( hooks.py:242 ): hourly biometric-bridge heartbeat check; every 5 min CBMS retry + reconciliation.
- app\_include\_js — 14 globally loaded scripts (fiscal-year cache, approval UI, company filter, VAT calculators, document numbering, report print orientation...).
- app\_include\_css — desk\_focus.css only.
- doctype\_js — per-form scripts for Material Request, Purchase Invoice, Journal Entry, Attendance, Payroll Entry, Customer, Supplier, Item.
- permission\_query\_conditions + has\_permission — generated per doctype in FILTERED\_DOCTYPES for fiscal-year access control (chapter 6).
- fixtures — exports the two balance-confirmation **Document Template** records.

4. Shared conventions

- **Company scoping.** Most masters carry custom\_company ; transactions use the standard company . The Company Filter system (chapter 6) restricts link dropdowns and blocks cross-company saves. Report filter dropdowns come from whitelisted get\_company\_\* helpers.
- **Nepali (BS) dates.** Transactions carry Miti custom fields ( custom\_invoice\_miti , custom\_nepali\_miti , custom\_posting\_miti ); attendance and payroll reports work in Bikram Sambat months.
- **Custom VAT scheme.** Line-level custom\_vat\_apply\_on ∈ { VAT 13% , VAT 0% , Amount }, plus excise and a custom TDS field set — chapter 4.
- **Numbers.** Indian lakh/crore grouping ( en\_IN ), currency NPR.
- **Script reports.** All synchronous script reports; heavy queries written to stay under Frappe's 15 s prepared-report auto-switch.
- **Console-applied setup.** Several features are installed per-site from the bench console rather than by migrate (workflow install, audit-field creation, company lock). Each feature's chapter has its Setup block. Run via bench --site avinas1 console or bench --site avinas1 execute <dotted.path> .

5. Client-asset versioning

Every entry in app\_include\_js carries a ?v= query ( hooks.py:13-28 ). **Bump the version when you edit a script** — this is the cache-buster; without it desk users keep the stale copy until a hard refresh.

6. Fixtures

`fixtures/document_template.json` exports exactly two Document Template records — “Customer Balance Confirmation” and “Vendor Balance Confirmation” — filtered by name in `hooks.py:274-281` so site-specific templates are not dragged along.

## 7. Patches (`patches.txt`)

---

Pre-model-sync:

- `create_document_generator_roles` — creates the Document Generator roles
- `rename_vocher_number_settings` — renames the voucher-number settings doctype

Post-model-sync (in order): `setup_attendance_allowance`,  
`v1 add biometric_device_serial`, `user_fiscal_year_access_fields`,  
`migrate_fiscal_year_access_control`, `cleanup_document_generator_sections`,  
`create_document_generator_image_fields`,  
`create_document_generator_signatory_field`, `leave_application_dynamic_approval`,  
`enable_track_changes_audited_doctypes`, `add_creation_index_daily_entry`,  
`create_cbms_custom_fields`, `add_gl_entry_fin_stmt_agg_index`,  
`vehicle_mandatory_property_setters` (run twice — second run removes the property  
setters in favour of per-row JS validation), `add_company_to_employee_checkin`,  
`company_scoped_attendance_device_id`, `add_company_to_shift_type`,  
`add_sales_invoice_print_count` (adds `custom_print_count` for IRD copy  
labeling).

(Per-patch details are in each feature's chapter.)

## 8. Development workflow

---

- Default branch: `develop`. Pre-commit hooks: `ruff`, `eslint`, `prettier`, `pyupgrade`.
- After changing Python: `bench restart (live) / auto-reload (dev)`.
- After changing client JS: `bump ?v=` in `hooks.py`, `bench build --app avinashgroup_app` if assets are imported, then hard-refresh.
- Migrations: `bench --site avinas1 migrate`.
- Tests: `bench --site avinas1 run-tests --app avinashgroup_app --module avinashgroup_app.biometric.test_attendance_pipeline` (the attendance pipeline suite is currently the main automated test).

## 9. Operational notes

---

- The `avinas1` dev server masks real errors: any server error can render as a 500 `BrokenPipeError` page. Check `bench --site avinas1 console / logs` instead of trusting the HTTP response.
- Database replication runbook: `docs/db-master-slave-replication.md`.

# Dynamic Approval System — Technical Reference

Chapter 2 of the technical documentation. Audience: developers.  
User-facing guide: [../user\\_guide/02-approvals.md](#)

⚠ This chapter documents the **shipping code** (verified 2026-07-05). The older docs `dynamic_approval_guide.md`, `dynamic_approval_workflow.md` and `dynamic_workflow.md` in this folder have diverged from the code in places — see §9 for the divergence list. Where they conflict, **this chapter wins**.

## 1. Purpose

A configurable, multi-level **sequential** approval workflow attachable to **any** doctype without editing `hooks.py` or writing per-doctype code. A Dynamic Approval Setting (per doctype + company) plus its **Setup Workflow** button injects hidden driver fields, a visible "Approval Hierarchy" child table, an audit-log table, and a 4-state Frappe Workflow onto the target doctype.

Approvers are a **union**: the requester's own per-document hierarchy rows (Levels 1..N) followed by **fixed approvers** from the Setting (Levels N+1..N+M). The document stays in Pending Approval through all intermediate approvals (no ping-pong back to Draft), flipping to Approved / `docstatus=1` only at the final level.

## 2. Wiring

`hooks.py:225-228` registers on `"**"` (all doctypes):

| Event                               | Handler                                                |
|-------------------------------------|--------------------------------------------------------|
| <code>validate</code>               | <code>custom_code/dynamic_approval.py::validate</code> |
| <code>before_save</code>            | <code>::before_save</code>                             |
| <code>on_update</code>              | <code>::on_update</code>                               |
| <code>before_workflow_action</code> | <code>::before_workflow_action</code>                  |

`hooks.py:261-262` overrides `frappe.model.workflow.get_transitions / apply_workflow` with `custom_code/workflow_admin_bypass.py`.

The universal hooks are cheap: `_is_managed_doctype(doc)` (`dynamic_approval.py:25-33`) short-circuits with a DB-free `doc.meta.has_field("custom_current_approver")` check.

## 3. Data model

### 3.1 Dynamic Approval Setting (parent, System Manager only)

Autoname `{document_type}-{company_abbr}-{hash6}`; title field: `document_type`; `track_changes`.

| Field                                 | Type                                                             | Meaning                                               |
|---------------------------------------|------------------------------------------------------------------|-------------------------------------------------------|
| <code>document_type</code>            | Link → DocType, reqd                                             | target doctype                                        |
| <code>company</code>                  | Link → Company, reqd                                             | scope                                                 |
| <code>company_abbr</code>             | Data, RO (fetch)                                                 | used in autoname                                      |
| <code>is_active</code>                | Check (1)                                                        | criteria scan filters on this                         |
| <code>approver_table_fieldname</code> | Data, reqd (default <code>custom_approval_approvers</code> )     | fieldname of the hierarchy table on the target        |
| <code>current_level_fieldname</code>  | Data, reqd (default <code>custom_current_approval_level</code> ) | fieldname of the level tracker                        |
| <code>approver_fieldname</code>       | Data                                                             | <b>dead field</b> — defined but referenced by no code |
| <code>dept_config_html</code>         | HTML                                                             | canvas for the section-card UI                        |
| <code>match_criteria</code>           | Table → Dynamic Approval Match Criteria, hidden                  | rule rows (managed via dialog)                        |
| <code>approvers</code>                | Table → Dynamic Approval Fixed Approver, hidden                  | fixed approvers (managed via dialog)                  |
| <code>setup_workflow_html</code>      | HTML                                                             | help text for the Setup Workflow button               |

Controller `validate` only checks criteria rows are complete.

### 3.2 Child tables

- **Dynamic Approval Match Criteria:** `section` (Data, reqd — groups rows into one rule), `field_name` (Autocomplete, reqd), `field_value` (Autocomplete, reqd). Equality-only, string compare, trimmed.
- **Dynamic Approval Fixed Approver:** `section`, `approver` (Link → User), `approver_name` (fetch). Order = row `idx`; levels are derived by appending after the user hierarchy.
- **Dynamic Approval Approver** (injected on targets as `custom_approval_approvers`): `level` (Int — rewritten to `idx` to stay gap-free), `approver` (Link → User), `approver_name` (fetch).
- **Dynamic Approval History** — created **at runtime** by `_ensure_history_doctype` (`dynamic_approval.py:541-558`), no source JSON: `action`, `user`, `user_name`, `timestamp`. Injected as `custom_approval_history`.

### 3.3 Custom fields injected on the target by Setup Workflow

`_ensure_custom_fields` ( `dynamic_approval.py:561-706` ), idempotent (repeat runs reconcile drift):

1. `custom_approval_setting` (Link, hidden) — pinned Setting for fast lookup
2. `custom_approval_section` (Data, hidden) — pinned section
3. `custom_current_approval_level` (Int, hidden)
4. `custom_current_approver` (Data, hidden) — whose turn it is; drives workflow conditions
5. `custom_total_approval_levels` (Int, hidden)
6. `custom_section_approval_hierarchy` (Section Break, visible)
7. `custom_approval_approvers` (Table, visible, `allow_on_submit`)
8. `custom_section_approval_history` (Section Break, visible)
9. `custom_approval_history` (Table, visible, `allow_on_submit`)

## 4. Approver resolution

### 4.1 Config lookup — `_get_config_for_doc` ( `dynamic_approval.py:54-145` )

- **Fast path** ( `:61-65` ): if `custom_approval_setting` + `custom_approval_section` are pinned on the doc → `_fetch_config_by_section` (2 queries, no scan).
- **Slow path**: requires `doc.company`. Loads all active Settings for (`document_type`, `company`), bulk-loads criteria + fixed approvers in two queries, groups by (`setting`, `section`).
- **Scoring** ( `:103-143` ): a section matches when **all** its criteria rows match ( `str(doc.get(field)).strip() == str(value).strip()` ). Score = criteria count; **highest score wins** (most specific). The winner is pinned onto the doc. No match anywhere → `None`.
- $\Delta$  A section with **zero criteria is skipped** ( `:118-119` ) — an empty section is *not* a catch-all, despite the docstring at `:110` claiming otherwise. To make a catch-all rule, give it one criterion that always matches (e.g. `company = <the company>`).
- **No caching** — deliberate ( `:49-51` ). The pin is the optimization. There is no Redis cache and no `clear_config_cache`.

### 4.2 Level math

- `_get_total_levels` ( `:178-185` ) = user hierarchy rows + fixed approvers.
- `_get_effective_approver_at_level` ( `:195-211` ): levels  $\leq$  user-count come from the document's hierarchy table (positional fallback if `level` unset); above that, `fixed_approvers[level - user_count - 1]`.
- There are **no amount thresholds and no reports-to/org-chart resolution** — matching is doctype+company scoping, equality criteria, and a fixed sequence.
- Degenerate cases: no config at submit → **auto-approve** (`docstatus 1`, log "Auto-approved (no matching approval rule)", `:343-353`); config with zero levels → single `Administrator` approver fallback ( `:360-363` ).

## 5. Runtime flow

**Draft save** — `before_save` logs "Created" on first save. `validate` returns early (state  $\neq$  Pending Approval).

**Submit for Approval** — single Draft→Pending transition, empty condition (always shown). `before_workflow_action` ( `:308-402` ) resolves config, computes total, sets `level=1 / total / custom_current_approver`, sets `flags.approval_level_changed`, logs "Submitted for Approval". `on_update` ( `:405-459` ) re-syncs level fields via direct DB writes ( `update_modified=False` ) and emails the Level-1 approver.

**Intermediate Approve** — transition condition `approve_more` (current user is `custom_current_approver` or `Administrator`, and `level < total`). The Approve branch ( `:372-398` ) recomputes total, increments level, sets the next approver, forces `workflow_state="Pending Approval"` + `docstatus=0`, logs "Approved (Level N)". `on_update` emails the next approver.

**Final Approve** — condition `approve_final` ( `level == total` ). Sets `Approved`, `docstatus=1`, logs "Approved (Final)".

**Reject** — condition `can_reject` (current approver or `Administrator`). Sets `flags.is_rejection`, logs "Rejected"; state → `Rejected` (`docstatus 0`, editable). **There is no Resubmit transition** — from `Rejected` the user edits and the workflow restarts from Draft semantics.

### Generated workflow ( `_create_or_update_workflow`, `:709-852` )

4 states — Draft, Pending Approval, Approved (`docstatus 1`), Rejected (`docstatus 0`), all `allow_edit: All`. 4 transitions — Submit for Approval (unconditional), Approve $\times 2$  ( `approve_more / approve_final` ), Reject. Conditions are baked `doc.*` field-comparison strings (no `frappe.get_attr` calls), each OR-ing `frappe.session.user == "Administrator"`. Transition roles are derived from the target's DocPerm roles (excluding All/Guest; fallback System Manager). Idempotent — reuses/rebuilds an existing active workflow, else inserts "{DocType} Approval Workflow". Missing Workflow State / Action Master records are auto-created.

### Status bridge

If the target has a `Select status` field containing both "Approved" and "Rejected" (e.g. HRMS Leave Application), the workflow states also set `status (:758-770)`. Patch `leave_application_dynamic_approval.py` hides the native `leave_approver` fields so Dynamic Approval owns leave approval.

6. Guards in `validate (:262-299)`

- While Pending, a save by anyone who is not the current approver throws Only {approver} may make changes. The approver is read **from DB**, not `doc.get` — `before_workflow_action` has already overwritten the in-memory field to the next approver. Exemptions: the transition entering Pending (`has_value_changed("workflow_state")`) and intermediate approves (`flags.approval_level_changed`).
- Rows of levels already approved (`level < current level`) cannot be modified or deleted — DB rows are compared to in-memory rows.
- `on_update` rewrites user rows' `level = idx+1 (row.db_update())`, recomputes the approver, clamps `current_level` to `total` if a deletion made it overshoot (`:445-447`).

7. Whitelisted API + client scripts

| Method                                                                          | Caller                                           | Purpose                                                                             |
|---------------------------------------------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>dynamic_approval.has_approval_config(doctype, company)</code>             | <code>approval_field_visibility.js:95-104</code> | toggle driver-field visibility                                                      |
| <code>dynamic_approval.setup_workflow(config_name)</code> (System Manager only) | <code>dynamic_approval_setting.js:21-24</code>   | the Setup Workflow button                                                           |
| <code>dynamic_approval.is_current_approver(doctype, docname)</code>             | <i>(exported, currently unused by JS)</i>        | server-side check                                                                   |
| <code>workflow.set_reject_reason(doctype, name, reason)</code>                  | <code>approval_workflow_auto.js:26</code>        | store reject reason → <code>custom_reason</code> field if it exists, else a Comment |
| <code>workflow_admin_bypass.get_transitions / apply_workflow</code>             | Frappe core (overridden)                         | see §8                                                                              |

Client scripts (all in `app_include.js`, order matters — common first):

- `approval_workflow_common.js` — defines `window.avinashgroup.app.approval_workflow`: hides Approve/Reject for non-current approvers (`:62-103`); the mandatory "Reason for Rejection" dialog (`:105-189`) which calls `set_reject_reason` and only then lets the Reject transition proceed.
- `approval_workflow_auto.js` — generic `form-refresh` binding for **any** doctype that has the approval fields: progress banner via `frm.set_intro (✓ Level 1 → ✗ Level 2 (current) → ○ Level 3, plus "Your approval is required..." / "Waiting for ...")`, and lazily binds the reject dialog once per doctype unless the doctype ships its own handler.
- `approval_field_visibility.js` — filters the Setting picker; while Pending and the session user is not the current approver → `frm.disable_form()`; visually locks already-approved hierarchy rows (opacity, no delete button); shows/hides the hidden driver fields per `has_approval_config`.

No per-doctype approval JS exists any more (`hooks.py:38-40`).

8. Administrator bypass (`workflow_admin_bypass.py`)

`is_admin_bypass (:13-25)` — true only for the literal user **Administrator** AND workflow name in {"Material Request One-Line Approver", "Purchase Order Workflow"} or ending in "Approval Workflow".

- `get_transitions (:27-64)`: for admin-bypass cases, returns all condition-satisfied transitions, skipping the **role** check (conditions still evaluated).
- `apply_workflow (:67-152)`: fires `before_workflow_action` server-side for **all** users (stock Frappe never does), still enforces `has_approval_access` (self-approval block), then applies the state change / save / submit / cancel.

Note: only `Administrator` is bypassed. There is **no** generic System Manager override; Administrator still advances one level at a time.

9. Divergence of the older docs (do not trust these points)

- `dynamic_approval_guide.md` (2026-04-15): describes a department-based model with functions that no longer exist (`wf_can_approve*`, `clear_config_cache...`), `frappe.get_attr` conditions, Memo roles, a Resubmit flow, and only 2 injected fields. **Fully superseded.**
- `dynamic_approval_workflow.md`: development log; concepts current but uses the old `group (Int)` naming — the code renamed it to `section (Data)`.
- `dynamic_workflow.md` (2026-04-24): best narrative for data model/hooks, but its workflow sections are aspirational: it documents 9 transitions with System-Manager parallels and Resubmit, a `tf==0` auto-approve transition, and a Draft-time `validate precompute` — none of which the code implements (4 transitions; auto-approve is imperative inside `before_workflow_action`).

10. Email notifications

`_send_approval_notification (:466-517)` fires only when

`flags.approval_level_changed` and state is Pending; distinct subject/body for rejection vs approval; `frappe.sendmail(now=False)` in try/except so mail failures never abort a save.

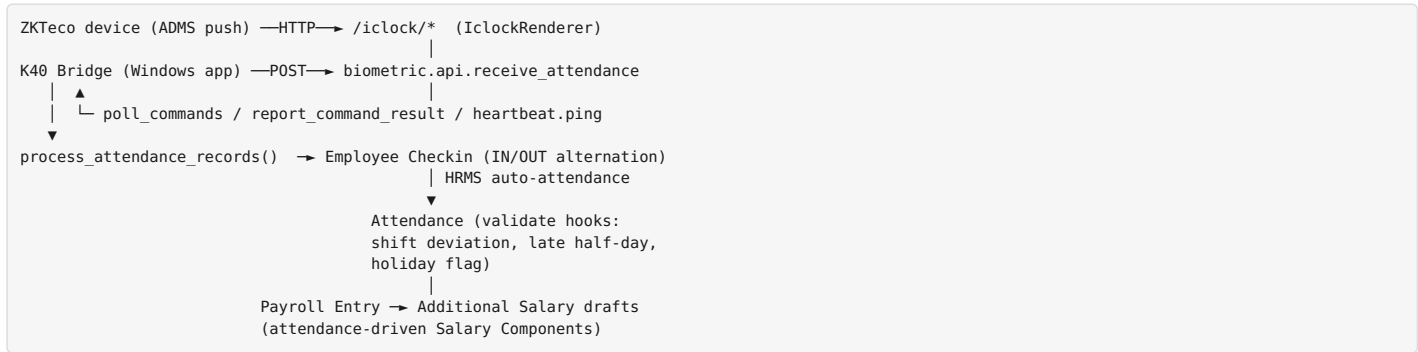
# Biometric Attendance, K40 Bridge & Attendance Payroll — Technical Reference

Chapter 3 of the technical documentation. Audience: developers.

User-facing guide: [../user\\_guide/03-attendance-hr.md](#)

Bridge install runbook: [k40\\_bridge/SETUP.md](#). Legacy migration: [docs/migrate\\_from\\_biometric\\_and\\_nepal\\_hrms.md](#) (historical — its endpoints refer to the old apps).

## 1. Architecture at a glance



Two ingestion paths converge on one core pipeline; everything is **company-scoped through the device serial**.

## 2. Ingestion

### 2.1 ADMS push — `biometric/iclock.py`

`IclockRenderer(BaseRenderer)` registered as a `page_renderer (hooks.py:11)`; claims any `/iclock/*` path. Implements the ZKTeco ADMS subset:  
GET `/iclock/cdata?options=all` returns the config block (`ATTLOGStamp=9999`, `Realtime=1`, ...); POST `/iclock/cdata?...&table=ATTLOG` parses tab-separated punch rows (`_process_attlog`, `iclock.py:129-169`); `getrequest / ping / devicecmd / fdata` just ack OK. Always returns plain-text 200 even on error (prevents device retry-storms); failures go to Error Log and `logs/iclock.log`. `assert_known_device(sn)` gates every upload.

### 2.2 Bridge push — `biometric/api.py`

`receive_attendance(attendance_data, device_identifier)` — `@frappe.whitelist(methods=["POST"])`. 403 for unknown/disabled serials; validates `user_id` + `timestamp` per record; hands to the core.

### 2.3 Core pipeline — `biometric/utils.py::process_attendance_records ( :42-195 )`

- Resolves the **device's company** from `Biometric Device.company` by serial — the linchpin of multi-company support.
- Groups punches by `(user_id, date)`; malformed timestamps → permanent `skipped_punches`. Parser accepts `%Y-%m-%d %H:%M:%S`, ISO variants.
- Employee lookup is **company-scoped**: `{attendance_device_id, company}`. No match → group `skipped` (retryable once the employee is registered).
- Each `(employee, date)` group runs in a DB savepoint.
- Returns per-punch outcome lists (`synced_punches / failed_punches / skipped_punches`, ids `"{user_id}_{timestamp}"`) — the bridge uses these to decide what to dedup vs resend.
- Updates `Biometric Device.last_sync_time / total_synced` on success.

### 2.4 Checkin reconciliation — `_reconcile_day_checkins ( utils.py:198-244 )`

Every punch = one Employee Checkin. Merges existing + new for the day, dedups by exact timestamp, sorts chronologically, assigns **alternating log types** (index 0 → IN, 1 → OUT, ...). Idempotent; a late-arriving earlier punch slots in by time and downstream rows flip IN↔OUT. New checkins get `custom_company = employee.company`, `device_id = serial`, `skip_auto_attendance = 0`.

## 3. Attendance hooks

All Attendance hooks run on **validate** — auto-attendance inserts rows already submitted, so `before_save` would never fire (`hooks.py:118-129`).

| Hook                                                               | File                                    | Behavior                                                                                                                                                                                    |
|--------------------------------------------------------------------|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| set_shift_deviation_fields                                         | biometric/attendance_override.py:15-68  | computes custom_late_entry / custom_early_entry / custom_early_exit / custom_late_exit (seconds, rounded to nearest minute, ≥30 s rounds up) vs Shift Type start/end; overnight-shift aware |
| enforce_late_arrival_half_day                                      | :117-148                                | if Shift Type custom_late_arrival_cutoff_time is set and first check-in is later → force status Half Day + leave_type "Leave Without Pay". Blank cutoff disables                            |
| set_holiday_flag                                                   | payroll/attendance_allowance.py:207-229 | sets read-only custom_worked_on_holiday when the date is in the employee's (or company default) Holiday List                                                                                |
| reconcile_with_existing_attendance (Employee Checkin after_insert) | attendance_override.py:71-114           | past-date orphan punches only: link checkin to an existing Present/Half Day Attendance; never creates/changes status (that's Attendance Fix's job)                                          |
| validate_unique_device_id (Employee validate)                      | biometric/employee.py                   | attendance_device_id unique per company (patch company_scoped_attendance_device_id dropped the stock global-unique index)                                                                   |

Test suite: biometric/test\_attendance\_pipeline.py — company-scoped mapping, IN/OUT alternation + idempotency + punch reflow, duplicate device-id block, deviation math, late-arrival rule, and end-to-end auto-attendance  
( bench --site avinas1 run-tests --app avinashgroup\_app --module avinashgroup\_app.biometric.test\_attendance\_pipeline ).

## 4. The K40 Bridge ( k40\_bridge/ , Windows desktop app )

Python/Tkinter, PyInstaller + Inno Setup → K40BridgeSetup.exe .  
VERSION = "1.6.6" ( k40\_bridge.py:48 , matches latest\_version.txt ).

### 4.1 Server endpoints it uses ( k40\_bridge.py:50-65 )

| Purpose               | Endpoint                                                         |
|-----------------------|------------------------------------------------------------------|
| push punches          | avinashgroup_app.biometric.api.receive_attendance                |
| liveness              | avinashgroup_app.biometric.heartbeat.ping (every cycle)          |
| pull admin commands   | avinashgroup_app.biometric.bridge_commands.poll_commands         |
| report command result | avinashgroup_app.biometric.bridge_commands.report_command_result |
| auth test             | frappe.auth.get_logged_user                                      |

Auth: Authorization: token KEY:SECRET ( ErpNextClient , k40\_bridge.py:1481-1655 ). Credentials Fernet-encrypted at rest.

### 4.2 Device sources

- ZKTecoClient — ZK TCP protocol port 4370 (K40/K20/F18/MB360/eSSL) via pyzk
- HikvisionClient — ISAPI HTTP
- HtmsClient — reads HTMS-86/HAMS Microsoft Access .mdb directly ( PubEvent.personID → Emp.Emp\_no as user\_id ); one HTMS source collapses all controllers into one feed

Each configured device row: name, type, ip/port or db\_folder, **serial**, **company**.

### 4.3 Sync engine ( SyncEngine , k40\_bridge.py:1656-2100 )

Two daemon threads (scheduled sync loop + 30 s command poller) serialized by a lock. Per device: heartbeat first (so a quiet device still looks alive), pull punches from persisted last-sync date (first run = everything, no upper bound), filter against a local SQLite DedupStore keyed {serial}\_{user\_id}\_{ts} , POST in batches of 100. Only server-confirmed synced punches get dedup-marked — skipped ones (employee not yet registered) resend next cycle. Outcomes also logged in a local SyncLogStore . Default interval 1440 min (configurable down to 2 min).

### 4.4 Command tunnel ( biometric/bridge\_commands.py )

Outbound-pollled so admins can reach devices behind NAT/LAN:

1. Desk enqueues a **Biometric Device Command** (Pending).
2. Bridge poll\_commands(serial) atomically claims Pending→Running (raw SQL + ROW\_COUNT() race guard, increments attempts).
3. Bridge executes: force\_sync (optional from\_date payload) or test\_connection .
4. report\_command\_result → Done/Failed (serial-ownership verified, idempotent).

enqueue\_command(device, command\_type, payload) is the desk helper (used by the "Force Bridge Sync" button on Biometric Device, which then polls the command row every 3 s for up to 90 s).

### 4.5 Heartbeat alerting ( biometric/heartbeat.py )

ping(serial) updates last\_contact\_time . Hourly scheduler check\_bridge\_heartbeats ( hooks.py:242-245 ): per enabled device with alert recipients, compares last\_contact\_time (fallback last\_sync\_time ) to alert\_threshold\_minutes (default 120). Emails **only on state transitions** (Connected→Disconnected and back), state persisted in connection\_status — a permanently dead bridge emails once, not hourly. The recovery email tells HR to run Attendance Fix for the outage range.

## 5. Doctypes

| Doctype                                                                                   | Purpose / key fields                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Biometric Device</b> (autoname field:device_name)                                      | one row per device/feed: device_serial (unique — matched against SN), company (reqd), enabled, ip/port/model, RO sync stats (last_contact_time, last_sync_time, total_synced), alert_threshold_minutes + alert_recipients table, connection_status . JS adds Force Bridge Sync button |
| <b>Biometric Device Command</b> ( BDC- .##### )                                           | device, command_type (force_sync/test_connection), status Pending/Running/Done/Failed, attempts ("Bridge Poll Count"), payload JSON, result                                                                                                                                           |
| <b>Biometric Device Alert Recipient</b> (child)                                           | email, recipient_name                                                                                                                                                                                                                                                                 |
| <b>Attendance Fix</b> (submittable, AF- .YYYY. - )                                        | repair tool: shift_type (reqd), from_date / to_date (reqd), optional employee / company / devices ; RO progress + counters + log                                                                                                                                                      |
| <b>Attendance Fix Device</b> (child)                                                      | device link                                                                                                                                                                                                                                                                           |
| <b>Employee Attendance Allowance</b> (child on Employee as custom_attendance_allowances ) | salary_component, eligible (default 1), rate (overrides component default), effective_from                                                                                                                                                                                            |

Attendance Fix internals ( attendance\_fix.py )

on\_submit → status Queued → background job on the long queue (4 h timeout), runs as Administrator with frappe.flags.audit\_user = doc.owner .  
\_resolve\_employees : shift-assigned employees filtered by employee/company/device-serial checkins, Active only. \_reconcile\_day ( :190-332 ) matrix:

| Checkins? | Existing Attendance | Action                          |
|-----------|---------------------|---------------------------------|
| yes       | none                | create from checkins & link     |
| yes       | Absent              | delete stale Absent, recreate   |
| yes       | Present/Half Day    | relink orphan checkins          |
| no        | none                | mark Absent (skipping holidays) |
| no        | any                 | leave alone                     |

Reuses stock HRMS mark\_attendance, mark\_attendance\_and\_link\_log, update\_attendance\_in\_checkins; calls fetch\_shift() for shiftless checkins.  
Per-day savepoints; realtime progress via attendance\_fix\_progress (the form shows a live progress bar and auto-reloads while Running).

6. Attendance allowance engine ( payroll/attendance\_allowance.py )

Turns Attendance rows + rates into **Additional Salary** drafts.

Custom fields (patch setup\_attendance\_allowance): Salary Component — custom\_is\_attendance\_driven, custom\_condition\_type (Working Hours ≥ Threshold / Status = Present / Status = Half Day / Worked on Holiday / Early Entry Before / Late Stay After / Late Arrival After), custom\_threshold\_hours, custom\_time\_offset\_hours, custom\_unit (Per Day / Per Hour / Flat per Period), custom\_default\_rate, custom\_summary\_group; Employee — custom\_ot\_eligibility, custom\_attendance\_allowances; Additional Salary — custom\_source.

Flow: trigger\_for\_payroll\_entry(pe) (whitelisted; the **Calculate Attendance Allowances** button in payroll\_entry.js) → for each employee × attendance-driven component: skip OT-only conditions for non-OT-eligible employees; resolve rate (employee override row → component default → skip); qty = Σ evaluate\_rule(attendance) over submitted rows in the period ("Flat per Period" → 1 if any match); delete prior draft tagged custom\_source = "Nepal HRMS Attendance Allowance" and recreate (idempotent). evaluate\_rule ( :150-186 ) is shared with the BS attendance reports: Present = 1.0/day (Half Day 0.5); holiday rule needs custom\_worked\_on\_holiday

- Present/Half Day; the time rules compare custom\_early\_entry / custom\_late\_exit / custom\_late\_entry seconds to custom\_time\_offset\_hours . recompute\_holiday\_flags (whitelisted) backfills the holiday flag.

public/js/attendance.js renders a read-only "Checkin Log" table (IN green / OUT orange, linked) on the Attendance form.

7. HR/BS reports (module reports; see also chapter 8)

| Report                        | One-liner                                                                                                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Monthly Attendance BS         | per-day grid for a BS month (BS+AD dates, IN/OUT, hours, late/early minutes, dynamic per-component allowance qty columns, holiday awareness)                                                       |
| Monthly Attendance Summary BS | one row/employee/BS month mirroring the Nepal Gas physical sheet; components collapsed by custom_summary_group (Meal, Tea & Conveyance...), OT hrs, leave current/previous/cumulative-from-Shrawan |
| Work On Holiday BS            | per-employee holiday-work counts bucketed into 12 BS months (Shrawan→Ashadh), OT-eligibility column                                                                                                |
| Yearly Leave Details BS       | per-employee/FY: 12 BS-month leave cells (approved Leave Applications split day-by-day, half-days 0.5), allocation total (excl. carry-forward), remaining                                          |
| Avinas Salary Statement       | Salary Slip pivot for a BS month: dynamic earning/deduction columns, gross/net, previous-month net, grouped by designation with sub-totals                                                         |

All storage is AD; BS is presentation-only via rdp\_common\_app.utils.bs\_boundaries / nepali\_datetime . Roles: HR Manager/HR User/System Manager.

8. Patches in this domain

setup\_attendance\_allowance, v1\_add\_biometric\_device\_serial, add\_company\_to\_employee\_checkin (fetch\_from company on Employee Checkin), company\_scoped\_attendance\_device\_id, add\_company\_to\_shift\_type (required

company on Shift Type — Shift Types are per-company).

# VAT / Excise / TDS, Doctype Overrides & Document Numbering — Technical Reference

Chapter 4 of the technical documentation. Audience: developers.  
User-facing guide: [../user\\_guide/04-buying-selling-vat.md](#)

This chapter covers the custom tax scheme, the nine ERPNext class overrides, the branch-wise warehouse system, the voucher-numbering engine, credit control, cheque bounce, vehicle-mandatory validation, stock adjustment, and the legacy/dormant code you should *not* build on.

## 1. The custom VAT / Excise / TDS scheme

Replaces ERPNext's Item-Tax-Template VAT with a **per-line "Apply On" selector** plus manual excise.

### 1.1 Custom fields

Item rows (PI/PO/PR/SQ items; SI/Quotation/SO/DN items):

| Field               | Behavior                                                                                                                               |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| custom_vat_apply_on | Select VAT 13% / VAT 0% / Amount . JSON default is VAT 0% but every code path forces VAT 13% when empty — the effective default is 13% |
| custom_vat_rate     | Percent, read-only (13 or 0)                                                                                                           |
| custom_vat_amount   | auto-calculated; editable only in Amount mode                                                                                          |
| custom_excise_value | always manual, never recalculated                                                                                                      |
| custom_total        | base_net_amount + custom_excise_value — the VAT base                                                                                   |
| custom_subtype      | Link → Vehicle (PI items + JE accounts; see §7)                                                                                        |
| PI-only TDS         | custom_tds_apply_on (Percentage (%) / Amount), custom_tds_rate (RO), custom_tds_amount, stock_apply_tds                                |

Document level: PI — custom\_total\_amount\_including\_excise, custom\_total\_excise\_amount, custom\_total\_vat\_amount, custom\_total\_tds\_amount, custom\_tax\_withholding\_category\_custom (a **separate** Link to Tax Withholding Category so ERPNext's native TDS engine never triggers). SI — custom\_excise, custom\_vat, excise/VAT totals.

### 1.2 Calculation rules (identical everywhere)

- VAT 13% → rate 13, vat = custom\_total × 0.13 (recalculated, RO)
- VAT 0% → rate 0, amount 0
- Amount → rate 0, manual amount kept

### 1.3 Taxes-table injection

Handlers write Actual rows into the standard taxes table, positioned: Excise (account prefix **348204**, Add, pos 0) → VAT (account prefix **VAT**, Add, pos 1) → TDS (account from the custom category's per-company Accounts row, **Deduct**, pos 2). Accounts are resolved by prefix per company ( find\_account\_by\_prefix ), then doc.calculate\_taxes\_and\_totals() folds them into totals and standard GL. Selling/SI handlers zero a stale row (rather than leaving old charges) when total becomes 0 but a tax row exists.

### 1.4 Handlers and wiring

| File                                           | Doctypes                                      | Events                                                                                                                                                               |
|------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| custom_code/common/purchase_taxes_handler.py   | PI, PO, PR, SQ (+ MR/RFQ warehouse-only)      | before_validate (return qty sign + fill missing warehouse), before_save (full recompute + tax injection), validate (validate_custom_fields + force_buying_warehouse) |
| custom_code/common/selling_taxes_handler.py    | Quotation, SO, DN                             | before_validate / before_save : the selling validate events route to salesinvoice_taxes.validate_* (warehouse forcing)                                               |
| custom_code/SalesInvoice/salesinvoice_taxes.py | Sales Invoice (+ selling validate_* wrappers) | full recompute, tax injection, force_selling_warehouse, return signs                                                                                                 |

TDS (PI only, purchase\_taxes\_handler.py:173 ): fires only when the document's custom\_tax\_withholding\_category\_custom is set **and** the row's apply\_tds is checked. Percentage (%) mode pulls the rate from the category's first rates row; Amount mode keeps the manual amount. Stale TDS rows are removed when the account/total disappears.

Return handling: is\_return PI and SI force qty and custom\_vat\_amount negative (apply\_return\_qty\_sign / apply\_return\_vat\_sign ); DN is the only selling doctype with return sign logic.

### 1.5 Cross-document mapping — common/purchase\_taxes\_mapper.py

get\_taxes\_from\_source (whitelisted) returns doc- and item-level tax values; called by purchase\_taxes\_common.js when a PO/PR/PI is created from a source doc, matching items by link field then item\_code. (The after\_mapping\_\* functions in the same file are unreferenced legacy.)

### 1.6 Client scripts

| Script                           | Load   | Role                                                                                                                                                                                                                                                                                                               |
|----------------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| purchase_taxes_common.js (v1.8)  | global | PI/PO/PR/SQ: VAT/TDS defaults, live math mirror, field-visibility toggles per mode, return signs, source-doc tax population, TDS-rate fetch, PI due-date from Supplier custom payment term_days , warehouse forcing on save; wraps frappe.call around get_item_details to inject custom_buying_warehouse race-free |
| selling_taxes_common.js          | global | Quotation/SO/DN mirror (no TDS)                                                                                                                                                                                                                                                                                    |
| sales_invoice.js (v10.5)         | global | SI mirror + selling warehouse injection + due date from Customer custom_days_limit                                                                                                                                                                                                                                 |
| sales_warehouse_common.js        | global | Quotation/SO/DN warehouse injection + save sweep                                                                                                                                                                                                                                                                   |
| pi.js (doctype.js PI)            | PI     | Vehicle picker for custom_subtype : allowed vehicles = the item's expense Account's custom_sub_type_list.vehicle_list                                                                                                                                                                                              |
| journal_entry.js (doctype.js JE) | JE     | same vehicle picker keyed by account                                                                                                                                                                                                                                                                               |

## 2. Doctype class overrides ( custom\_code/Override/overrides.py )

Registered in hooks.py:230-240 . Two purposes: **(a)** preserve item warehouses copied from source docs during get\_mapped\_doc flows ( \_restore\_warehouse , only at creation — never overwrites later user edits); **(b)** soften ERPNext's "Warehouse is mandatory for stock Item" validation so the app's own warehouse-forcing can run (either swallow validate\_warehouse exceptions or selectively pass only that specific ValidationError).

| Class               | Restores warehouse from                   | Extra                                                                              |
|---------------------|-------------------------------------------|------------------------------------------------------------------------------------|
| SalesOrder          | Quotation Item                            |                                                                                    |
| DeliveryNote        | Sales Order Item                          |                                                                                    |
| CustomSalesInvoice  | Delivery Note Item, else Sales Order Item |                                                                                    |
| RequestforQuotation | Material Request Item                     | selective validate swallow                                                         |
| SupplierQuotation   | RFQ Item                                  | selective swallow                                                                  |
| PurchaseOrder       | Supplier Quotation Item                   | + validate_workflow bypass for Administrator on "Purchase Order Workflow"          |
| PurchaseReceipt     | Purchase Order Item                       |                                                                                    |
| PurchaseInvoice     | Purchase Receipt Item, else PO Item       |                                                                                    |
| MaterialRequest     | —                                         | validate_workflow bypass for Administrator on "Material Request One-Line Approver" |

Other overrides in Override/ :

- **get\_item\_details.py** — wraps ERPNext's whitelisted get\_item\_details : applies the item-price patch, strips the cmd kwarg, delegates.
- **auto\_insert\_item\_price.py** — idempotent monkey-patch (also a global before\_request hook) making ERPNext's auto-created Item Prices carry company / custom\_company from the transaction (Item Price company is mandatory on this site).
- **query\_report.py** — report "Add Column" shows a Link field's display name instead of its id.

## 3. Branch-wise warehouse system

Mapping lives on **Item** child table custom\_branch\_wise\_warehouse (child doctype **Branch Wise Warehouse**; △ its operative fields custom\_branch , custom\_buying\_warehouse , custom\_selling\_warehouse are site-DB custom fields — the exported JSON only has a placeholder). Item-level custom\_buying\_warehouse / custom\_selling\_warehouse are the fallback.

Resolution per item row: branch row matching the doc's custom\_branch → item-level fallback → leave ERPNext's value (a manually chosen warehouse is preserved when no mapping exists). Enforced in three layers: JS frappe.call wrap of get\_item\_details (race-free), JS before\_save sweeps, and the server force\_buying\_warehouse / force\_selling\_warehouse in the validate hooks (these win over ERPNext).

## 4. Voucher numbering engine ( Override/naming\_series.py , 31 KB)

**Wiring:** not directly in hooks.py — dispatched by AuditEventManager ( utils/audit\_file\_manager.py ) on autoname , validate , before\_save , before\_insert , after\_delete for every audited doctype (chapter 6 §3.3).

The type codes embedded in voucher numbers come from **DB-only code-list doctypes**: Purchase Type, Receipt type, Payment - Receipt Type, JV Type (each maps a type name → a code, fetched into custom\_p\_type\_code ). They are not in the repo — see [chapter 11 Part C](#).

- **NAMING\_CONFIG** ( :28-530 ) — ~100 doctypes → prefix / sequence length / fiscal-year flag / return prefix (e.g. Sales Invoice SB / SRTN , PI PI / PRTN , PR GRN , JE JE , PE PAY.REC ). FY pattern: {abbr}-{prefix}-{fy}-{#####} , date from posting\_date → transaction\_date → attendance\_date → custom\_created\_on .
- **AUTO\_NUMBER\_CONFIG** ( :532-562 ) — which (doctype, type-field, types) get the **auto-incrementing custom\_document\_no** : PR "Other Purchase Receipt"; PI "Purchase Return"; PE "Bank Customers Receipt"/"NOC Payment"/"Contra Voucher- cash to bank"; JE "Bank Entry"/"Party Journal"/"Debit Note"/"Credit

Note". A user-typed number is **left untouched** (:750-757) — number+word (custom\_document\_word, e.g. 65A) together identify the voucher.

- set\_custom\_name\_field (:782) assembles the human voucher no:  
custom\_name = "{abbr}-{p\_type\_code}-{doc\_no}{word}-{fy}{amend\_suffix}" →  
e.g. **SGU-RC-000006-82/83**. Zero-pad width from **Voucher Number Settings Item** (voucher\_no\_digits, default 6). Company "Grihalaxmi Metal Industries Pvt. Ltd" gets a blank custom\_name by design.
- validate\_custom\_name\_unique (:857) — voucher uniqueness among docstatus<2 (cancelled excluded so amendments reuse numbers);  
validate\_document\_no — positive integer only.
- set\_custom\_branch\_name (:933) — per-branch series  
{abbr}-{branch\_code}-{seq6}-{fy} via getseries, **only** for company Grishma Enterprises Pvt. Ltd. (BRANCH\_CODE\_CONFIG, :9-25); other companies fall back to doc.name. This is what CBMS uses as the invoice number (chapter 5).
- revert\_series\_on\_delete (:1011) — decrements tabSeries if the deleted doc held the highest number, so numbers are reused.
- get\_next\_custom\_document\_no (whitelisted) — preview for public/js/auto\_update\_document\_no.js (PR/PI/PE/JE forms auto-fill custom\_document\_no on load and when series-scope fields change).

Console backfills: custom\_code/voucher\_no\_console.py (legacy, zfill 5) and scripts/pad\_custom\_name.py (pad to 6 digits, dry-run by default).

---

## 5a. IRD print-copy labeling (SalesInvoice/print\_count.py)

---

**Why:** Nepal IRD e-billing rules require the software to count how many times an invoice is printed and to label reprints as copies of the original:

```
1st print → Tax Invoice
2nd print → Copy of Original
3rd print → Copy of Original 2
nth print → Copy of Original (n-1)
```

**Wiring:** Sales Invoice before\_print (hooks.py:82).

- The counter (custom\_print\_count, an Int custom field added by patch add\_sales\_invoice\_print\_count) increments **only on an actual print** — detected by is\_actual\_print() (print\_count.py:32-36): the browser Print button (trigger\_print=1) or a PDF/server-print cmd in PRINT\_OUTPUT\_CMDS (download\_pdf, download\_multi\_pdf, print\_by\_server, weasyprint). Rendering the Print **preview** does **not** consume a number.
- before\_print (print\_count.py:39-66): submitted invoice + real print → atomic UPDATE ... SET custom\_print\_count = custom\_print\_count + 1 then frappe.db.commit() (printview/download are GET requests, so it must commit explicitly to survive request-end rollback). Otherwise (preview / draft / cancelled) it stamps stored + 1 **in memory only** — the render shows the title the next print *will* get, without consuming it.
- The print format reads custom\_print\_count to choose the title.

---

## 5. GL manipulation

---

- **custom\_code/excise\_ledger.py** — only modify\_gl\_entries (:657) is wired (PI before\_submit): monkey-patches the instance's get\_gl\_entries so TDS-payable GL lines (accounts under 348100 - TDS Payable - {abbr}) get party\_type="Supplier" + party stamped — TDS ledgers reconcile per supplier. ⚠ Everything else in this 32 KB file is dead code with hardcoded GLMI accounts — do not revive.
- **custom\_code/payment\_entry/cheque\_bounce.py** — make\_cheque\_bounce\_entry(pe) (whitelisted; "Cheque Bounce" button on submitted Payment Entries, payment\_entry.js): posts a full debit↔credit swap of the PE's GL (same voucher\_type/no, posting\_date today, remarks "Cheque Bounce - {name}", update\_outstanding="Yes" → invoice outstanding reopens), sets custom\_cheque\_bounce="Cheque Bounced". Duplicate-guarded. The PE itself is not cancelled.
- **custom\_code/stock\_revaluation.py** — on\_purchase\_invoice\_submit (PI on\_submit): for backdated update-stock PIs, msgprints that valuation reposting runs in the nightly job. Helpers nightly\_process\_pending\_reposts (restarts failed Repost Item Valuation, long queue), whitelisted reprocess\_all\_pending / get\_repost\_status. ⚠ The nightly job is **not** in scheduler\_events — invoke manually or schedule separately.
- **custom\_code/regional\_deletion\_override.py** — monkey-patch applied at app import (\_\_init\_\_.py): Administrator/System Manager may delete submitted transactions of Nepal companies; everyone else falls through to ERPNext's regional guard.

---

## 6. Credit control (⚠ currently dormant)

---

SalesInvoice/salesinvoice\_customer.py — whitelisted  
check\_customer\_credit\_limit\_on\_load / \_on\_save reading Customer  
custom\_days, custom\_amount, custom\_bill\_count (outstanding from Customer Ledger Summary; overdue via SQL). These are called only from public/js/si.js, **which is not loaded by hooks.py** — so the feature is currently unwired.

SalesInvoice/credit\_control.py is dead (largely commented out and contains a debugpy.listen()+wait\_for\_client() that would hang requests). A third variant validate\_sales\_invoice inside salesinvoice\_taxes.py:412 is also unwired. Note the field mismatch between variants ( custom\_days/amount vs custom\_days\_limit/amount\_limit on Customer). **To re-enable credit control, pick one implementation, reconcile field names, and wire it explicitly.**

7. Vehicle-mandatory validation ( custom\_code/vehicle\_mandatory.py )

Vehicle expense lines must carry a Vehicle ( custom\_subtype ). Account matched by name patterns ( "Fuel Expenses", "R & M - Vehicles", "Other Vehicle Expenses" ). Wired on JE validate (rows in accounts by account ) and PI validate (rows in items by expense\_account ); throws per offending row, with a hint when the Account's custom\_sub\_type\_list has no vehicles configured. That per-account vehicle whitelist is the **Vehicle List** child doctype (DB-only — chapter 11 Part C), attached to Account as custom\_sub\_type\_list . Client mirror vehicle\_mandatory.js (per-row on purpose — avoids Frappe's shared-docfield mandatory-propagation bug). Property-setter approach was tried and removed (patch run twice) because mandatory\_depends\_on isn't server-enforced.

8. Stock Adjustment (doctype stock\_adjustment + child items)

Submittable, quantity-only stock correction (System/Stock Manager). validate: ≥1 item, adjustment\_qty > 0 , **rate must be 0** (value movements must use Stock Entry), warehouse must belong to the company. on\_submit /on\_cancel post SLEs (sign +1 Gain / −1 Loss, incoming\_rate=0 , is\_adjustment\_entry=1 ) and repost future SLE/GLE. engine\_patch.py (applied at module import) monkey-patches update\_entries\_after.process\_sle so zero-rate Stock Adjustment SLEs **preserve the warehouse's stock value** (qty changes, stock\_value\_difference=0 , valuation rate re-derived) — and this survives reposts triggered by later backdated vouchers. Background: docs/stock\_ledger\_guide.md and docs/sales\_stock\_ledger\_fixes.md .

Diagnostics: custom\_code/stock\_health\_check.py ( bench --site avinas1 execute avinashgroup\_app.custom\_code.stock\_health\_check.run ) — negative-stock flag, negative SLE balances, zero-valuation sales SLEs, repost job status, backdated update-stock PIs.

9. Portal & supplier flows (server side)

templates/pages/rfq.py overrides ERPNext's create\_supplier\_quotation (supplier portal): copies document-level discounts, seeds per-line VAT custom fields, runs calculate\_taxes\_and\_totals , saves with ignore\_permissions , re-links uploaded Files. upload\_portal\_item\_attachment (whitelisted) stores base64 portal uploads as private Files. (Customer portal pages: chapter 8 §5.)

10. Master-form helpers (client-only)

- party\_duplicate\_check.js (Customer/Supplier): same-company duplicate name/tax\_id → non-blocking Yes/No confirm.
- party\_default\_account.js / item\_default\_account.js : auto-add one default accounts/item-defaults row with Company from custom\_company , kept in sync.
- See docs/customer\_supplier\_item\_duplicate\_checks\_and\_default\_accounts.md .

11. Legacy / dead code inventory (do not build on)

| File                                                       | Status                                                                                                  |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| custom_code/override_rounding.py (71 KB)                   | unwired; 57% commented; excise-inclusive recompute superseded by the live handlers                      |
| custom_code/purchase_invoice/purchase_invoice_taxes_tds.py | superseded alternate PI handler (Percentage-based VAT) — not wired                                      |
| SalesInvoice/credit_control.py                             | dead + debugpy trap                                                                                     |
| public/js/si.js , purchase_invoice.js                      | not loaded by hooks                                                                                     |
| custom_code/api.py naming/fiscal helpers                   | superseded by naming_series.py (the created_by installer/backfill parts are still useful console tools) |
| custom_itemname.py ( CustomItem )                          | never registered                                                                                        |
| custom_customer.py                                         | misnamed; only locks company on two test doctypes                                                       |
| excise_ledger.py everything except modify_gl_entries       | dead, hardcoded GLMI accounts                                                                           |
| purchase_taxes_mapper.after_mapping_*                      | unreferenced                                                                                            |
| workflow_material_request.py                               | thin legacy alias of workflow.set_reject_reason                                                         |

Console-only utilities (live but manual): setup\_company\_lock.py / company\_field\_lock.py setup functions (the validate\_company\_field\_lock runtime guard **is** wired via the audit map), update\_workflows.run() , create\_log\_doctype.py , voucher\_no\_console.py , stock\_health\_check.py , api.create\_created\_by\_and\_created\_on\_fields / populate....

# CBMS Integration (Nepal IRD e-Billing) — Technical Reference

Chapter 5 of the technical documentation. Audience: developers.  
User-facing guide: [../user\\_guide/05-cbms-billing.md](#)

CBMS is Nepal IRD's **Central Billing Monitoring System**. Every submitted Sales Invoice (and Sales Return / credit note) must be reported to IRD in real time. The integration is entirely server-side: Python hooks + three doctypes. There is no client script and no print-format change.

## 1. File inventory

| File                                                | Role                                                                           |
|-----------------------------------------------------|--------------------------------------------------------------------------------|
| custom_code/CBMS/sales_invoice_hooks.py             | on_submit / before_cancel doc_events; payload field mapping; CBMS doc creation |
| custom_code/CBMS/api_client.py                      | HTTP client to IRD; payload builders; result recording                         |
| custom_code/CBMS/scheduler.py                       | retry + reconcile cron jobs (every 5 min)                                      |
| custom_code/CBMS/utils.py                           | BS-date, fiscal-year, invoice-number helpers                                   |
| doctype/cbms_bill/, cbms_bill_return/, cbms_config/ | the three doctypes                                                             |
| patches/create_cbms_custom_fields.py                | adds custom_reason_for_return to Sales Invoice                                 |
| hooks.py:83-85, 168-169, 246-251                    | doc_event + scheduler wiring                                                   |
| pyproject.toml:12                                   | nepali-datetime dependency                                                     |

## 2. Design invariants

1. **A Sales Invoice submission can never be blocked by CBMS**. The submit hook wraps everything in try/except and logs to Error Log ( sales\_invoice\_hooks.py:199-203 ).
2. **Every network call runs in a background job** ( frappe.enqueue , queue default , timeout 300 s), never inline ( sales\_invoice\_hooks.py:197-198 ).
3. **Two independent safety nets** run every 5 minutes: *retry* (CBMS docs exist but aren't Synced) and *reconcile* (submitted invoices with no CBMS doc at all).
4. **A Synced invoice cannot be cancelled** — corrections flow through a Sales Return, which becomes a CBMS Bill Return.
5. **Returns are ordered after their originals** — a Bill Return is only POSTed once the referenced Bill is Synced ( api\_client.py:171-175 ).
6. **Production IRD endpoints only** — there is no sandbox mode. Go-live is controlled solely by enable\_cbms + enable\_from\_date in CBMS Config.

## 3. Payload construction

### 3.1 Shared fields — build\_cbms\_fields ( sales\_invoice\_hooks.py:68-126 )

- buyer\_pan ← Customer tax\_id ; seller\_pan ← Company tax\_id (cached)
- buyer\_name ← customer\_name or customer
- fiscal\_year ← utils.cbms\_fiscal\_year(posting\_date) → IRD dotted format, e.g. "2081.082" ( utils.py:25-35 )
- **Line-value basis** \_line\_value ( :44-49 ): per item custom\_total (net + excise, maintained by the selling taxes handler — chapter 4) when present, else base\_net\_amount . VAT is levied on net+excise, so this is the correct base.
- tax\_exempted\_sales — sum of line values where custom\_vat\_apply\_on == "VAT 0%"
- taxable\_sales\_vat — sum of line values of all other lines
- vat — the VAT **actually booked** ( \_booked\_vat :52-65 ): custom\_total\_vat\_amount , falling back to summing base\_tax\_amount of tax rows whose account\_head starts with VAT . Never recomputed.
- **Export detection** is\_export\_invoice ( :32-41 ): currency ≠ NPR, or the Customer's Territory ancestor tree lies outside Nepal (empty / "Nepal" / "All Territories" = domestic). Exports put the whole value in export\_sales , zero taxable/exempt, and total\_sales = base\_grand\_total .
- total\_sales = base\_grand\_total - exempt\_sales for domestic invoices
- discount — base\_discount\_amount only when apply\_discount\_on == "Net Total" ; stored on the CBMS doc **only** (the IRD API has no discount field)
- All amounts `flt(..., 2)` ; absolute values throughout, so returns store positive numbers.

### 3.2 Bill extras ( create\_cbms\_bill :129-143 )

invoice\_number = utils.cbms\_invoice\_number — the invoice's branch-wise running number custom\_branch\_name (set by the naming engine, chapter 4), falling back to doc.name ; invoice\_date (AD) and invoice\_date\_bs (BS, dash format).

### 3.3 Return extras ( create\_cbms\_bill\_return :146-171 )

ref\_invoice\_number = the **original** invoice's invoice\_number looked up from the CBMS Bill of return\_against ; credit\_note\_number = this return's branch number ; credit\_note\_date / \_bs ; reason\_for\_return = Sales Invoice custom\_reason\_for\_return or default "Goods Returned" .

3.4 HTTP payloads ( api\_client.py )

- Bill → POST https://cbapi.ird.gov.np/api/bill ( \_build\_bill\_payload :53-69 ): username / password (from CBMS Config, get\_password ), seller/buyer PAN, buyer name, fiscal year, invoice number, invoice\_date (dotted BS), total\_sales , taxable\_sales\_vat , vat , \_other\_sales\_fields ( :38-50 — excise/hst/esf all zero for this business, plus export\_sales , tax\_exempted\_sales ), isrealtime: True , datetimeClient (ISO).
- Return → POST https://cbapi.ird.gov.np/api/billreturn ( \_build\_return\_payload :79-97 ): same base, plus ref\_invoice\_number , credit\_note\_number , credit\_note\_date , reason\_for\_return .  
△ buyer\_pan goes through \_buyer\_pan\_decimal ( :72-76 ) → int or None, because the billreturn endpoint declares buyer\_pan as a nullable decimal and rejects any string (even "" ) with HTTP 400.
- REQUEST\_TIMEOUT = 30 s. URLs are hardcoded constants ( api\_client.py:11-13 ).

4. Submit / cancel flow

on\_submit ( sales\_invoice\_hooks.py:174-205 )

1. Idempotency lock: cache key cbms\_processing\_{doc.name} , 300 s TTL; released in finally .
2. Load the enabled CBMS Config for the company ( get\_cbms\_config :18-21 ); return if none, or if posting\_date < enable\_from\_date ( in\_cbms\_scope :24-29 ) — no retroactive reporting.
3. is\_return → create\_cbms\_bill\_return + enqueue\_send\_return\_to\_cbms ; else create\_cbms\_bill + enqueue\_send\_bill\_to\_cbms .
4. Creation helpers are idempotent — they skip if a CBMS doc already exists for this invoice (also enforced by the unique constraint on sales\_invoice ).
5. Return special case: if the original invoice has no CBMS Bill yet, creation returns None — the reconcile job picks it up later.
6. frappe.db.commit() before enqueue.

before\_cancel ( :208-220 )

If the matching CBMS doc has sync\_status == "Synced" → frappe.throw blocks cancellation. Pending/Failed/absent → cancel freely. There is **no reversing call to IRD**.

5. Scheduler (every 5 minutes, hooks.py:246-251 )

- retry\_failed\_cbms\_syncs → queue\_failed\_for\_company ( scheduler.py:26-59 ): per enabled company, re-enqueue sends for CBMS Bills>Returns with sync\_status != "Synced" , capped at bill\_retry\_batch\_size / return\_retry\_batch\_size (default 50 each).
- reconcile\_missing\_cbms\_bills ( scheduler.py:87-135 ): per company, find submitted Sales Invoices ( posting\_date >= enable\_from\_date ) with **no** CBMS doc at all, create the doc and enqueue the send. Each invoice is try/except-wrapped with Error Log on failure.

6. Send logic and status model

send\_bill\_to\_cbms ( api\_client.py:127-158 ) / send\_return\_to\_cbms ( :160-200 ):

- Skip if already Synced; abort if config disabled.
- Response classification \_response\_code ( :100-108 ): accepts bare "200" text or {"message":"200"} JSON.
- **Bills**: success codes {"200", "101"} — 101 ("Bill already exists") counts as success.
- **Returns**: success code {"200"} **only** — for returns 101 means "Bill does not exist" and is a failure. This asymmetry is deliberate ( api\_client.py:15-17 ).
- Error codes ( BILL\_ERRORS / RETURN\_ERRORS :19-35 ): 100 credentials mismatch, 102 save exception, 103 unknown, 104 model invalid, 105 bill does not exist (returns).
- \_record\_result ( :111-124 ): sets sync\_status , truncates sync\_response to 500 chars, increments attempt\_count , sets last\_attempt , update\_modified=False , commits.
- Both senders **never raise**: exceptions are logged, rolled back, status forced to Failed, committed, return False — a failure loops harmlessly every 5 minutes until fixed.

Status fields on both CBMS Bill and CBMS Bill Return:

| Field           | Type                                           | Meaning                     |
|-----------------|------------------------------------------------|-----------------------------|
| sync_status     | Select Pending/Synced/Failed (default Pending) | list-view + standard filter |
| attempt_count   | Int                                            | send attempts               |
| last_attempt    | Datetime                                       | last try                    |
| sync_response   | Small Text ≤500                                | last IRD code+message       |
| datetime_client | Datetime                                       | timestamp reported to IRD   |

All CBMS doc fields are read-only in the UI; track\_changes is on.

7. CBMS Config doctype (one per company)

| Field                                           | Type                  | Notes                           |
|-------------------------------------------------|-----------------------|---------------------------------|
| company                                         | Link, unique          | autoname: field:company         |
| enable_cbms                                     | Check (default 0)     | master switch                   |
| enable_from_date                                | Date                  | go-live; mandatory when enabled |
| username / password                             | Data / Password, reqd | IRD API credentials             |
| bill_retry_batch_size / return_retry_batch_size | Int (default 50)      | per-run retry caps              |

Controller ( `cbms_config.py:10-15` ) enforces one config per company. Whitelisted `sync_failed_now(cbms_config_name)` ( `:18-24` ) manually queues all failed docs for that company — callable from console/API (no UI button exists).  
Permissions: System Manager + Accounts Manager (full).

8. Troubleshooting quick reference

| Symptom                                                        | Cause / fix                                                                                                                                                                    |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sync_response = 100: API credentials do not match</code> | Fix Username/Password in CBMS Config                                                                                                                                           |
| Return stuck Pending, never POSTs                              | Original invoice's CBMS Bill not Synced yet — fix that first; return auto-recovers                                                                                             |
| Invoice submitted but no CBMS Bill                             | <code>on_submit</code> hook failed (see Error Log) — reconcile job creates it within 5 min                                                                                     |
| Cannot cancel invoice                                          | It is already reported to IRD — issue a Sales Return instead                                                                                                                   |
| Need immediate retry                                           | <code>bench --site avinash1 console → frappe.get_doc("CBMS Config", "&lt;company&gt;").sync_failed_now("&lt;company&gt;")</code> or call <code>queue_failed_for_company</code> |

9. IRD print-copy labeling (Tax Invoice / Copy of Original)

IRD e-billing requires the software to count how many times an invoice is printed and to label reprints as copies of the original. The print title is derived from a counter on each Sales Invoice:

| Print # | Title shown            |
|---------|------------------------|
| 1st     | Tax Invoice            |
| 2nd     | Copy of Original       |
| 3rd     | Copy of Original 2     |
| nth     | Copy of Original (n-1) |

Pieces:

- Field** `Sales Invoice.custom_print_count` (Int, read-only, `no_copy`, `print_hide`) — created by patch `avinashgroup_app.patches.add_sales_invoice_print_count`.
- Hook** `custom_code/SalesInvoice/print_count.py::before_print` (registered on Sales Invoice `before_print`). On a *real* print of a submitted invoice it does an atomic `custom_print_count = custom_print_count + 1` and commits (`printview/download` are GET requests, so the commit is explicit). A "real print" = `trigger_print=1` (browser Print button) or a PDF/server print `cmd`. Opening the Print **preview** does NOT consume a number — the preview shows the title the *next* print will get.
- Print format** `Avinash Sales invoice` reads `doc.custom_print_count` to choose the title (see the `{% set print_count_no ... %}` block at the top of its HTML). Sales **Returns** are unaffected — they print via `Avinash Sales Return` and the title block is gated on `not doc.is_return`.

The print format HTML lives only in the DB (custom, non-standard format), not in a fixture — re-running the patch will not touch it. If the format is ever re-imported/overwritten, re-apply the `print_count_no` title block.

# Access Control, Company Filtering & Audit — Technical Reference

Chapter 6 of the technical documentation. Audience: developers.  
User-facing guide: [../user\\_guide/08-admin-setup.md](#)

Four related subsystems: **fiscal-year access control**, **company filtering**, **audit trail**, and **user daily entry summary**.

Path note: doctypes/reports live under `avinashgroup_app/avinash_group_app/...` (with underscore); custom code and utils under `avinashgroup_app/...` directly.

## 1. Fiscal-Year Access Control

Restricts which fiscal years' transactions a user can see.  
Core file: `custom_code/fiscal_year_filter.py`.

### 1.1 Scope

- `FILTERED_DOCTYPES` (:8-84) — 77 transaction doctypes (Sales Invoice, Purchase Invoice, Payment Entry, Journal Entry, Stock Entry, Attendance, Leave Application, Salary Slip...). Anything not in the tuple is never filtered.
- `DATE_FIELD_MAP` (:103-179) — per-doctype date field used for the comparison (Attendance → `attendance_date`, Sales Order → `transaction_date`...); fallback `posting_date`.

### 1.2 Who is restricted

- `_is_admin` (:87-100): **Administrator or any System Manager always bypasses** all checks.
- `_get_user_fiscal_access(user)` (:196-256, `@request_cache`): reads the user's Fiscal Year Access Control record.  
**No record → full access (allow-all default).** `full_access=1` → full access. Otherwise child rows (User Fiscal Year Access) build `{doctype: "full_access" | [fiscal years]}`. A legacy fallback (:259-301) reads the pre-migration per-User custom fields.

### 1.3 Enforcement — three live layers

| Layer                                             | Function                                                                                                                      | Wired at                                                                  |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| List views / reports (SQL WHERE)                  | <code>_build_query_conditions</code> (:445-484) via generated <code>query_conditions_&lt;slug&gt;</code> functions (:487-497) | <code>hooks.py:214-217</code><br><code>permission_query_conditions</code> |
| Direct document access                            | <code>has_fiscal_year_permission</code> (:500-538)                                                                            | <code>hooks.py:220-223</code> <code>has_permission</code>                 |
| Client API<br><code>frappe.client.get_list</code> | <code>filtered_get_list</code> (:541-581) injects date filters or returns <code>[]</code>                                     | <code>hooks.py:263</code><br><code>override_whitelisted_methods</code>    |

Query conditions return `""` (no restriction), `"1=0"` (no access), or `(BETWEEN 'from' AND 'to' OR ...)` per allowed fiscal year, dates escaped.  
`has_fiscal_year_permission` is permissive when the doc has no parseable date (:525-526).

### 1.4 Known gaps / quirks (documented on purpose)

- No write-time block:** `validate_fiscal_year_access` (:390-429) and `apply_fiscal_year_filter_to_list` (:342-387) are defined but **not wired** in `hooks.py` — a user can still save a document dated outside their range; they just won't see it afterwards.
- Cache invalidation is per-request only:** the resolver uses `@request_cache` (in-memory, dies at request end), but the invalidation hooks (`clear_user_fiscal_cache` on User update, the controller's `on_update / on_trash`) delete `redis` keys that the resolver never writes — and the two key formats don't even match each other (`user_fiscal_access_*` vs `_get_user_fiscal_access_*`). Net effect: changes apply on the next request anyway; the `redis` clears are no-ops.
- An older `FISCAL_YEAR_ACCESS.md` claim that bypass is by `user_type == "System User"` is stale — bypass is Administrator/System Manager role.

### 1.5 Doctypes

- Fiscal Year Access Control** — `autoname` `field:user` (one per user); fields `user` (Link, unique reqd), `full_access` (Check), `access_details` (Table → User Fiscal Year Access). System Manager only; `track_changes`.
- User Fiscal Year Access** (child) — `doctype_name` (Link → DocType, reqd), `fiscal_year` (Link, mandatory unless row `full_access`), `full_access` (Check = all fiscal years for that doctype).
- Patch `migrate_fiscal_year_access_control.py` migrates the legacy per-User custom-field config and deletes those fields.

### 1.6 Related: default fiscal year helper

`utils/fiscal_year_utils.py::get_default_fiscal_year()` computes the Nepali BS fiscal year label ( "82/83" ; Shrawan→Ashadh via

`rdp_common_app.utils.bs_boundaries`). Consumed by `public/js/fiscal_year_cache.js` (`window.FiscalYearCache`, once-per-day client cache, midnight rollover). This is a convenience for form scripts — **not** part of access enforcement.

---

## 2. Company Filter System

---

Ensures every Link selection on a document belongs to the document's company.  
Core file: `custom_code/globalfilter/globalfilter.py`.

### 2.1 Server side

- `FILTER_CONFIG` (:12-170) — hardcoded fallback config per doctype: `company_field` (`company/custom_company`), top-level `fields`, `child_tables`. Covers the Asset family, masters, and the full buying/selling chain.  $\Delta$  The party-aware custom validation for Journal Entry / Payment Entry / Bank Account is **commented out** in `CompanyValidator.validate()` (:479-494).
- `_resolve_company_field(linked_doctype)` (:216-235): `Company`  $\rightarrow$  `name`, else `company`, else `custom_company`, else skip. Customer/Supplier scoped via the Allowed To Transact With `child` table (:238-257).
- **CompanyValidator** (:290-496) — 3-phase validator: collect all linked values (zero DB), one batch query per linked doctype, then compare each value's company to the parent's. Human-readable mismatch errors.
- `validate_company_matching(doc, method)` (:503-520) — the entry point; **throws** on mismatch (warns instead during data import). Wired via the audit engine: `audit_file_manager.validate` calls it for every audited doctype (\$3).
- `get_filter_config()` (:527-636, whitelisted) — the JS config. Redis-cached (`company_filter_config`); built from the **Company Filter Config** / **Company Filter Field** doctypes; falls back to `FILTER_CONFIG` when the tables are empty. Pre-resolves `__filter_keys__ = {linked_doctype: company_field}` so the client never needs meta lookups.
- `clear_filter_config_cache` — wired on `Config/Field on_update/on_trash` (`hooks.py:204-207`).
- Whitelisted searches used by `set_query: search_party` (:655-712) and the generic dynamic-link `search_link_by_company` (:719-779).

### 2.2 Client side

- `public/js/company_filter.js` (`avinash.filter_engine`) — on `app_ready` loads the config, then for each configured doctype: `set_query` filters on every configured field (dynamic links delegate to `search_link_by_company`); on company change, `validate_and_clear` clears mismatched top-level values (orange alert) and removes mismatched child rows. Preserves pre-existing `get_query` from the form's own scripts.
- `public/js/global_filter.js` — row-level guard: hooks `item_code` on 12 item child tables (+ RFQ `supplier`); a selected value whose company mismatches the parent is cleared with a red "Company Mismatch" message. Catches what dropdown filtering can't (paste/programmatic set).

### 2.3 Doctypes

- **Company Filter Config** — `autoname field:doctype_name; company_field` Select; `fields` Table. Controller clears the cache and asks users to refresh.
- **Company Filter Field** (child) — `fieldname, is_child_table + child_fieldname, is_dynamic_link + dynamic_link_field`.
- Patch `seed_company_filter_config.py` seeds the tables from `FILTER_CONFIG` (currently commented out in `patches.txt`).

---

## 3. Audit Trail

---

Core file: `utils/audit_file_manager.py`.

### 3.1 What is audited

`AuditBase.doctypes` (:10-111) — ~85 doctypes (masters + all transactions).  
`AuditBase.master_doctypes` (:116-145) — the subset that also gets a `custom_naming_series` field.

### 3.2 Fields and stamping

`AuditFieldsManager` (:149-462) installs per-doctype custom fields: an `audit_section break, custom_company` (Link, reqd — only when the doctype has no native company field), `custom_naming_series` (masters), and `custom_created_by / custom_created_on / custom_modified_by` (read-only, no-copy, print-hide). `set_audit_fields` (:551-578) populates them on save (session user; background jobs honor `frappe.flags.audit_user`).

$\Delta$  `AuditFieldsManager()` with **no argument targets all ~85 doctypes** — always pass an explicit list when running from the console.

### 3.3 The event map — the app's `doc_events` backbone

`AuditEventManager.get_doc_events()` ( :466-492 ) is what `hooks.py:131` assigns as the base `doc_events` . For every audited doctype:

| Event                      | Dispatcher does                                                                                                 |
|----------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>before_insert</code> | <code>set_audit_fields</code> + naming requirements                                                             |
| <code>before_save</code>   | <code>set_audit_fields</code> + naming save handling                                                            |
| <code>validate</code>      | company-field lock check + <b><code>validate_company_matching</code></b> (company filter \$2) + naming validate |
| <code>autoname</code>      | custom naming series (chapter 4 \$naming)                                                                       |
| <code>after_delete</code>  | revert the number series                                                                                        |

So audit stamping, company matching, and custom document numbering all ride this one map; feature-specific handlers are merged on top with `_add_doc_event` .

3.4 Change history and report

- **WHO** → the custom fields on each row. **WHAT** → Frappe's `Version` doctype; patch `enable_track_changes_audited_doctypes.py` switches on `track_changes` for every audited doctype (only edits after the patch generate Versions).
- **User Audit Trail** report ( `report/user_audit_trail/` , roles: System Manager, Auditor, Accounts Manager, HR Manager): filters user\* / from-to dates\* / action / document types. "Created" rows from `custom_created_by` ; "Modified" rows parsed from `Version.data` JSON (field diffs, child-row add/remove, humanized docstatus). Only doctypes the *running* user can read are shown. Document column links to the record.

4. User Daily Entry Summary

Per user + day: counts of documents **created** ( `owner` ), by doctype and docstatus. Full feature doc: `docs/user_daily_entry_summary.md` .

- **User Daily Entry Summary Settings** (Single, System Manager) — child table of tracked doctypes. Untracked = one aggregate SQL per doctype per run, so keep the list short.
- Report ( `report/user_daily_entry_summary/` ): filters user, *date* (default today), document types. One conditional-aggregate query per doctype ( SUM(docstatus=...) over owner + creation day window). Count cells are drill-down links to the filtered list view. Always runs fresh ( `ignore_prepared_report` ).
- Patch `add_creation_index_daily_entry.py` adds a composite (owner, creation) index to each tracked table.
- Caveat: bulk-imported docs count against the importing account.

5. Utility scripts & misc assets

| File                                                                         | Purpose                                                                                                                                                                           |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>utils/export_field.py</code>                                           | <code>export_custom_fields_to_excel()</code> — dump all <code>custom_*</code> fields to a timestamped <code>.xlsx</code> in <code>private/files/</code> (inventory/audit tooling) |
| <code>scripts/cleanup_audit_fields.py</code>                                 | remove naming/company/audit custom fields + property setters from doctypes dropped from the audit list; restores original autoname; never renames records                         |
| <code>scripts/pad_custom_name.py</code>                                      | zero-pad the numeric run in <code>custom_name</code> to 6 digits (PR/PI/IE/PE); dry-run by default; skips collisions                                                              |
| <code>public/js/auto_update_document_no.js</code>                            | auto-fills <code>custom_document_no</code> on new PR/PI/IE/PE via <code>naming_series.get_next_custom_document_no</code> (chapter 4 \$naming)                                     |
| <code>public/css/desk_focus.css</code>                                       | cosmetic — orange focus ring + dropdown highlight, tunable via CSS variables                                                                                                      |
| <code>public/css/report_print_portrait.css</code>                            | injected dynamically by <code>report_print_orientation.js</code> only on query-report routes (loading it globally would restyle doctype print formats)                            |
| <code>scripts/pad custom name.py</code> / <code>voucher_no_console.py</code> | voucher-number backfills (chapter 4 \$4)                                                                                                                                          |

6. Admin setup checklists

**Fiscal-year access** — ensure Fiscal Year records have correct start/end dates; create a Fiscal Year Access Control per restricted user; either check Full Access or add per-doctype rows (specific FY or row-level full access). Users **without** a record have full access. Admin/System Managers always bypass.

**Company filter** — manage Company Filter Config records (one per doctype); add Company Filter Field rows for plain, child-table, or dynamic-link fields. Cache clears on save; users must refresh the browser.

**Audit trail** — migrate (installs fields + `track_changes`); grant Auditor/Accounts Manager/HR Manager for the report. Console management:

```
from avinashgroup_app.utils.audit_file_manager import AuditFieldsManager, update_custom_company_reqd
AuditFieldsManager(["Item Price"]).create_fields() # always pass a list!
AuditFieldsManager(["Item Price"]).verify_fields()
AuditFieldsManager(["Item Price"]).remove_fields()
update_custom_company_reqd()
```

**Daily entry summary** — add tracked doctypes in the Settings single; migrate for the index; run the report.

# Document Generator — Technical Reference

Chapter 7 of the technical documentation. Audience: developers & template authors.  
User-facing guide: [../user\\_guide/06-document-generator.md](#)

Generates formal letters (e.g. balance-confirmation letters) from admin-authorized HTML+Jinja templates with SQL/Python data sources, editable in a WYSIWYG page, output as PDF (letterhead-aware) or email. Every printed/emailed copy is persisted as a **Generated Document**.

## 1. File inventory

| Path                                        | Role                                                                                                                                                    |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| custom_code/document_generator/api.py       | all whitelisted endpoints                                                                                                                               |
| custom_code/document_generator/providers.py | context builders, SQL/Python source runners                                                                                                             |
| custom_code/document_generator/pdf.py       | pdfkit/wkhtmltopdf wrapper                                                                                                                              |
| custom_code/document_generator/styles.py    | HTML page wrapper + print-safe CSS                                                                                                                      |
| avinash_group_app/page/document_generator/  | the desk page (JS 527 lines + CSS)                                                                                                                      |
| doctypes                                    | document_template, document_template_input, document_template_data_source, document_template_company, generated_document                                |
| fixtures/document_template.json             | the two bundled balance-confirmation templates                                                                                                          |
| patches                                     | create_document_generator_roles, cleanup_document_generator_sections, create_document_generator_image_fields, create_document_generator_signatory_field |

End-to-end call chain (api.py:6-8): get\_matching\_templates → get\_template\_meta → instantiate\_document → user edits → save\_generated\_document → download\_document\_pdf / send\_document\_email.

## 2. Data model

### Document Template (autoname field:template\_name , track\_changes)

| Field                                                                                     | Meaning                                                                      |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| template_name                                                                             | unique id                                                                    |
| target_doctype                                                                            | optional — only for default email-recipient resolution                       |
| companies (Table MultiSelect → Document Template Company)                                 | scoping; empty = all companies                                               |
| letter_head                                                                               | fallback header/footer when header_html / footer_html blank (api.py:108-112) |
| print_orientation                                                                         | Portrait/Landscape                                                           |
| default_recipient_field                                                                   | default email_id                                                             |
| is_active                                                                                 | only active templates appear in the picker                                   |
| email_subject                                                                             | Jinja-enabled                                                                |
| inputs / data_sources                                                                     | child tables (below)                                                         |
| header_html + header_height (default 25 mm) / footer_html + footer_height (default 15 mm) | letterhead bands                                                             |
| body_html                                                                                 | the HTML+Jinja body                                                          |

Controller: validate\_companies de-dupes; validate\_jinja (document\_template.py:24-42) dry-renders body/header/footer against a stub context at save so Jinja syntax errors are caught early.

Permissions: **System Manager** and **Document Template Manager** full; **Document Template User** read/print/report only.

### Document Template Input (child)

fieldname (becomes the %(fieldname)s SQL param and inputs.fieldname Jinja var), label, input\_type (Data/Link/Date/Select/Int/Float/Check), options (Link target or Select options), reqd (⚠ client-side cue only — nothing server-side enforces it), exclusive\_group (inputs filled together as a bundle), exclusive\_set (bundles in the same set are mutually exclusive).

### Document Template Data Source (child)

source\_name (exposed as data.<name>), source\_type (SQL | Python), description, query.

### Generated Document (autoname DOC-GEN-.YYYY.-.####)

title, template, company, target\_doctype/reference\_name/party (RO), status (Draft/Finalized/Sent), print\_orientation (snapshotted so the PDF reproduces even if the template changes), payload (exact inputs JSON), rendered\_document (preview), body\_html (final rendered/edited HTML — the PDF/email source), header/footer snapshot fields, output\_action, recipients, email\_status (Not Sent/Queued/Sent/Failed), error. Controller: title defaults to template name; Sent without recipients throws. Document Template User has full CRUD but if\_owner only.

### 3. Rendering pipeline

- **Jinja** via `frappe.render_template` for body (`api.py:132`), header/footer (`_render_hf`, `api.py:104-115`), preview (`api.py:167-168`).
- **Context** (`providers.py:182-198`, `:278-286`): `data.<source>.rows / .row` (first row), `inputs.<field>`, `company`, `org` (`company_name`, `vat = Company.tax_id`, `logo`, `stamp` — images as base64 data URIs so wkhtmltopdf renders them reliably), `user` (`full_name`, `designation`, `signature` — Employee resolved via the **custom** `Employee.custom_document_user` → **User link**, not `user_id`), `today`, `today_bs`, `bs()` (AD→BS via `nepali_datetime`), `fmt` (Indian grouping, blanks zeros), `money` (2 dp, shows 0.00).
- **Inputs → params** (`providers.py:256-286`): every declared input is bound as a named SQL param; missing inputs normalized to `None` (no `KeyError`). A `fiscal_year` input auto-fills `from_date / to_date` from the Fiscal Year (`_resolve_fiscal_year`, `providers.py:242-253`) unless explicit dates given.
- **SQL sources** → `frappe.db.sql(query, params, as_dict=True)` guarded by `_assert_safe_select` (§5). **Python sources** → `Frappe safe_exec` sandbox; script reads `params`, assigns `result` (`dict = one row, list = rows`).
- **PDF** (`pdf.py:24-50`): pdfkit directly (not `frappe.utils.pdf.get_pdf`) for exact margin control. Two modes (`api._doc_pdf`, `api.py:196-219`): `include_hf=True` draws header/footer (digital/email); `include_hf=False` reserves `header_height / footer_height` mm as blank margins for pre-printed letterhead paper.
- **CSS** (`styles.py`): print-safe, no flexbox — `inline-block .row / .col-*` grid + `.bordered` tables that wkhtmltopdf renders correctly.

### 4. The desk page (`page/document_generator/document_generator.js`)

Restricted to roles System Manager / Document Template Manager / Document Template User. Flow:

1. Pick Template (Link filtered `is_active:1`).
2. Company control appears — Select from the template's companies list, else a free Company Link. Changing company clears company-scoped Link inputs.
3. Inputs render as native controls. Link inputs whose target has a `company / custom_company` field are auto-filtered to the chosen company (server detection `_company_filter_field`, `api.py:86-99`).
4. **BS date pairing**: every AD Date input auto-gets a paired "(BS)" `nepaliDatePicker` with two-way sync (`js:182-265`); the AD control is the payload source of truth.
5. **Exclusive bundles**: filling one bundle clears competing bundles in the same `exclusive_set` and drops their mandatory markers (`js:297-353`).
6. **Auto-build** — no Generate button; a 400 ms-debounced regen fires once company + all mandatory inputs are filled, calling `instantiate_document` and loading the HTML into a contenteditable A4-ish editor with a formatting toolbar (bold/italic/lists/alignment/image-insert-as-data-URI).
7. **Actions menu**: *Print (letterhead paper)* → `download_pdf(name, 0)`; *Print (with letterhead)* → `download_pdf(name, 1)`; *Email* → recipient prompt (blank = auto-resolve via `default_recipient_field`). Every action saves a Generated Document first. PDFs open in a synchronously-opened tab via blob URL (popup-block and insecure-download-banner safe).

Generated Document form: "Edit in Generator" button routes back to the page; read-only sandboxed-iframe preview from `get_generated_document_html`.  
Document Template form: "Preview" button renders with stub data in a dialog.

### 5. Security

- Every endpoint checks permissions (`check_permission`): read for `instantiate/preview/html`, write for `save`, email for `send` (`api.py:123,156,183,255,291,315`).
- **SQL injection**: inputs are always parameterized (`%(name)s`); query text is admin-authorized only. `_assert_safe_select` (`providers.py:210-221`) additionally enforces: single statement (no `;`), must start `SELECT / WITH`, and a forbidden-keyword regex (`insert/update/delete/drop/.../into outfile/load data`).
- Python sources run in `safe_exec`; Jinja context never exposes raw `frappe`.
- `_scrub` (`api.py:34-36`) strips `<script>` tags from saved HTML; rendered output is only shown in sandboxed iframes and PDFs. (Lenient regex — inline event handlers survive; mitigated by the sandbox.)
- `△` Input `reqd` is client-side only; a crafted API call can pass a partial payload (queries just receive `None`).
- wkhtmltopdf runs with `enable-local-file-access`, but the HTML is self-contained (data-URI images).

### 6. The two bundled templates (fixture)

Customer Balance Confirmation and Vendor Balance Confirmation — exported via `hooks.py:274-281`. Both: all companies, Portrait, subject "Transaction & Balance Confirmation - {{ company }}" , `header_height 32 / footer_height 16`, header = logo + company name + VAT no, footer = company tel/email/website from a `contact` source.

Inputs (both): party Link (`customer / supplier`, `reqd`) + **either**

`fiscal_year` **or** `from_date + to_date` (`exclusive_set` `period` ; picking a Fiscal Year auto-fills the dates server-side).

Data sources (4 SQL each): `party` (name/tax\_id/address from Customer/Supplier), `balances` (opening/closing from GL Entry — `SUM(debit-credit)` before `from_date` / up to `to_date`, incl. `is_opening`), `txn` (period taxable / VAT / TDS / total from Sales Invoice or Purchase Invoice — customer TDS is literal 0; vendor uses `SUM(custom_total_tds_amount)`), `contact` (Company phone/email/website).

Body: BS-date letterhead, "To" block, underlined subject with FY label or BS date range, figures table (vendor version adds a TDS column), 7-day confirmation paragraph, two-column signature block using `user.signature`, `org.stamp` (columns swapped between the two templates).

## 7. Authoring a new template (admin runbook)

---

1. Role: Document Template Manager (or System Manager). New Document Template.
2. Name, Companies (empty = all), Orientation, Is Active.
3. Inputs: fieldname/label/type/options/reqd; `exclusive_group` + `exclusive_set` for either/or choices; name an input `fiscal_year` to get auto date-fill; Date inputs get BS pickers automatically.
4. Data sources: SQL (single read-only SELECT with `%(input)s` params — columns become `data.<name>.row.<col>` / iterable `.rows`) or Python ( `params` in, `result` out).
5. Body HTML+Jinja using the context of `$3`; use `.bordered` tables and the `.row/ .col-*` grid for print-safe layout.
6. Header/Footer HTML + heights (mm) matched to the pre-printed letterhead band; blank falls back to the linked Letter Head.
7. Email subject + Default Recipient Field (+ Target DocType for auto-resolution).
8. One-time site assets (created by patches): `Company.custom_document_stamp`, `Employee.custom_signature_image`, `Employee.custom_document_user` (the signatory User link).
9. Save (validates Jinja) → **Preview** (stub data) → generate from the Document Generator page.

## 8. Patches

---

- `create_document_generator_roles` — roles "Document Template Manager" / "Document Template User" (pre-model-sync).
- `cleanup_document_generator_sections` — removes the obsolete block-based Section doctypes (the design moved to a single HTML body; a stale `sections.pyc` may remain).
- `create_document_generator_image_fields` — Company stamp + Employee signature Attach Image fields.
- `create_document_generator_signatory_field` — `Employee.custom_document_user`.

Tests: `test_document_template.py` (Jinja validation rejects bad templates), `test_generated_document.py` (Sent-without-recipients throws).

# Reports & Portal Pages — Technical Reference

Chapter 8 of the technical documentation. Audience: developers.  
User-facing guide: [../user\\_guide/07-reports.md](#)  
The HR/BS attendance reports are detailed in chapter 3 §7; this chapter catalogues the financial/sales/purchase suite and the portal pages.

All reports are **Script Reports** under `avinash_group_app/report/`, run synchronously, and mostly build their own total rows. Company/party/item filter dropdowns come from whitelisted `get_company_*` helpers scoped by `custom_company`. Numbers use Indian lakh/crore grouping (`_fmt_inr`), NPR.

## 1. Cross-cutting print/PDF infrastructure

- **Shared BS-date filters:** reports with plain Date filters get a BS "Miti" twin input + " Select Month" (whole-BS-month) from `rdp_common_app/public/js/report_nepali_date.js` (external app hook). Used by Gas Purchase, Sales Stock Ledger, Party Ledger Summary.
- **`public/js/report_print_orientation.js`** (global, query-report routes only): `window.askPrintOrientation` dialog; wraps every "Download PDF" inner button to append `&orientation=`; shims `window.open` to inject `report_print_portrait.css` + fit-to-width for print popups; shims `frappe.render_pdf` to inline black-grid styling plus an optional per-report footer (`window.__rdpPrintFooter`, used by Advance Tax TDS's Prepared/Checked/Verified signature block).
- **Three PDF patterns:** **A** — report-local `*_pdf.html` template with whitelisted `download_pdf` (re-runs `execute`, paginates in Python, in-body "Page X of Y"; Print calls the same PDF inline): Party Ledger, Party Ledger Summary, Sales/Purchase Register, Receipt Register, Loan Summary, One Lakh Above, the three Sales Analysis reports. **B** — shared helper only: Advance Tax TDS, Loan Summary dialogs. **C** — stock Frappe print.

## 2. Report catalogue

(Full field-level details are in the code; this is the authoritative summary.  
"Miti" = BS date sourced from `custom_*_miti` custom fields.)

### Compliance (Nepal IRD)

| Report                      | Purpose                                                                                                                                                       | Source                                                                                                                     | Key filters                                         |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Advance Tax TDS Details     | अग्रिम कर TDS withholding statement per supplier per Tax Withholding Category, sectioned by IRD खाता नं with Nepali headers and signature footer              | PI+items ⇄ Supplier; category parsed from <code>custom_tax_withholding_category_custom</code> via regex (rate/khata/title) | company, dates                                      |
| One Lakh Above Transactions | parties whose taxable or exempt totals ≥ NPR 100,000 (IRD annexure); taxable = item amounts with <code>custom_vat_amount≠0</code> ; returns subtract          | SI+items ∪ PI+items                                                                                                        | company, dates                                      |
| Sales Register              | खरिद/बिक्री VAT register: per SI, buckets tax-free / export / taxable / VAT via <code>custom_vat_apply_on</code> + territory                                  | SI ⇄ SII ⇄ Customer                                                                                                        | company, dates*, customer, is_return                |
| Purchase Register           | per PI: tax-free / taxable / import / capitalized buckets ( <code>custom_vat_apply_on</code> × <code>is_fixed_asset</code> × supplier territory) + VATs + qty | PI ⇄ PII ⇄ Supplier ⇄ Item                                                                                                 | company, dates*, supplier, purchase_type, is_return |

### Ledgers & treasury

| Report                        | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Party Ledger (1278 lines)     | Tally-style party statement: opening → vouchers with running balance → closing; BS Miti per doctype; custom voucher display names ( <code>custom_branch_name</code> / <code>custom_name</code> ); optional item-level detailed mapping, remarks; JEs split per account; multi-party grouped mode; gated <code>exclude_account_patterns</code> (used by the Customer Statement portal to hide deposit/security accounts); Contract-Form JEs included by default. Landscape PDF pattern A. <b>Engine of the /customer_statement portal</b> |
| Party Ledger Summary          | one line per party (opening/debit/credit/closing); Super Summary or Group Wise layouts; custom Excel export with Dr/Cr columns                                                                                                                                                                                                                                                                                                                                                                                                           |
| Net Position of Cash and Bank | every leaf account under "Cash and Cash Equivalent": opening / receipts / payments / closing; PDC gating; multi-currency suppression                                                                                                                                                                                                                                                                                                                                                                                                     |
| Loan Summary                  | borrowings matrix: loan types (children of Short/Long Term Borrowings) × company columns, subtotals, ratio row, Show Details expands sub-accounts. <code>Δ</code> from_date collected but unused                                                                                                                                                                                                                                                                                                                                         |
| Receipt Register              | customer receipts (PE type Receive) in 3 views: Date Wise / Customer Wise (banner headers) / Summary; cheque no "1" suppressed                                                                                                                                                                                                                                                                                                                                                                                                           |

### Financial statements

| Report                                                 | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consolidated Financial Statement Hierarchy (734 lines) | consolidated BS/P&L/Cash-Flow across an <b>arbitrary company set</b> (no parent group company needed): monkey-patches ERPNext's consolidated engine, replaces per-root GL fetches with one aggregated SQL (needs the <code>fin_stmt_agg_index</code> from patch <code>add_gl_entry_fin_stmt_agg_index</code> ), adds row Total column, common-accounts filtering, depth cap, fixes an ERPNext cash-flow bug. Has its own <code>USER_DOCS.md</code> |
| Profit and Loss Hierarchy                              | single-company ERPNext P&L + "Account Hierarchy Level" control that blanks group rows above depth N; Report/Growth/Margin views                                                                                                                                                                                                                                                                                                                    |

### Sales analysis & industry

| Report                               | Purpose                                                                                                                                                                                                                                                            |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SA — Customer wise Summary           | one row per customer: qty, gross (incl. excise), optional return/net columns                                                                                                                                                                                       |
| SA — Customer wise Details           | Customer → product (UOM) drill-down with product/customer/grand totals                                                                                                                                                                                             |
| SA — Product wise Invoice Details    | Product → customer → invoice/return rows; <code>build_rows(include_agent)</code> reused by the portal (agent rows dropped there). ⚠ Frappe drops falsy filters — unchecked Include Return never reaches the server (treated OFF); JS default keeps it ON initially |
| Gas Purchase Report                  | LPG procurement register per Store Receipt combining gas Pls, PRs, Service Charge ICP/NA invoices; refinery/tanker/IOC-challan columns; BS-miti period filtering in Python; voucher-code parsing ( <code>NGK-ICP-00218-82/83</code> → 218)                         |
| Sales Stock Ledger                   | sales-driven stock movement (submitted SIs of stock items), Detail/Summarized modes, BS dates via <code>nepali_datetime</code> , sales/return merge, colored voucher badges. Fix history: <code>docs/sales_stock_ledger_fixes.md</code>                            |
| Avinas Vehicle Expense               | vehicle-wise Fuel/Repair/Others buckets from PI items + JE lines matched by account-name LIKE patterns; vehicle = <code>custom_subtype</code> shown as license plate. Doc: <code>docs/vehicle_expense_report.md</code>                                             |
| Custom Supplier Quotation Comparison | pivoted item × supplier price comparison with net/VAT/discount/invoice-amount summary rows, near-expiry coloring, "Select Default Supplier" tool, PO→RFQ auto-fill                                                                                                 |

Missing

`customer_vendor_ledger_summary/` contains only a stale `__pycache__` — the report has **no source** and is effectively deleted. (The Customer Ledger Summary referenced by credit-control code is ERPNext's standard report.)

3. Common gotchas

- Sales/Purchase Register interpolate `company / customer` strings into SQL (dates are bound params) — keep filter values from link fields only.
- One row per voucher × party in Party Ledger → wide ranges get large; enrichment is batched in IN-chunks of 500.
- Reports stay under Frappe's 15 s prepared-report auto-switch by hitting indexed columns first; the daily-entry and fin-stmt indexes come from patches.

4. HR/BS reports

Monthly Attendance BS, Monthly Attendance Summary BS, Work On Holiday BS, Yearly Leave Details BS, Avinas Salary Statement — see chapter 3 §7.

5. Portal pages ( `templates/pages/` )

All require login; customer pages require the **Customer** role and resolve the user's customers via `Portal User`, re-validating every AJAX request (`_resolve_request` — companies/customers outside the user's set are rejected; an empty party list is filled with the user's own customers, never "all").

| Route                                      | Purpose                                                                                                                                                                                                                                                 |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/customer_statement</code>           | the customer's own Party-Ledger statement + Portrait PDF; passes <code>exclude_account_patterns</code> so deposit/security accounts (313101/313102/313201 name patterns) never show or affect balances; dual AD+BS date pickers (AD is source of truth) |
| <code>/product_wise_invoice_details</code> | product → invoice drill-down for the customer's own accounts; reuses the report's <code>build_rows(include_agent=False)</code> ; default company prefers Nepal Gas Udhog (Karnali)                                                                      |
| <code>/place_order</code>                  | LP Gas Sales Order placement: fixed LP Gas item, cylinder-size UOMs, rates via Item Price, client VAT math, <code>create_sales_order</code> re-validates server-side and submits with <code>ignore_permissions</code>                                   |
| <code>/rfq/&lt;name&gt;</code>             | supplier RFQ portal — see chapter 4 §9                                                                                                                                                                                                                  |

Docs: `docs/product wise invoice customer portal.md`.

# Custom Doctypes Reference — avinashgroup\_app

Chapter 9 of the technical documentation. The module "Avinash Group App" owns **41 doctypes: 28 ship as files** under `avinash_group_app/ddoctype/` (below), and **13 are DB-only** (created via the Customize UI, not in the repo) — those are documented in [chapter 11 Part C](#).  
For the **full field list** of every file-based doctype see [chapter 11 Part B](#);  
for exported custom fields on **core** ERPNext doctypes see [chapter 11 Part A](#).

## Dynamic Approval (chapter 2)

| Doctype                           | Kind                             | Role                                                           |
|-----------------------------------|----------------------------------|----------------------------------------------------------------|
| Dynamic Approval Setting          | master (System Manager)          | per doctype+company approval config; Setup Workflow button     |
| Dynamic Approval Match Criteria   | child                            | <code>section + field_name = field_value</code> equality rules |
| Dynamic Approval Fixed Approver   | child                            | ordered fixed approvers per section                            |
| Dynamic Approval Approver         | child (injected on targets)      | the requester's per-document hierarchy rows                    |
| <i>(Dynamic Approval History)</i> | child, <b>created at runtime</b> | audit-log rows on target docs (no source JSON)                 |

## Biometric / Attendance (chapter 3)

| Doctype                          | Kind                | Role                                                                          |
|----------------------------------|---------------------|-------------------------------------------------------------------------------|
| Biometric Device                 | master              | one per device/feed: serial (unique), company, sync stats, heartbeat alerting |
| Biometric Device Command         | master              | outbound-pollled command queue (force_sync / test_connection)                 |
| Biometric Device Alert Recipient | child               | alert emails                                                                  |
| Attendance Fix                   | submittable         | background attendance repair for a shift + date range                         |
| Attendance Fix Device            | child               | device filter rows                                                            |
| Employee Attendance Allowance    | child (on Employee) | per-employee allowance rate/eligibility overrides                             |

## Taxes / Stock / Numbering (chapter 4)

| Doctype                      | Kind            | Role                                                                                |
|------------------------------|-----------------|-------------------------------------------------------------------------------------|
| Voucher Number Settings Item | child           | per-doctype <code>voucher_no_digits</code> zero-pad width                           |
| Branch Wise Warehouse        | child (on Item) | branch → buying/selling warehouse mapping (operative fields are site custom fields) |
| Stock Adjustment             | submittable     | quantity-only stock correction (rate must be 0); engine patch preserves stock value |
| Stock Adjustment Item        | child           | item/warehouse/qty rows                                                             |

## CBMS (chapter 5)

| Doctype          | Kind            | Role                                                           |
|------------------|-----------------|----------------------------------------------------------------|
| CBMS Config      | one per company | enable flag, go-live date, IRD credentials, retry batch sizes  |
| CBMS Bill        | auto-created    | one per submitted Sales Invoice; sync status/attempts/response |
| CBMS Bill Return | auto-created    | one per submitted Sales Return (credit note)                   |

## Access control / Audit (chapter 6)

| Doctype                           | Kind            | Role                                                                   |
|-----------------------------------|-----------------|------------------------------------------------------------------------|
| Fiscal Year Access Control        | one per user    | full-access flag or per-doctype fiscal-year grants                     |
| User Fiscal Year Access           | child           | doctype + fiscal year (or row-level full access)                       |
| Company Filter Config             | one per doctype | which fields are company-filtered; <code>company_field</code> selector |
| Company Filter Field              | child           | plain / child-table / dynamic-link field rows                          |
| User Daily Entry Summary Settings | Single          | tracked doctypes for the daily entry report                            |
| User Daily Entry Summary Doctype  | child           | one tracked doctype per row                                            |

## Document Generator (chapter 7)

| Doctype                       | Kind                                | Role                                                                            |
|-------------------------------|-------------------------------------|---------------------------------------------------------------------------------|
| Document Template             | master                              | HTML+Jinja letter template: inputs, data sources, header/footer, email settings |
| Document Template Input       | child                               | generation-time input fields (incl. exclusive groups/sets)                      |
| Document Template Data Source | child                               | SQL (guarded SELECT) or Python (safe_exec) source                               |
| Document Template Company     | child                               | company scoping                                                                 |
| Generated Document            | master ( DOC-GEN- .YYYY. - .##### ) | persisted rendered/edited output; status, payload snapshot, email status        |

## Custom fields on core doctypes ( `avinash_group_app/custom/*.json` )

Exported custom-field bundles (all also carry the audit block — `custom_created_by/on` , `custom_modified_by` , plus `custom_abbr` /

custom\_company on parents):

| File                              | Highlights                                                                                                                                                                              |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| customer.json                     | credit fields custom_amount_limit, custom_bill_count, custom_days_limit                                                                                                                 |
| sales_invoice.json (28 fields)    | excise/VAT totals, rounding adjustment, custom_invoice_miti + BS dates, vehicle no                                                                                                      |
| sales_invoice_item.json           | custom_vat_apply_on/rate/amount, custom_excise_value, custom_total                                                                                                                      |
| purchase_invoice.json (40 fields) | custom_document_no, custom_name (Voucher No), custom_purchase_type, TDS/VAT/excise totals, custom_tax_withholding_category_custom, IOC/PDO/refinery/store-receipt/miti/transport fields |
| purchase_invoice_item.json        | VAT/TDS/excise item fields + custom_subtype (Vehicle)                                                                                                                                   |
| purchase_order(.item).json        | approval fields ( custom_reason ...), VAT/TDS/excise                                                                                                                                    |
| purchase_receipt(.item).json      | custom_receipt_type, voucher fields, IOC/miti, VAT/TDS/excise                                                                                                                           |
| supplier_quotation(.item).json    | custom_preferred_quotation, VAT/TDS/excise                                                                                                                                              |

Many more custom fields are installed programmatically (not exported):  
AuditFieldsManager audit fields across ~85 doctypes, attendance-allowance fields (patch), Document Generator image/signatory fields (patches), CBMS custom\_reason\_for\_return (patch), Employee Checkin / Shift Type company fields (patches), and the Branch Wise Warehouse operative fields (site DB only).  
utils/export\_field.py can dump the live inventory to Excel.

# Complete File Index — Every File, Where It Is & Why

Chapter 10 of the technical documentation. The exhaustive map: every source file in the app, what it is, why it exists, and whether it is **wired** (active in `hooks.py` / `__init__.py`), **console** (run manually), or **dead/legacy** (present but unreferenced). `__init__.py` package markers and `__pycache__` are omitted. Deep behavior lives in chapters 2-9; this chapter is the index that guarantees nothing is missed.

Legend: ● wired · console/setup only ● dead/legacy · data/asset · test

## Root of the app ( apps/avinashgroup\_app/ )

| File                                 | What / why                                                                                                                                                                                                                                                                                           | Status |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| README.md                            | Frappe boilerplate: install + pre-commit note, MIT                                                                                                                                                                                                                                                   |        |
| license.txt                          | MIT license text (placeholders unfilled)                                                                                                                                                                                                                                                             |        |
| pyproject.toml                       | package metadata; <b>dependency nepali-datetime</b> (BS dates for CBMS); ruff config (line 110, tab indent, double quotes)                                                                                                                                                                           |        |
| .pre-commit-config.yaml              | pre-commit hooks: ruff (import-sort + lint + format), prettier (js/vue/scss), eslint, standard checks                                                                                                                                                                                                |        |
| .editorconfig                        | LF, UTF-8, tab size 4 (py/js), 2-space for JSON                                                                                                                                                                                                                                                      |        |
| .gitignore                           | .pyc, __pycache__, node_modules, egg-info, swap files                                                                                                                                                                                                                                                |        |
| CURRENT_CHANGES_DOCUMENTATION.txt    | ⚠ <b>stale</b> mid-session scratch log (Apr 2026) about a dynamic-approval default-section idea + a reverted JE search handler — superseded by chapter 2                                                                                                                                             | ●      |
| IMPLEMENTATION_SUMMARY.md            | fiscal-year access implementation writeup (May 2026) — superseded by chapter 6                                                                                                                                                                                                                       | ●      |
| FISCAL_YEAR_ACCESS.md                | fiscal-year access user/dev guide — superseded by chapter 6; ⚠ its "System User bypass" claim is wrong (it's Administrator/System Manager)                                                                                                                                                           | ●      |
| testing.md                           | * <b>current &amp; unique</b> — 38 KB adversarial test plan + partial live run for the naming-series engine (60+ cases, race/TOCTOU risks, real findings: duplicate <code>custom_name</code> in PI/PR with no unique index, Grihalaxmi bypass, dotted-key loss). The authoritative testing reference |        |
| Document_Numbering_System.pdf        | design/spec PDF for voucher numbering (belongs under docs/ ideally)                                                                                                                                                                                                                                  |        |
| wkhtmltox_0.12.6.1-3.jammy_amd64.deb | ⚠ 17 MB vendored wkhtmltopdf installer — should not be in the repo; make it an install step                                                                                                                                                                                                          |        |
| create_test_sq.py                    | throwaway seed script — creates 3 Supplier Quotations for Grihalaxmi; run via <code>bench execute</code>                                                                                                                                                                                             |        |

## Package root ( avinashgroup\_app/ )

| File                             | What / why                                                                                           | Status |
|----------------------------------|------------------------------------------------------------------------------------------------------|--------|
| hooks.py                         | <b>the wiring hub</b> — all events, includes, overrides, scheduler, fixtures (chapter 1 §3)          | ●      |
| __init__.py                      | app version + applies <code>regional_deletion_override.apply_patch()</code> at import (chapter 4 §5) | ●      |
| modules.txt                      | single module "Avinash Group App"                                                                    |        |
| patches.txt                      | migration patch order (see Patches table below)                                                      |        |
| create_test_sq.py (package copy) | duplicate of the root seed script                                                                    |        |
| test_data.py, test_stock_adj.py  | ad-hoc developer test scripts (not the automated suite)                                              |        |

## custom\_code/ — server business logic

### Top level

| File                                  | What / why                                                                                                                              | Chapter | Status |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------|--------|
| dynamic_approval.py                   | the whole configurable approval engine ( * -hooked)                                                                                     | 2       | ●      |
| workflow_admin_bypass.py              | overrides <code>get_transitions / apply_workflow</code> ; Administrator bypass; fires <code>before_workflow_action</code> for all users | 2       | ●      |
| workflow.py                           | <code>set_reject_reason</code> — stores reject reason to <code>custom_reason</code> or a comment                                        | 2       | ●      |
| workflow_material_request.py          | thin legacy alias of <code>set_reject_reason</code>                                                                                     | 2       | ●      |
| update_workflows.py                   | <code>run()</code> rebuilds all Dynamic Approval workflows                                                                              | 2       |        |
| create_log_doctype.py                 | bootstraps the runtime <b>Dynamic Approval History</b> child doctype                                                                    | 2       |        |
| fiscal_year_filter.py                 | fiscal-year access control (list SQL, <code>has_permission</code> , <code>get_list</code> override)                                     | 6       | ●      |
| excise_ledger.py                      | only <code>modify_gl_entries</code> (PI before <code>submit</code> ) party-tags TDS-payable GL; rest is dead GLMI code                  | 4       | ●+●    |
| stock_revaluation.py                  | backdated-PI repost notice + nightly repost helpers (nightly job not scheduled)                                                         | 4       | ●+     |
| stock_health_check.py                 | <code>run()</code> stock diagnostics                                                                                                    | 4       |        |
| vehicle_mandatory.py                  | Vehicle mandatory on vehicle-expense JE/PI lines                                                                                        | 4       | ●      |
| regional_deletion_override.py (+ .md) | lets Admin/System Manager delete submitted Nepal-company docs                                                                           | 4       | ●      |
| regional_deletion_override.md         | short design note for the above                                                                                                         | —       |        |
| override_rounding.py                  | 71 KB excise-inclusive tax/rounding recompute — unwired, 57% commented                                                                  | 4       | ●      |
| voucher_no_console.py                 | console backfills of <code>custom_name</code> (PI/JE)                                                                                   | 4       |        |
| api.py                                | fiscal-year/naming legacy helpers + <code>create_created_by_and_created_on_fields</code> installer/backfill                             | 4/6     | ●      |
| custom_customer.py                    | misnamed — locks company on two <code>test</code> doctypes                                                                              | 4       |        |
| custom_itemname.py                    | <code>CustomItem.autoname</code> override — never registered                                                                            | 4       | ●      |

custom\_code/common/

| File                                   | What / why                                                                              | Status |
|----------------------------------------|-----------------------------------------------------------------------------------------|--------|
| <code>purchase_taxes_handler.py</code> | VAT/Excise/TDS + warehouse forcing for PI/PO/PR/SQ (+MR/RFQ)                            | ●      |
| <code>selling_taxes_handler.py</code>  | VAT/Excise for Quotation/SO/DN                                                          | ●      |
| <code>purchase_taxes_mapper.py</code>  | <code>get_taxes_from_source</code> (JS-used); <code>after_mapping_*</code> unreferenced | ●+●    |

`custom_code/SalesInvoice/`

| File                                  | What / why                                                                              | Status |
|---------------------------------------|-----------------------------------------------------------------------------------------|--------|
| <code>salesinvoice_taxes.py</code>    | SI VAT/Excise + selling <code>validate_*</code> warehouse forcing (the live SI handler) | ●      |
| <code>print_count.py</code>           | IRD print-copy counter on <code>before_print</code> (chapter 4 §5a)                     | ●      |
| <code>salesinvoice_customer.py</code> | customer credit checks — called only by unloaded <code>si.js</code>                     | ●      |
| <code>credit_control.py</code>        | alternate credit engine — dead + <code>debugpy</code> breakpoint trap                   | ●      |

`custom_code/Override/`

| File                                   | What / why                                                                                                   | Status |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------|--------|
| <code>overrides.py</code>              | 9 doctype class overrides (warehouse restore + soften warehouse-mandatory)                                   | ●      |
| <code>naming_series.py</code>          | 31 KB voucher-numbering engine ( <code>custom_name</code> , <code>custom_document_no</code> , branch series) | ●      |
| <code>get_item_details.py</code>       | wraps <code>get_item_details</code> (item-price patch + cmd strip)                                           | ●      |
| <code>auto_insert_item_price.py</code> | <code>before_request</code> patch stamping company on auto Item Prices                                       | ●      |
| <code>query_report.py</code>           | report "Add Column" shows Link titles not ids                                                                | ●      |
| <code>company_field_lock.py</code>     | <code>validate_company_field_lock</code> (wired via audit) + read-only property setter                       | ●+     |
| <code>setup_company_lock.py</code>     | console tools for the company lock                                                                           |        |

`custom_code/CBMS/` (chapter 5)

| File                                | What / why                                                                | Status |
|-------------------------------------|---------------------------------------------------------------------------|--------|
| <code>sales_invoice_hooks.py</code> | <code>on_submit/before_cancel</code> ; payload mapping; CBMS doc creation | ●      |
| <code>api_client.py</code>          | HTTP client to IRD; payload builders; result recording                    | ●      |
| <code>scheduler.py</code>           | 5-min retry + reconcile jobs                                              | ●      |
| <code>utils.py</code>               | BS-date / fiscal-year / invoice-number helpers                            | ●      |

`custom_code/globalfilter/` (chapter 6)

| File                         | What / why                                                          | Status |
|------------------------------|---------------------------------------------------------------------|--------|
| <code>globalfilter.py</code> | company-matching validation + filtered link searches + config cache | ●      |

`custom_code/document_generator/` (chapter 7)

| File                      | What / why                                                     | Status |
|---------------------------|----------------------------------------------------------------|--------|
| <code>api.py</code>       | all whitelisted endpoints (instantiate/save/pdf/email/preview) | ●      |
| <code>providers.py</code> | context builders, SQL/Python source runners, safety guards     | ●      |
| <code>pdf.py</code>       | pdfkit/wkhtmltopdf wrapper (margin control)                    | ●      |
| <code>styles.py</code>    | print-safe HTML wrapper + CSS                                  | ●      |

`custom_code/payment_entry/` and `purchase_invoice/`

| File                                                        | What / why                                   | Status |
|-------------------------------------------------------------|----------------------------------------------|--------|
| <code>payment_entry/cheque_bounce.py</code>                 | reversing GL entry on cheque bounce (button) | ●      |
| <code>purchase_invoice/purchase_invoice_taxes_tds.py</code> | superseded alternate PI tax handler          | ●      |

`biometric/` (chapter 3)

| File                                     | What / why                                                                                         | Status |
|------------------------------------------|----------------------------------------------------------------------------------------------------|--------|
| <code>iclock.py</code>                   | ZKTeco ADMS <code>/iclock/*</code> page renderer                                                   | ●      |
| <code>api.py</code>                      | <code>receive_attendance</code> bridge endpoint                                                    | ●      |
| <code>utils.py</code>                    | <code>process_attendance_records</code> , checkin reconciliation, <code>assert_known_device</code> | ●      |
| <code>attendance_override.py</code>      | shift-deviation fields, late half-day, checkin↔attendance reconcile                                | ●      |
| <code>employee.py</code>                 | unique device-id per company                                                                       | ●      |
| <code>heartbeat.py</code>                | <code>ping</code> + hourly dead-bridge alerting                                                    | ●      |
| <code>bridge_commands.py</code>          | poll/report command tunnel                                                                         | ●      |
| <code>test_attendance_pipeline.py</code> | the main automated test suite                                                                      |        |

`payroll/`

| File                                 | What / why                                                                    | Status |
|--------------------------------------|-------------------------------------------------------------------------------|--------|
| <code>attendance_allowance.py</code> | holiday flag hook + attendance-driven Additional Salary engine (chapter 3 §6) | ●      |

`utils/`

| File                  | What / why                                                                                                | Status |
|-----------------------|-----------------------------------------------------------------------------------------------------------|--------|
| audit_file_manager.py | audit-field engine + <code>AuditEventManager</code> — the base <code>doc_events</code> map (chapter 6 §3) | ●      |
| fiscal_year_utils.py  | <code>get_default_fiscal_year()</code> BS FY label helper (JS-used)                                       | ●      |
| export_field.py       | export all custom fields to Excel                                                                         |        |

scripts/

| File                    | What / why                                                              | Status |
|-------------------------|-------------------------------------------------------------------------|--------|
| cleanup_audit_fields.py | remove audit/naming fields + property setters from a doctype            |        |
| pad_custom_name.py      | zero-pad <code>custom_name</code> numbers to 6 digits (dry-run default) |        |

templates/pages/ — portal & web pages (chapters 4 §9, 8 §5)

| File                                    | What / why                                                                                                                                                 | Status |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| customer_statement.py + .html           | <code>/customer_statement</code> — customer's own Party-Ledger statement + PDF; <code>_resolve_request</code> security; excludes deposit/security accounts | ●      |
| product_wise_invoice_details.py + .html | <code>/product_wise_invoice_details</code> — reuses the report's <code>build_rows(include_agent=False)</code> ; per-customer security                      | ●      |
| place_order.py + .html                  | <code>/place_order</code> — LP Gas Sales Order placement                                                                                                   | ●      |
| rfq.py + .html                          | supplier RFQ portal override ( <code>create_supplier_quotation</code> ) + attachment upload                                                                | ●      |

public/js/ — client scripts

Loaded globally via `app_include_js` (●g) or per-form via `doctype_js` (●f);  
`si.js/purchase_invoice.js` are **not loaded** (●).

| File                                      | Load | What / why                                                                    |
|-------------------------------------------|------|-------------------------------------------------------------------------------|
| <code>fiscal_year_cache.js</code>         | ●g   | once-a-day client cache of the default BS fiscal year                         |
| <code>approval_workflow_common.js</code>  | ●g   | hides Approve/Reject for non-approvers; reject-reason dialog                  |
| <code>approval_workflow_auto.js</code>    | ●g   | generic approval progress banner + reject binding                             |
| <code>approval_field_visibility.js</code> | ●g   | disables form for non-approvers; locks approved rows; driver-field visibility |
| <code>sales_invoice.js</code>             | ●g   | SI VAT UI + selling warehouse injection + due date                            |
| <code>purchase_taxes_common.js</code>     | ●g   | PI/PO/PR/SQ VAT/TDS UI, warehouse injection, source population                |
| <code>selling_taxes_common.js</code>      | ●g   | Quotation/SO/DN VAT UI                                                        |
| <code>sales_warehouse_common.js</code>    | ●g   | Quotation/SO/DN warehouse injection                                           |
| <code>global_filter.js</code>             | ●g   | row-level company-mismatch guard on item/supplier child rows                  |
| <code>company_filter.js</code>            | ●g   | the company dropdown-filter engine                                            |
| <code>payment_entry.js</code>             | ●g   | cheque-bounce button + party scoping                                          |
| <code>auto_update_document_no.js</code>   | ●g   | auto-fills <code>custom_document_no</code> on PR/PI/PE/JE                     |
| <code>report_print_orientation.js</code>  | ●g   | report Portrait/Landscape dialog + PDF styling shims                          |
| <code>vehicle_mandatory.js</code>         | ●g   | client mirror of vehicle-mandatory validation                                 |
| <code>material_request.js</code>          | ●f   | MR warehouse forcing + reject dialog                                          |
| <code>pi.js</code>                        | ●f   | PI vehicle picker ( <code>custom_subtype</code> )                             |
| <code>journal_entry.js</code>             | ●f   | JE vehicle picker                                                             |
| <code>attendance.js</code>                | ●f   | Attendance "Checkin Log" table                                                |
| <code>payroll_entry.js</code>             | ●f   | "Calculate Attendance Allowances" button                                      |
| <code>party_duplicate_check.js</code>     | ●f   | Customer/Supplier duplicate name/tax_id warning                               |
| <code>party_default_account.js</code>     | ●f   | Customer/Supplier default-accounts row                                        |
| <code>item_default_account.js</code>      | ●f   | Item default row                                                              |
| <code>si.js</code>                        | ●    | credit-control UI — <b>not loaded</b> (feature dormant)                       |
| <code>purchase_invoice.js</code>          | ●    | <b>not loaded</b>                                                             |

public/css/

| File                                   | What / why                                               | Status      |
|----------------------------------------|----------------------------------------------------------|-------------|
| <code>desk_focus.css</code>            | orange focus ring + dropdown highlight                   | ●g          |
| <code>report_print_portrait.css</code> | portrait print CSS injected dynamically on report routes | ● (dynamic) |

fixtures/

| File                                | What / why                                                     | Status |
|-------------------------------------|----------------------------------------------------------------|--------|
| <code>document_template.json</code> | the two balance-confirmation Document Templates (chapter 7 §6) |        |

translations/

| File                | What / why                                                              | Status |
|---------------------|-------------------------------------------------------------------------|--------|
| <code>en.csv</code> | single override: "Avinas Vehicle Expense" → "Nepal Gas Vehicle Expense" |        |

patches/ — one line each (order in `patches.txt` )

**Pre-model-sync:** `create_document_generator_roles` (roles),  
`rename_vocher_number_settings` (fix misspelled doctype).

**Post-model-sync:** `setup_attendance_allowance` (Salary Component rule fields), `v1_add_biometric_device_serial` (backfill serial),  
`user_fiscal_year_access_fields` (deprecated no-op),  
`migrate_fiscal_year_access_control` (legacy→new doctype),  
`cleanup_document_generator_sections` (drop old block doctypes),  
`create_document_generator_image_fields` (stamp/signature),  
`create_document_generator_signatory_field` (Employee→User signatory),  
`leave_application_dynamic_approval` (Leave App under approval),  
`enable_track_changes_audited_doctypes` (Version history),  
`add_creation_index_daily_entry` ((owner, creation) index),  
`create_cbms_custom_fields` (custom\_reason\_for\_return),  
`add_gl_entry_fin_stmt_agg_index` (consolidated-report index),  
`vehicle_mandatory_property_setters` (run twice: add then remove setters),  
`add_company_to_employee_checkin`, `company_scoped_attendance_device_id`  
(drop global-unique device-id index), `add_company_to_shift_type`,  
`add_sales_invoice_print_count` ( custom\_print\_count for IRD copies).  
`seed_company_filter_config` is present but **commented out** in patches.txt.

## Doctypes, custom fields & reports

---

- Custom doctypes: **28 file-based** (folders under `avinash_group_app/doctype/` ) + **13 DB-only** (in the module but created via Customize UI — *not in the repo*, so `git clone` won't recreate them). Full reference: [chapter 9](#) + [chapter 11 Parts B & C](#).  
The 13 DB-only ones: Purchase Type, Receipt type, Payment - Receipt Type, JV Type (voucher-code lists); Vehicle List (Account→Vehicle map); Purchase Order Request Approver, Material Request Approver (approval tables); Amount Calculation for sales invoice, Invoice Calculation (SI calc tables); Sub-Ledger Category + Sub-ledger table; Employee Cost Center Manager; Material request table.
- Custom fields on core doctypes ( `custom/*.json` ):  
[chapter 11](#).
- Reports (26) + `page/document_generator/` : [chapter 8](#)  
and the [reports appendix](#).

`.claude/settings.local.json`

---

Local Claude Code settings for this app dir — not application code.

## Field Reference — Custom Fields & Doctype Fields

---

Chapter 11 of the technical documentation. The exhaustive field-level reference: **Part A** every custom field added to core ERPNext doctypes ( `custom/*.json` ), **Part B** every field of the 28 custom doctypes. Layout-only Section/Column Breaks are summarized as counts (they carry no data); every data field is listed. RO = read-only, NC = no\_copy, "miti" = Nepali Bikram Sambat date.

The cross-cutting field families (why they exist) are:

- **Audit block** — `audit_section`, `custom_created_by`, `custom_created_on`, `custom_modified_by` (all RO/NC, system-stamped by the audit engine, chapter 6 §3). Present on Customer, Sales Invoice, Purchase Invoice/Order/Receipt, Supplier Quotation.
- **VAT scheme** — item rows: `custom_vat_apply_on` (VAT 13% / VAT 0% / Amount), `custom_vat_rate` (RO), `custom_vat_amount`; parents: `custom_total_vat_amount` (chapter 4 §1).
- **Excise scheme** — item rows: `custom_excise_value`, `custom_total`; parents: `custom_total_excise_amount`, `custom_total_amount_including_excise`.
- **TDS scheme** (purchase side only) — `custom_tds_apply_on` / `custom_tds_rate` / `custom_tds_amount`; parents `custom_total_tds_amount`; `custom_tax_withholding_category_custom` on PI.
- **Voucher numbering** — `custom_abbr` / `custom_company_abbr`, `custom_document_no`, `custom_name` (Voucher No.), `custom_p_type_code` (chapter 4 §4).
- **Nepali/IRD** — `custom_fiscal_year`, the `*_miti` BS-date fields, AD/BS pairs, `custom_print_count` (SI, added by patch — chapter 4 §5a).
- **Petroleum/IOC** — `custom_pdo_no`, `custom_refinery`, `custom_ioc_challan_no` / date / miti, `custom_name_of_transportor`, `custom_vehicle_no`, item `custom_subtype` (→ Vehicle).
- **Credit control** (Customer) — `custom_amount_limit`, `custom_days_limit`, `custom_bill_count`.

Total: **182 custom fields** across 11 core doctypes (+2 doctypes carry only property setters). Note `custom_print_count` is added by patch, not exported here, so it isn't in the counts below.

---

# Part A — Custom fields on core doctypes

## Customer (11 fields, 4 property setters)

| Field               | Type                 | Flags                      | Why                                                       |
|---------------------|----------------------|----------------------------|-----------------------------------------------------------|
| custom_company      | Link → Company       | reqd                       | ties customer to a group company; drives abbr + filtering |
| custom_abbr         | Data                 | hidden, fetch company.abbr | company abbreviation for naming/print                     |
| custom_amount_limit | Float                | —                          | credit control: max outstanding                           |
| custom_bill_count   | Int                  | —                          | credit control: max open unpaid bills                     |
| custom_days_limit   | Int                  | —                          | credit control: max credit age (days)                     |
| audit block (4)     | —                    | RO/NC                      | created/modified tracking                                 |
| 2 layout breaks     | Section/Column Break | structural                 | credit-limit + taxation layout                            |

## Sales Invoice (28 fields)

| Field                                         | Type                                         | Flags                        | Why                                    |
|-----------------------------------------------|----------------------------------------------|------------------------------|----------------------------------------|
| custom_abbr                                   | Data                                         | reqd, RO, fetch company.abbr | voucher numbering / print              |
| custom_company_abbr                           | Data                                         | hidden, in_list_view         | list view / naming                     |
| custom_fiscal_year                            | Data                                         | allow_on_submit              | Nepali FY label                        |
| custom_invoice_miti                           | Data                                         | allow_on_submit              | BS posting date (IRD)                  |
| custom_created_on_ad / custom_created_on_bs   | Date / Data                                  | —                            | AD + BS creation date pair             |
| custom_citytown                               | Data                                         | fetch customer_address.city  | prints city/town on IRD invoice        |
| custom_vehicle_no                             | Data                                         | —                            | delivery vehicle number                |
| custom_excise                                 | Currency                                     | hidden                       | working excise value                   |
| custom_vat                                    | Currency                                     | hidden                       | working VAT value                      |
| custom_total_amount_including_excise          | Currency                                     | —                            | excise scheme total                    |
| custom_total_excise_amount                    | Currency                                     | —                            | rolled-up excise                       |
| custom_total_excluding_excise (label "Total") | Currency                                     | hidden                       | working base excl. excise              |
| custom_total_vat_amount                       | Currency                                     | —                            | rolled-up VAT                          |
| custom_difference_adjustment                  | Currency (p3)                                | hidden                       | rounding tweak                         |
| custom_difference_calculation_table           | Table → Amount Calculation for sales invoice | hidden                       | VAT/excise/difference calc rows        |
| audit block (4) + 8 layout breaks             | —                                            | —                            | tracking + layout                      |
| ( custom_print_count )                        | Int                                          | RO/NC (added by patch)       | IRD print-copy counter (chapter 4 §5a) |

## Sales Invoice Item (6 fields)

custom\_vat\_apply\_on (Select VAT 13%/VAT 0%/Amount, JSON default VAT 0%), custom\_vat\_rate (Percent, RO unless not-Amount), custom\_vat\_amount (Currency p5, editable only in Amount mode), custom\_excise\_value (Currency, NC), custom\_total (Currency, NC = line incl. excise), + 1 section break.

## Purchase Invoice (40 fields — the largest bundle)

| Field                                                                                                                 | Type                                                    | Flags                                                            | Why                                             |
|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------|
| custom_document_no                                                                                                    | Int                                                     | reqd (except Grihalaxmi)                                         | sequential document number                      |
| custom_name                                                                                                           | Data                                                    | in_list_view                                                     | Voucher No.                                     |
| custom_purchase_type                                                                                                  | Link → Purchase Type                                    | reqd (except Grihalaxmi), default "Purchase Other Service/Goods" | classifies purchase; drives P Type Code         |
| custom_p_type_code                                                                                                    | Data                                                    | RO, fetch purchase_type.purchase_type_code                       | voucher coding                                  |
| custom_fiscal_year                                                                                                    | Data                                                    | RO                                                               | Nepali FY                                       |
| custom_abbr                                                                                                           | Data                                                    | hidden, fetch company.abbr                                       | numbering                                       |
| custom_tax_withholding_category_custom                                                                                | Link → Tax Withholding Category                         | —                                                                | the custom TDS category (keeps ERPNext TDS out) |
| custom_total amount including excise / custom_total_excise_amount / custom_total_vat_amount / custom_total_tds_amount | Currency                                                | NC                                                               | rolled-up excise/VAT/TDS                        |
| custom_refinery                                                                                                       | Select (Barauni/Haldia/Paradip/Mathura/Durgapur/Karnal) | list/filter/search                                               | petroleum source refinery                       |
| custom_pdo_no                                                                                                         | Data                                                    | list/filter/search                                               | Petroleum Delivery Order no                     |
| custom_ioc_challan_no / _date / custom_ico_challan_miti                                                               | Data/Date/Data                                          | —                                                                | IOC challan (no / AD / BS)                      |
| custom_store_receipt_no / _date / _miti                                                                               | Data/Date/Data                                          | —                                                                | store receipt (GRN) refs                        |
| custom_supplier_invoice_miti                                                                                          | Data                                                    | —                                                                | supplier invoice BS date                        |
| custom_nepali_miti                                                                                                    | Data                                                    | —                                                                | BS posting date                                 |
| custom_name_of_transportor                                                                                            | Select (3 transporters)                                 | —                                                                | freight transporter                             |
| custom_vehicle_no                                                                                                     | Data                                                    | —                                                                | goods-receipt vehicle                           |
| custom_voucher_receipt_no                                                                                             | Data                                                    | hidden                                                           | internal receipt ref                            |
| custom_memo                                                                                                           | Small Text                                              | —                                                                | free-text note                                  |
| audit block (4) + ~10 layout breaks                                                                                   | —                                                       | —                                                                | tracking + layout                               |

Purchase Invoice Item (11 fields)

VAT trio ( custom\_vat\_apply\_on / rate / amount ), TDS trio ( custom\_tds\_apply\_on Percentage(%)/Amount, custom\_tds\_rate RO, custom\_tds amount), custom\_excise\_value, custom\_total, custom\_subtype (Link → Vehicle — the only place custom\_subtype exists on an item table; drives fleet/vehicle-expense tracking), + 2 layout breaks.

Purchase Order (15 fields)

Approval fields: custom\_approver (Link User "PO Reviewer", reqd), custom\_po\_request\_approver (Table → Purchase Order Request Approver), custom\_reason (Small Text, RO), custom\_remarks (Small Text), workflow\_state (Link → Workflow State, hidden/NC). Plus the four excise/VAT/ TDS parent totals, audit block, custom\_abbr, 1 layout break.

Purchase Order Item (10) / Purchase Receipt Item (10) / Supplier Quotation Item (10)

Identical VAT + TDS + excise item field sets as PI Item (minus custom\_subtype ), plus layout breaks.

Purchase Receipt (32 fields)

custom\_document\_no (Data, reqd), custom\_name (Voucher No), custom\_receipt\_type (Link → Receipt type, reqd), custom\_p\_type\_code (fetch receipt\_type.receipt\_code, hidden), custom\_posting\_miti, the same IOC/store-receipt/refinery/transporter/vehicle petroleum fields as PI, custom\_remark, the four excise/VAT/TDS totals, audit block, custom\_abbr, layout breaks.

Supplier Quotation (9 fields)

custom\_preferred\_quotation (Check — marks the chosen quote in the comparison report), the four excise/VAT/TDS parent totals, audit block.

Packed Item / Purchase Taxes and Charges (0 custom fields)

Property setters only: Packed Item makes rate read-only; Purchase Taxes and

Charges defaults `add_deduct_tax` to **Deduct**.

---

### ⚠ Two field defects worth fixing

---

- `purchase_receipt_item.json`: `custom_excise_value` and `custom_total` have `depends_on` pointing at `custom_tds_apply_on=='Amount'` (copy-paste artifact — they should always show).
  - `supplier_quotation_item.json`: `custom_tds_rate` `depends_on` references `custom_vat_apply_on` instead of `custom_tds_apply_on`.
-

# Part B — Custom doctype fields

28 doctypes: 2 submittable (Attendance Fix, Stock Adjustment), 1 single (User Daily Entry Summary Settings), 15 child/istable, the rest standard. All carry `track_changes: 1`. Structural Section/Column Breaks omitted; data fields listed in full.

## Attendance Fix — submittable, AF-.YYYY.-

`shift_type` (Link Shift Type, reqd — scopes to this shift's employees), `company` (Link, optional filter), `from_date / to_date` (Date, reqd), `employee` (Link, optional single), `devices` (Table MultiSelect → Attendance Fix Device — limit to these devices' checkins), `status` (Pending/Queued/Running/Fixed/Failed, RO), `progress_percentage / progress_message` (RO, realtime), `counters employees_processed / absent_rows_deleted / attendance_created_or_updated / checkins_relinked` (RO), `log` (Long Text, RO), `amended_from`.  
**Perms:** HR Manager + System Manager full (submit/cancel/amend); HR User read.  
**Controller:** `validate` (date order); `on_submit` (Queued + enqueue `run_attendance_fix_in_background` on long queue, 14400 s); `_run_fix` (per-employee×day reconciliation, savepoints, realtime progress); `_resolve_employees`, `_reconcile_day`, `_prepare_checkins_for_shift`, `_cancel_and_delete`; worker runs as Administrator crediting `audit_user=owner`.

## Attendance Fix Device — child

`device` (Link Biometric Device, reqd).

## Biometric Device — field:device\_name, title device\_name

`device_name` (Data, reqd/unique), `device_serial` (Data, reqd/unique — matched against bridge SN), `company` (Link, reqd), `device_ip` (informational), `device_port` (Int, 4370), `enabled` (Check, 1 — off ⇒ punches rejected), `device_model`, `connection_status` (Connected/Disconnected). Sync (RO): `last_contact_time` (heartbeat basis), `last_sync_time`, `total_synced`, `last_employee_sync`, `total_employees_synced`. Settings: `clear_logs_after_sync` (1), `sync_interval` (30 min), `timeout` (5 s). Alerting: `alert_threshold_minutes` (120), `alert_recipients` (Table). `notes`.  
**Perms:** HR Manager + System Manager full; HR User read.  
**JS:** "Force Bridge Sync" button → `enqueue_command(force_sync)` then polls the command row every 3 s up to 90 s.

## Biometric Device Alert Recipient — child

`email` (Data/Email, reqd), `recipient_name`.

## Biometric Device Command — BDC-.#####

`device` (Link, reqd), `command_type` (force\_sync/test\_connection, reqd), `status` (Pending/Running/Done/Failed, RO), `requested_by / requested_at` (RO), `started_at / completed_at` (RO), `attempts` ("Bridge Poll Count", RO), `payload` (Long Text JSON), `result` (Long Text, RO).  
**Perms:** HR Manager + System Manager; HR User read.  
**Controller:** `before_insert` (stamp requester/time/status); `validate` (enum).

## Branch Wise Warehouse — child, effectively empty

Only a Section Break in the exported JSON — the operative fields (`custom_branch`, `custom_buying_warehouse`, `custom_selling_warehouse`) live as site-DB custom fields (chapter 4 §3).

## CBMS Bill — title invoice\_number

**All fields RO** (system-written). `company`, `sales_invoice` (unique), `invoice_number`, `invoice_date` (AD), `invoice_date_bs` (BS), `fiscal_year`, `buyer_name / buyer_pan / seller_pan`, `total_sales`, `taxable_sales_vat`, `vat`, `discount` (local only), `excisable_amount / excise / taxable_sales_hst / hst` (Health Service Tax)/ `amount_for_esf / esf` (Education Service Fee)/ `export_sales / tax_exempted_sales` (all default 0), `sync_status` (Pending/Synced/Failed), `attempt_count`, `last_attempt`, `sync_response`, `datetime_client`.  
**Perms:** System Manager full; Accounts Manager read/report/print/email.

## CBMS Bill Return — title credit\_note\_number

Same amounts/sync structure as CBMS Bill, plus `ref_invoice_number` (the original's running number — must be Synced first), `credit_note_number`, `credit_note_date / _bs`, `reason_for_return` (default "Goods Returned"). All

RO. Same perms.

**CBMS Config** — field:company

company (unique), enable\_cbms (Check 0), enable\_from\_date (mandatory when enabled — go-live cutoff), username (Data, reqd), password (Password, reqd), bill\_retry\_batch\_size / return\_retry\_batch\_size (Int 50).

**Perms:** System Manager + Accounts Manager full.

**Controller:** validate (one config per company); whitelisted sync\_failed\_now.

**Company Filter Config** — field:doctype\_name

doctype\_name (Link, unique), company\_field (Select company/custom\_company), fields (Table → Company Filter Field). **Perms:** System Manager only.

**Controller:** on\_update / on\_trash clear the config cache + ask to refresh.

**Company Filter Field — child**

fieldname (reqd), is\_child\_table (Check) + child\_fieldname, is\_dynamic\_link (Check) + dynamic\_link\_field.

**Document Template** — field:template\_name , title template\_name

template\_name (unique), target\_doctype (optional — email recipient), companies (MultiSelect), letter\_head, print\_orientation, default\_recipient\_field (email\_id), is\_active, email\_subject (Jinja), inputs / data\_sources (Tables), header\_html + header\_height (25 mm), footer\_html + footer\_height (15 mm), body\_html, help HTML blocks.

**Perms:** System Manager + Document Template Manager full; Document Template User read/print/report.

**Controller:** validate → dedup companies + dry-render Jinja.

**JS:** "Preview" (stub data).

**Document Template Company / Data Source / Input — children**

- **Company:** company (Link, reqd).
- **Data Source:** source\_name (→ data.<name> ), source\_type (SQL/Python), description, query (Code).
- **Input:** fieldname, label, input\_type (Data/Link/Date/Select/Int/Float/Check), options, reqd, exclusive\_group, exclusive\_set.

**Dynamic Approval Approver — child (injected on targets)**

level (Int, reqd), approver (Link User, reqd), approver\_name (fetch).

**Dynamic Approval Fixed Approver — child**

section (reqd), approver (Link User, reqd), approver\_name (fetch).

**Dynamic Approval Match Criteria — child**

section (reqd), field\_name (Autocomplete, reqd), field\_value (Autocomplete, reqd).

**Dynamic Approval Setting** — autoname() → {doctype}-{abbr}-{hash6}

document\_type (Link, reqd), company (Link, reqd), company\_abbr (fetch), is\_active (1), approver\_table\_fieldname (default custom\_approval\_approvers ), current\_level\_fieldname (default custom\_current\_approval\_level ), approver\_fieldname (⚠ dead — unused), dept\_config\_html (rules UI canvas), match\_criteria / approvers (hidden Tables), setup\_workflow\_html.

**Perms:** System Manager only.

**Controller:** autoname; validate (criteria rows complete).

**JS:** ~15 KB rules-card UI + Setup Workflow button.

**Employee Attendance Allowance — child (on Employee)**

salary\_component (Link, reqd), eligible (Check 1), rate (Currency — override), effective\_from (Date).

**Fiscal Year Access Control** — field:user

user (Link, unique), full\_access (Check 0), access\_details (Table → User Fiscal Year Access, shown when not full\_access). **Perms:** System Manager only.

**Controller:** on\_update / on\_trash clear cache key user\_fiscal\_access\_{user} (⚠ resolver uses a different key — chapter 6 §1.4).

Generated Document — DOC-GEN-.YYYY.-.##### , title title

title (reqd), template (Link, reqd), company, target\_doctype / reference\_name / party (RO), status (Draft/Finalized/Sent, RO), data\_provider (RO), print\_orientation (RO snapshot), payload (JSON, RO), rendered\_document (HTML preview), body\_html (RO — PDF/email source), header/footer snapshot fields, output\_action, recipients, email\_status (Not Sent/Queued/Sent/Failed), error.

**Perms:** System Manager + Document Template Manager full; Document Template User CRUD if\_owner.

**Controller:** validate → title default + recipients-required-when-Sent.

**JS:** "Edit in Generator" + sandboxed-iframe preview.

Stock Adjustment — submittable, extends StockController

posting\_date (reqd), posting\_time, company (reqd), adjustment\_type (Loss/Gain, reqd), reason (reqd), items (Table → Stock Adjustment Item, reqd), created\_by (RO), approved\_by (Link User), amended\_from.

**Perms:** System Manager + Stock Manager full (submit/cancel).

**Controller:** engine\_patch.apply\_patch() at import (zero-rate rows survive reposts); validate (≥1 item, qty>0, **rate must be 0**, warehouse company matches); on\_submit / on\_cancel post & reverse SLEs (sign by Loss/Gain, incoming\_rate=0, is\_adjustment\_entry=1).

**JS:** item query stock-items-only, warehouse query non-group + company, clears mismatched warehouses on company change.

Stock Adjustment Item — child

item\_code (Link, reqd), item\_name (fetch), warehouse (Link, reqd), uom (fetch stock\_uom), adjustment\_qty (Float p6, reqd), rate (Currency p6, default 0 — must stay 0).

User Daily Entry Summary Doctype — child

document\_type (Link, reqd).

User Daily Entry Summary Settings — Single

description (HTML help), tracked\_doctypes (Table). **Perms:** System Manager only.

User Fiscal Year Access — child, format:{user}.{#####}

doctype\_name (Link, reqd), fiscal\_year (Link, mandatory unless row full\_access), full\_access (Check — all fiscal years for that doctype).

Voucher Number Settings Item — child

doctype\_name (Link), voucher\_no\_digits (Int — zero-pad width, default 6).

**Controllers with real logic (10):** attendance\_fix, biometric\_device\_command, cbms\_config, company\_filter\_config, document\_template, dynamic\_approval\_setting, fiscal\_year\_access\_control, generated\_document, stock\_adjustment (+ the stock\_adjustment/engine\_patch.py). All other controllers are pass stubs.

**Doctypes with JS (7):** attendance\_fix, biometric\_device, document\_template, dynamic\_approval\_setting, generated\_document, stock\_adjustment.

## Part C — DB-only custom doctypes (in the module, not in the repo)

The "Avinash Group App" module owns **41 doctypes** on avinas1. The 28 above ship as files. The remaining **13 were created through the Customize UI** ( `custom = 1` ) and live only in the site database — they have **no folder in the repo**, so a `git clone` alone won't recreate them; they must be migrated from the site or recreated. Several are load-bearing (they back custom fields the code reads); a few are orphaned/legacy. This section documents all 13.

⚠ Because these are DB-only, treat them as part of the data model but note they are **not version-controlled**. Consider exporting them as fixtures if they matter (Purchase Type / Receipt type / Payment - Receipt Type / JV Type / Vehicle List / the two approver tables and the SI calc table are the ones that matter).

### Code-list doctypes driving voucher numbering (chapter 4 §4)

| Doctype                                                        | Fields                                                                                                                   | Backs                                                                                              | Why                                                                                                                                                            |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purchase Type</b><br>( <code>field:purchase_type</code> )   | <code>purchase_type</code> (Data, unique),<br><code>purchase_type_code</code> (Data)                                     | Purchase Invoice<br><code>custom_purchase_type</code> → fetches<br><code>custom_p_type_code</code> | classifies a purchase (gas/service/goods) and supplies the type code embedded in the voucher number; also filters the Gas Purchase & Purchase Register reports |
| <b>Receipt type</b><br>( <code>field:p_type</code> )           | <code>p_type</code> (Data, unique),<br><code>receipt_code</code> (Data)                                                  | Purchase Receipt<br><code>custom_receipt_type</code> →<br><code>custom_p_type_code</code>          | same idea for Purchase Receipts                                                                                                                                |
| <b>Payment - Receipt Type</b><br>( <code>field:p_type</code> ) | <code>p_type</code> (Data, unique), <code>data_hrcj</code><br>("Payment - Receipt Code"), <code>company</code><br>(Link) | Payment Entry <code>custom_p_type</code>                                                           | supplies the type code for Payment Entry voucher numbering                                                                                                     |
| <b>JV Type</b> ( <code>field:p_type</code> )                   | <code>p_type</code> (Data, unique),<br><code>jv_type_code</code> (Data), <code>company</code> (Link)                     | Journal Entry <code>custom_p_type</code>                                                           | supplies the JV type code (used by<br><code>voucher_no_console.update_journal_entry_custom_names</code> )                                                      |

### Approval helper tables

| Doctype                                        | Fields                                                                        | Backs                                                     | Why                                                                                          |
|------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Purchase Order Request Approver</b> (child) | <code>level</code> (Int, reqd), <code>approver</code> (Link → Employee, reqd) | Purchase Order<br><code>custom_po_request_approver</code> | the PO's own multi-level approver list (predates / complements the generic Dynamic Approval) |
| <b>Material Request Approver</b> (child)       | <code>level</code> (Int), <code>approver</code> (Link → Employee)             | (no live reference found)                                 | legacy MR approval table — superseded by Dynamic Approval                                    |

### Sub-ledger / vehicle mapping

| Doctype                                                                     | Fields                                                                                                                                                                                                                                                            | Backs                                               | Why                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vehicle List</b> (child)                                                 | <code>vehicle_list</code> (Link → Vehicle, reqd)                                                                                                                                                                                                                  | <b>Account</b><br><code>custom_sub_type_list</code> | * the per-Account vehicle whitelist — the vehicle pickers on PI/JE ( <code>pi.js</code> , <code>journal_entry.js</code> ) and the <code>vehicle_mandatory</code> validation read <code>Account.custom_sub_type_list.vehicle_list</code> to decide which vehicles a vehicle-expense account allows (chapter 4 §7) |
| <b>Sub-Ledger Category</b><br>( <code>field:sub_ledger</code> , nested-set) | <code>sub_ledger</code> (Data, reqd/unique), <code>is_group</code> (Check),<br><code>parent_sub_ledger_category/old_parent</code> (Link self),<br><code>company</code> (Link), <code>table_dcoy</code> (Table → Sub-ledger table),<br><code>lft/rgt</code> (tree) | self + Sub-ledger table                             | a tree of sub-ledger categories (an earlier sub-classification scheme; the vehicle-expense report's "Sub-Ledger Category" filter references this concept — now largely replaced by the direct Vehicle link)                                                                                                      |
| <b>Sub-ledger table</b> (child)                                             | <code>sub_type_list</code> (Link → Sub-Ledger Category)                                                                                                                                                                                                           | Sub-Ledger Category<br><code>table_dcoy</code>      | child rows of the sub-ledger tree                                                                                                                                                                                                                                                                                |

### Sales-invoice calculation tables

| Doctype                                             | Fields                                 | Backs                                                                         | Why                                                                                                                                                                                             |
|-----------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Amount Calculation for sales invoice</b> (child) | <code>item_code</code><br>(Data, reqd) | Sales Invoice<br><code>custom_difference_calculation_table</code><br>(hidden) | backing rows for the excise/VAT/rounding-difference reconciliation used by <code>override_rounding.py</code> (⚠ that engine is dead code — chapter 4 §11, so this table is effectively dormant) |
| <b>Invoice Calculation</b> (child)                  | (zero fields)                          | (nothing)                                                                     | empty orphan doctype — safe to delete                                                                                                                                                           |

### Other

| Doctype                                     | Fields                                                                                                                                                                                                                                                                 | Backs                                                               | Why                                                                                                            |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Employee Cost Center Manager</b> (child) | <code>cost_center</code> (Link → Cost Center), <code>user</code> (Link → User)                                                                                                                                                                                         | <b>Employee</b><br><code>custom_employee_cost_center_manager</code> | maps cost centres to a managing user on the Employee record; not referenced by documented app code (data-only) |
| <b>Material request table</b> (child)       | <code>supplier</code> , <code>item</code> , <code>qty</code> , <code>uom</code> , <code>rate</code> , <code>amount</code> , <code>discount</code> ,<br><code>taxable_amount</code> , <code>include_vat</code> (Check), <code>vat_13</code> , <code>total_amount</code> | (no live reference found)                                           | a custom MR line table with VAT columns — orphaned/legacy                                                      |

**Status summary:** load-bearing → Purchase Type, Receipt type, Payment - Receipt Type, JV Type, Vehicle List, Purchase Order Request Approver, Amount Calculation for sales invoice. Dormant (backs dead code) → Amount Calculation (via `override_rounding`). Orphaned/legacy → Invoice Calculation (empty), Material Request Approver, Material request table, Employee Cost Center Manager, Sub-Ledger Category + Sub-ledger table.

# Reports Appendix — Full Per-Report Reference

Chapter 12 of the technical documentation. The complete reference for every report: purpose, source tables, all filters (with defaults), key columns, notable logic, PDF pattern, roles, and audience. Chapter 8 is the catalogue; this is the deep detail. The 5 HR/BS attendance reports + salary statement are in chapter 3 §7. All are Script Reports under `avinash_group_app/report/<name>/` with `.py` (query/rows), `.js` (filters + print), `.json` (config), and often `<name>_pdf.html` (PDF template).

## Shared infrastructure

- **BS-date filter twins** come from `rdp_common_app's` `report_nepali_date.js`: any Date filter gets a BS "Miti" input + "Select Month". Used by Gas Purchase, Sales Stock Ledger, Party Ledger Summary.
- **report\_print\_orientation.js** (global on query-report routes): the Portrait/Landscape dialog (`window.askPrintOrientation`), a wrapper that appends `&orientation=` to every "Download PDF" button, a `window.open` shim that injects `report_print_portrait.css` and fits wide tables, and a `frappe.render_pdf` shim that inlines black-grid styling + optional `window.__rdpPrintFooter` (Advance Tax TDS uses it for the signature block).
- **PDF patterns: A** = report-local `*_pdf.html` + whitelisted `download_pdf` (re-runs execute, paginates in Python, in-body "Page X of Y", Print calls the same PDF inline, "Pick Columns" support). **B** = shared helper only. **C** = stock Frappe print.
- **Number formatting:** `_fmt_inr` (Indian lakh/crore grouping, blanks zero), `_fmt_qty` (3 dp). Currency NPR.

## 1. Advance Tax TDS Details

- **Purpose:** Nepali IRD advance-tax (अग्रिम कर) withholding statement — per supplier, per Tax Withholding Category, the TDS-eligible turnover and TDS withheld, sectioned by IRD account number (खाता नं) with Nepali headers and a Prepared/Checked/Verified signature footer.
- **Source:** Purchase Invoice  $\bowtie$  Purchase Invoice Item (INNER)  $\bowtie$  Supplier (LEFT, for PAN). Category from `custom_tax_withholding_category_custom`.
- **Filters:** `company*` (default user default), `from_date*` (month start), `to_date*` (today), `Fit Columns` (view toggle, default 1, excluded from print).
- **Columns:** क्र.सं. / नाम / कारोबार रकम (turnover) / खाता नं / अग्रिम कर रकम (TDS) / पान नम्बर / रेट.
- **Logic:** category strings like "2.5% -11124 TDS-Other Entities" are regex-parsed into rate/khata/title. Turnover = `SUM(item amount WHERE apply_tds=1)`; TDS = `SUM(custom_tds_amount)`; grouped by supplier + category. Section header/total + grand total rows bolded. `custom_tds_apply_on='Amount'` shows rate as "amount".
- **PDF:** B (shared helper + signature footer). **Roles:** Accounts Manager/User.
- **Audience:** accountant/auditor — IRD TDS return.

## 2. Avinas Vehicle Expense ("Nepal Gas Vehicle Expense")

- **Purpose:** vehicle-wise operating expense (Fuel / Repair / Others) across both Purchase Invoices and Journal Entries, one row per vehicle + grand total. Company-agnostic — buckets keyed by account-name LIKE patterns.
- **Source:** `PI  $\bowtie$  PII (amount pic.amount, vehicle pic.custom_subtype, account pic.expense_account) UNION ALL JE  $\bowtie$  JEA (amount jea.debit - jea.credit, vehicle jea.custom_subtype, account jea.account)`, both  $\bowtie$  Account, LEFT  $\bowtie$  Vehicle for the license plate.
- **Bucket mapping:** Fuel %Fuel Expenses%; Repair %R & M - Vehicles%; Others %Other Vehicle Expenses%.
- **Filters:** `company*` (user default), Vehicle No (Link), Filter By (Fiscal Year / Date Range, default Fiscal Year), Fiscal Year, Start/End Date. Company applies to `acc.company`.
- **Columns:** Vehicle No / Fuel / Repair / Others (Currency).
- **Logic:** blank vehicles excluded; grouped by `COALESCE(license_plate, vehicle)`; Vehicle No renders as a link to the Vehicle. Doc: `docs/vehicle_expense_report.md` (notes open items: filter label mismatch, BS filter not yet done, stale test).
- **Roles:** Accounts User/Manager, System Manager. **Audience:** management/accountant (fleet cost).

## 3. Consolidated Financial Statement Hierarchy

- **Purpose:** consolidated Balance Sheet / P&L / Cash Flow across **any** arbitrary set of companies — no shared parent group company required (unlike ERPNext's built-in). Adds a per-row Total column, hides accounts not common to all selected companies, caps tree depth.
- **Source:** GL Entry (aggregated), Account, Company, Fiscal Year, Finance Book.
- **Filters:** Companies (MultiSelect, empty = all), Filter Based On (Fiscal Year / Date Range), Start/End Date, Start/End Year, Finance Book, Report (P&L / Balance Sheet / Cash Flow, default Balance Sheet), Common Accounts scope (Selected / All), Currency,

Accumulated Values in Group Company, Include Default FB Entries (1), Show zero values, Account Hierarchy Level (1..max depth).

- **Logic:** monkey-patches ERPNext's consolidated engine (`get_companies`, `set_gl_entries_by_account`, `get_account_type_based_gl_data`, `get_accounts`) to accept an explicit company list and replace per-root GL fetches with **one aggregated SQL** producing two synthetic opening/period entries per account/company (with presentation-currency conversion + finance-book scope). Fixes an ERPNext Cash Flow bug. `add_row_total_column` sums the row; `filter_shared_accounts` drops accounts absent from any company (with orphan re-parenting); `apply_account_level_filter` caps depth. **Needs** the `fin_stmt_agg_index` (patch `add_gl_entry_fin_stmt_agg_index`) for speed. Has `USER_DOCS.md`.
- **Roles:** Accounts User/Manager, **Auditor**. **Audience:** management/auditor.

## 4. Profit and Loss Hierarchy

---

- **Purpose:** single-company ERPNext P&L extended with an "Account Hierarchy Level" control that blanks group-account rows above a chosen indent level; Report/Growth/Margin views.
- **Source:** GL Entry via `erpnext.financial_statements`.
- **Filters:** base financial-statement filters + Select View (Report/Growth/Margin), Accumulated Values (1), Include Default FB Entries (1), Show zero values, Account Hierarchy Level (1-6, default 3).
- **Logic:** Income (Credit) + Expense (Debit) with `ignore_closing_entries`; net profit/loss appended; group rows with indent < level are cleared while leaves + "Profit for the year" stay.
- **Roles:** Accounts User/Manager, Auditor. **Audience:** management/auditor.

## 5. Custom Supplier Quotation Comparison

---

- **Purpose:** pivoted procurement comparison — items as rows, suppliers as dynamic columns, per-item price + summary rows (Net Total, discount, Taxable, VAT, Invoice Amount). Includes a "Select Default Supplier" tool.
- **Source:** `Supplier Quotation` ⋈ `Supplier Quotation Item`, optionally driven from a PO → RFQ link.
- **Filters:** `company *`, `from_date` (−1 month), `to_date` (today), `Item` (scoped), `Supplier` (MultiSelect, company-scoped `get_company_suppliers`), `Supplier Quotation` (MultiSelect `docstatus<2`), `Request for Quotation`, `Categorize by` (Supplier/Item), `Include Expired` (0), `Preferred Quotation` (1 → `custom_preferred_quotation=1`), `Purchase Order` (auto-fills RFQ).
- **Logic:** price field default `base_amount`; keeps the lowest price when multiple quotes; summary rows from quotation-level discount/tax (`apply_discount_on` splits Net-Total vs Grand-Total discount); Invoice Amount from `base_grand_total`; supplier columns generated via `frappe.scrub`; near-expiry `valid_till` colored.
- **Roles:** Manufacturing/Purchase Manager, Purchase/Stock User. **Audience:** procurement.

## 6. Gas Purchase Report

---

- **Purpose:** LPG procurement register — one row per Store Receipt combining gas PIs, PRs, and Service Charge (ICP/NA) invoices into refinery/tanker/IOC-challan/qty/rate/price/VAT/ICP/ICT-VAT/N.A./total columns, by BS month.
- **Source:** `PI(+Item)`, `PR(+Item)` keyed by custom fields (`custom_purchase_type`, `custom_receipt_type`, `custom_store_receipt_*`, `custom_refinery`, `custom_pdo_no`, `custom_vehicle_no`, `custom_ioc_challan_*`, `custom_name_of_transportor`). Type constants: "Gas Purchase Invoice", "Gas Purchase Receipt", "Service Charge ICP", "Service Charge NA".
- **Filters:** `company` (MultiSelect), `from_date` / `to_date` (BS twins), `Refinery` (MultiSelect from the field's own options via `get_refineries`).
- **Logic:** an invoiced receipt is represented by its invoice (`pii.purchase_receipt`), else by the receipt; period filtered in Python on BS miti (From/To Miti or BS Year+Month + optional AD range); service charges mapped by document number parsed from `custom_store_receipt_no` or a coded voucher name (NGK-ICP-00218-82/83 → 218). Rate = Price/Qty; Total = Price + other\_expense − ICT VAT. Total row appended.
- **Roles:** Accounts Manager/User. **Audience:** accountant/management (NOC gas procurement, excise/ICP/VAT).

## 7. Loan Summary

---

- **Purpose:** borrowings matrix — each loan type (children of "Short/Long Term Borrowings") × company columns, subtotals, grand total, per-company ratio; "Show Details" expands sub-accounts.
- **Source:** `Account` (tree under the two group names) + `GL Entry` (`SUM(credit − debit) WHERE posting_date <= to_date`).
- **Filters:** `Company` (MultiSelect, empty = all), `from_date` (year start), `to_date` \* (today), `Show Details` (0).
- **Logic:** `_normalize` collapses label variants ("Short Term Loan (STL)"); Cr/Dr suffix with accounting dash for zero; Ratio = company grand / all-company grand. Standalone PDF (Landscape) with in-body pagination; header falls back to "Nepal Gas Group". `from_date` collected but **unused** (cumulative to To Date).

- **PDF:** A. **Roles:** Accounts Manager/User. **Audience:** management/accountant.

## 8. Net Position of Cash and Bank

- **Purpose:** treasury cash-position — every leaf account under "Cash and Cash Equivalent": Opening / Receipts / Payments / Closing over the period + a consolidated total.
- **Source:** Account (group matched by name, & → and) + GL Entry (opening = balance before from\_date + in-period is\_opening; movements = debit/credit sums).
- **Filters:** company \* (user default), from\_date \* (year start), to\_date \* (year end), Cash/Bank Codes (MultiSelect), Suppress Local Currency Equivalents, Consider Post Dated Cheques (gates the is\_pdc clause via \_gl\_has\_pdc\_column), Show Zero Balance.
- **Columns:** Cash/Bank Code / Description (Link Account) / Cash Book / Currency Code / Opening / Receipts / Payments / Closing.
- **Logic:** Closing = Opening + Receipts – Payments; zero-movement accounts skipped unless Show Zero; multi-currency optionally forced to company currency; totals in company currency.
- **Roles:** Accounts Manager/User, System Manager. **Audience:** treasury/management.

## 9. One Lakh Above Transactions

- **Purpose:** Nepal IRD annexure — every customer (Sale) and supplier (Purchase) whose taxable OR exempt total ≥ Rs100,000 in the period. THRESHOLD = 100000.
- **Source:** SI+Item ⋈ Customer and PI+Item ⋈ Supplier.
- **Filters:** company \* (MultiSelect), from\_date \* (month start), to\_date \* (today), Fit Columns (1).
- **Columns:** PAN / Name of Tax Payer / Trade Name Type (constant 'E') / Purchase-Sale ('S'/'P') / Taxable Amount / Exempted Amount.
- **Logic:** taxable = SUM(ABS(amount) WHERE custom\_vat\_amount≠0), exempt = VAT-0 lines; returns subtract as negative ABS; HAVING ≥ threshold + Python re-filter; sorted by name.
- **PDF:** C (standalone HTML/PDF, Portrait). **Roles:** Accounts Manager/User. **Audience:** auditor/accountant — IRD filing. Δ trade\_name\_type hardcoded 'E'; no totals row.

## 10. Party Ledger (1278 lines)

- **Purpose:** full customer/supplier statement — Opening → each voucher (with running balance) → "For the Periods" → Closing, with BS Miti dates, custom voucher display names, optional item-level detail, remarks, Contract-Form handling. Multi-party = one section per party. **Engine of the /customer\_statement portal.**
- **Source:** GL Entry (is\_cancelled=0, company, party\_type) LEFT ⋈ SI/PI for return detection; enrichment from SI/PI Items, Payment Entry (+refs), Journal Entry, Customer/Supplier.
- **Filters:** company \* (user default), Party Type (Customer/Supplier, default Customer), Party (MultiSelect; 1 → flat, 0/2+ → grouped), Account (MultiSelect scoped to ledger accounts), from\_date \* (month start), to\_date \* (today), Voucher No (LIKE), Show Remarks, Show Balance (default 1 — blanks running balance on transaction rows), Detailed Mapping (item sub-rows), Show Contract Form (default 1 — includes JE custom\_p\_type='Contract Form'), Fit Columns.
- **Columns:** S.No / Date / Miti / Voucher No / Description / [detail qty/uom / rate/amount] / Debit / Credit / Balance / [Remarks].
- **Logic:** Opening = pre-period balance + in-period is\_opening; merge collapses multi-line vouchers per voucher but splits JEs per account (contras shown separately); BS miti from custom\_invoice\_miti / custom\_posting\_miti / custom\_nepali\_miti; display names from custom\_branch\_name / custom\_name (real docname kept for links); remarks from custom\_narration / memo / remarks / user\_remark; detailed mapping adds indented item + VAT + payment-reference/advance rows. Two gated WHERE conditions applied to opening+period alike: Contract-Form exclusion and exclude\_account\_patterns (the portal uses it to hide deposit/security accounts). Enrichment batched in IN-chunks of 500.
- **PDF:** A (Landscape, party\_ledger\_pdf.html). **Roles:** Accounts Manager/User, Employee Self Service, Sales person, Account Team. **Audience:** accountant/management + customers (portal). Δ One row per voucher × party → large for wide ranges.

## 11. Party Ledger Summary (526 lines)

- **Purpose:** one aggregate line per party (Opening / Debit / Credit / Closing) — "Super Summary" (flat + grand total) or "Group Wise" (per-group subtotals). Custom Excel export with Dr/Cr columns.
- **Source:** GL Entry GROUP BY party ⋈ Customer/Supplier + their Groups.
- **Filters:** Report Type \* (Super Summary/Group Wise), company, Party Type, Party Group (MultiSelect, get\_company\_party\_groups), Party (MultiSelect), from\_date \* (month start), to\_date \* (today), Show Zero Balance, Closing DB/CR (blank/DB/CR), Fit Columns.
- **Columns:** Customer/Vendor Code / Name / VAT-PAN / Opening / Debit / Credit / Closing.
- **Logic:** company-scoped via party custom\_company EXISTS; opening =

pre-period + is\_opening; net-zero hidden unless Show Zero; Closing DB/CR keeps debit(>0)/credit(<0) closings. Super Summary appends "Closing Totals (N Customers/Vendors)"; Group Wise emits group header → parties → "Group Total for: X". Excel export shows Opening/Closing magnitude + Dr/Cr columns (standard Export overridden). BS month picker repositioned next to Fit Columns.

- **PDF:** A (default Portrait). **Roles:** Accounts Manager/User, Account Team. **Audience:** accountant/management.

## 12. Purchase Register Report

---

- **Purpose:** Nepal VAT purchase register (खरिद खाता) — per PI, value split into Tax-Free / Taxable (domestic) / Taxable Import / Capitalized (fixed-asset) with their VATs + Total VAT + qty. "Is Return" mode shows returns as positives.
- **Source:** `PI+Item ⋈ Supplier (tax_id, custom_territory) ⋈ Item (is_fixed_asset)`, GROUP BY `pi.name`.
- **Filters:** `company` (MultiSelect), `from_date` \* (month start), `to_date` \* (today), `Supplier` (MultiSelect, `get_company_suppliers`), Purchase Type (MultiSelect → filters `custom_purchase_type`), Is Return, Fit Columns (1).
- **Columns:** Date / Miti / Purchase Type / Voucher No / Supplier Invoice No/Date/Miti / Supplier Name / VAT Number / Purchase / Tax Free / Taxable / VAT / Taxable Import / Import VAT / Capitalized / Capitalized VAT / Total VAT / QTY.
- **Logic:** bucketing by `custom_vat_apply_on` × `is_fixed_asset` × `custom_territory` (Nepal / non-Nepal). Purchase = `custom_total_amount_including_excise`. Miti first token. Returns → `abs()`. Voucher No encoded `custom_name::docname`. Shares the Pick-Columns patch with Sales Register.
- **PDF:** A (Landscape). **Roles:** Accounts Manager/User. **Audience:** accountant/auditor — IRD VAT register.

## 13. Sales Register Report

---

- **Purpose:** VAT sales register — per submitted SI, split tax-free / export / taxable / VAT. Returns view (`abs`).
- **Source:** `SI ⋈ SII ⋈ Customer`, GROUP BY `si.name`. Buckets = conditional SUM over `custom_vat_apply_on` + territory.
- **Filters:** `company`, `from_date` / `to_date`, `customer`, `is_return`, Fit Columns.
- **PDF:** A (Landscape, Pick-Columns). `Δ` `company` / `customer` string-interpolated into SQL; dates are bound params.
- **Roles:** Accounts Manager/User. **Audience:** accountant/auditor — VAT return.

## 14. Receipt Register (451 lines)

---

- **Purpose:** customer receipts (Payment Entry, Receive) in 3 views — Date-Wise / Customer-Wise (merged banner) / Summary. Shows cheque no + remarks.
- **Source:** Payment Entry (`docstatus=1`, `party_type='Customer'`, `payment_type='Receive'`); fields incl. `custom_posting_miti`, `custom_name`, `paid_to`, `reference_no`, `received_amount`, `custom_remark`.
- **Filters:** View \* (3, default Date Wise), `company` \* (user default), `from_date` \* (month start), `to_date` \* (today), `Customer` (MultiSelect, `get_company_customers`), Bank/Cash Account (MultiSelect of `company` Bank/Cash via `get_company_bank_accounts`, filters `paid_to`), Fit Columns.
- **Columns:** vary by view; all views total per group + grand total.
- **Logic:** cheque "1" placeholder suppressed; Miti first token of `custom_posting_miti`; customer-header rows render as full-width grey banners (re-tagged on scroll due to datatable virtualization). Own PDF: Date-Wise → Landscape, others → Portrait.
- **PDF:** A. **Roles:** Accounts Manager/User. **Audience:** accountant/cashier. `Δ` "GL Code" column is an unfinalized placeholder.

## 15. Sales Analysis — Customer wise Summary (391 lines)

---

- **Purpose:** one row per customer — Sales Qty + Gross Value (incl. excise); with Include Return: Return Qty/Value + Net Sales; grand total row.
- **Source:** `SI+Item ⋈ Customer`, GROUP BY `customer`.
- **Filters:** `company` (MultiSelect), `from_date` / `to_date`, `Customer` (`get_company_customers`), Customer Group (`get_company_customer_groups`), Item (`get_company_items`), Include Return (default 1), Fit Columns.
- **Columns:** Customer Code (Link) / Name / VAT-PAN / Sales Qty / Gross Value; +Return Qty/Value, Net Sales Qty/Value when Include Return.
- **Logic:** Gross = `amount` + `custom_excise_value`; returns negated to positive; Net = Sales – Return in Python; Return/Net columns only generated when Include Return on.
- **PDF:** A (Landscape). **Roles:** Accounts Manager/User. **Audience:** sales management. `Δ` `item_code` narrows lines but grouping stays per customer.

## 16. Sales Analysis — Customer wise Details (360 lines)

---

- **Purpose:** Customer → Product (UOM) drill-down: qty / value / VAT / total-incl-VAT + per-customer and grand rollups.

- **Source:** `SII ⋈ SI ⋈ Item ⋈ Customer, GROUP BY customer, item, uom, is_return.`
- **Filters:** `company, from_date/to_date, Customer, Product (get_company_items), Include Return (default 1), Fit Columns.`
- **Logic:** `Value = amount + custom_excise_value; VAT = custom_vat_amount;` returns flipped positive; UOM suffix only when an item sold in >1 UOM.  $\Delta$  Frappe drops falsy filters  $\rightarrow$  unchecked Include Return never reaches the server (`!int(None)=0 = OFF`); JS default 1 keeps it on at first load.
- **PDF:** A (Portrait). **Roles:** Accounts Manager/User. **Audience:** sales.  $\Delta$  Granularity customer  $\times$  item  $\times$  UOM  $\rightarrow$  large.

## 17. Sales Analysis — Product wise Invoice Details (424 lines)

- **Purpose:** Product  $\rightarrow$  (No Agent)  $\rightarrow$  Customer  $\rightarrow$  invoice-level rows (Invoice then Return sections), with Customer/Agent/Product totals + grand totals. `build_rows` is reused by the `/product_wise_invoice_details` portal (agent rows dropped there).
- **Source:** `SII ⋈ SI`; item/customer names via small IN-lookups (deliberately not joined, to keep the heavy query lean), sorted in Python.
- **Filters:** `company, from_date/to_date, Customer, Product, Include Return (default 1), Fit Columns.`
- **Logic:** same Value/VAT/return-flip + Include-Return quirk. `include_agent` toggles the No-Agent/Agent rows (portal passes False). Per-customer order (2026-06-28): Invoice rows  $\rightarrow$  Return rows  $\rightarrow$  summary (Customer Sales above Customer Returns); section labels bold. Miti from `custom_invoice_miti`.
- **PDF:** A (Landscape). **Roles:** Accounts Manager/User. **Audience:** sales + customers (portal).

## 18. Sales Stock Ledger (500 lines)

- **Purpose:** sales-driven stock movement from submitted SIs of stock items — Detail (per line) or Summarized (per item/UOM, optional sales/return merge). BS posting date per row.
- **Source:** `SI ⋈ SII ⋈ Item (is_stock_item=1).`
- **Filters:** `Report Type * (Detail/Summarized), company` (user default; on-change clears/refreshes dependents), `Branch` (company-scoped), `from_date * (month start), to_date * (month end)` — BS twins, `Warehouse` (company-scoped), `Item` (via `Item Default.company`), `Item Group` (custom\_company), `Price List`, `UOM`, `Voucher No` (SI, scoped), `Voucher Type` (Sales Invoice/Sales Return), `Fit Columns (1)`, `Sales/Return Merge`.
- **Columns:** Detail — `Date / Nepali Date / Voucher Type / Voucher No / Item / Sales Qty / UOM / Sales Rate (5 dp) / Stock Qty / Stock UOM / Rate of Stock UOM NPR (5 dp) / Balance`. Summarized — `Item / Type / Sales Qty / UOM / Stock Qty / Stock UOM / Balance`. Bold Total row.
- **Logic:** `nepali_datetime` for BS conversion; `_resolve_period` returns empty (no error) for an incomplete period, throws only From>To; stock rate = `COALESCE(NULLIF(stock_uom_rate,0), rate/conversion_factor)`; merge nets returns `( ELSE -ABS(qty) );` 5-dp rate precision. JS: colored Sales Invoice/Return pill badges, zebra striping, drag-scroll, "Rs" $\rightarrow$ "Rs".
- **Fix history:** `docs/sales_stock_ledger_fixes.md` (item-group column, missing SLE join, return sign in merge, float precision, company scoping, company-change refresh; + 2026-06-04 Nepali-month + UI polish).
- **Roles:** Accounts/Sales User & Manager, System Manager. **Audience:** sales + stock.

## Missing report

`customer_vendor_ledger_summary/` contains only a stale `__pycache__` — **no source** (`.json / .py / .js`). The report is effectively deleted. (The "Customer Ledger Summary" referenced by credit-control code is ERPNext's standard report, not this.)

## Audit / access reports (chapter 6)

- **User Audit Trail** — everything a user created/changed in a range, per field, old $\rightarrow$ new value, incl. child-row add/remove; only doctypes the running user can read. Roles: System Manager, Auditor, Accounts Manager, HR Manager.
- **User Daily Entry Summary** — per user + day, document counts by type and docstatus, each a drill-down link. Always runs fresh.

## HR / BS reports (chapter 3 §7)

Monthly Attendance BS, Monthly Attendance Summary BS, Work On Holiday BS, Yearly Leave Details BS, Avinas Salary Statement.

# Per-Doctype Reference — What the App Does to Every Doctype

Chapter 13 of the technical documentation. This is the **by-doctype** view: for each doctype, exactly what `avinashgroup.app` adds or overrides — fields, server hooks, client scripts, class overrides, reports — and why. It complements the by-feature chapters (2–8), which explain the mechanisms; here you look up a doctype and see everything done to it. Field lists are the live `avinash1` definitions (authoritative — includes patch-added fields).

Notation: **Fields** = app custom fields (the audit block is noted once below, not repeated). **Hooks** = `server doc_events`. **Class** = `override_doctype_class`. **JS** = client scripts. → points to the deep chapter.

## 0. Cross-cutting — applied to (almost) every doctype

Before the per-doctype list, three things the app does **universally**, so individual entries below don't repeat them:

1. **Audit block** (chapter 6 §3). ~250 doctypes carry `custom_created_by` + `custom_created_on` + `custom_modified_by` (RO, stamped on save). Masters also get `custom_naming_series`; doctypes without a native company field also get `custom_company`. The ~85 doctypes in `AuditBase.doctypes` additionally run the `audit doc_events` (before\_insert/before\_save/validate/autoname/after\_delete) which also do **company-matching validation** (chapter 6 §2) and **custom voucher numbering** (chapter 4 §4).
2. **Dynamic Approval** (chapter 2). Registered on `*` — `validate`, `before_save`, `on_update`, `before_workflow_action` fire on **every** doctype, but short-circuit instantly unless the doctype has the injected `custom_current_approver` field (i.e. was set up for approval).
3. **Fiscal-year access** (chapter 6 §1). The 77 transaction doctypes in `FILTERED_DOCTYPES` get `permission_query_conditions` + `has_permission` (list/read filtering by the user's allowed fiscal years).

Doctypes whose **only** customization is the audit block (≈250 of the 379) are not listed individually — e.g. most CRM/HR/manufacturing/quality masters. The entries below are the doctypes with app-specific fields, hooks, JS, overrides, or reports.

## 1. Selling transactions

### Sales Invoice — the most heavily customized doctype

- **Fields (23):** `custom_vat`/`custom_excise` + `custom_total_vat_amount`/`custom_total_excise_amount`/`custom_total_amount_including_excise`/`custom_total_excluding_excise`/`custom_total_amount` (VAT+excise scheme); `custom_difference_adjustment` + `custom_difference_calculation_table` (→ Amount Calculation for sales invoice) (rounding recon, dormant); `custom_abbr`/`custom_company_abbr`/`custom_fiscal_year`/`custom_invoice_miti` (numbering + Nepali date); `custom_created_on_ad`/`custom_created_on_bs`; `custom_vehicle_no`; `custom_citytown`; `custom_narration`; `custom_reason_for_return` (CBMS, patch); `custom_print_count` (IRD copies, patch); audit block.
- **Class:** `CustomSalesInvoice` — restores warehouse from DN/SO item, softens warehouse-mandatory (chapter 4 §2).
- **Hooks:** `before_validate`/`before_save`/`validate` → selling taxes (`salesinvoice_taxes.py`); `on_submit` + `before_cancel` → **CBMS** sync (chapter 5); `before_print` → **IRD print-copy** counter (chapter 4 §5a); `audit` + `dynamic-approval` + `fiscal-year`.
- **JS:** `sales_invoice.js` (VAT UI, selling-warehouse injection, due date from Customer `custom_days_limit`); `si.js` **not loaded** (dormant credit control).
- **Reports:** Sales Register, One Lakh Above, all 3 Sales Analysis, Party Ledger (via GL), Sales Stock Ledger. Portal: `/customer_statement`, `/product_wise_invoice_details`.
- → chapters 4, 5, 8, 12.

### Sales Invoice Item

- **Fields (7):** `custom_vat_apply_on` (VAT 13%/0%/Amount), `custom_vat_rate`, `custom_vat_amount`, `custom_vat` (Float), `custom_excise_duty`, `custom_excise_value`, `custom_total`. → the VAT/excise line scheme (chapter 4 §1).

### Quotation / Sales Order / Delivery Note

- **Fields:** parent totals `custom_total_amount_including_excise` / `custom_total_excise_amount` / `custom_total_vat_amount` / `custom_excise`; DN adds `custom_abbr`; audit block. Item children (Quotation/SO/DN Item, 7 each): the same VAT/excise line set as Sales Invoice Item.
- **Class:** `SalesOrder` (restore from Quotation Item), `DeliveryNote` (from SO

- Item) — warehouse restore + soften mandatory (chapter 4 §2).
- **Hooks:** `before_validate` / `before_save` → selling taxes handler;  
`validate` → `salesinvoice_taxes.validate_quotation/_sales_order/_delivery_note` (warehouse forcing). DN returns force negative signs.
- **JS:** `selling_taxes_common.js` (VAT UI), `sales_warehouse_common.js` (warehouse injection).
- → chapter 4 §1-3.

## 2. Buying transactions

### Purchase Invoice — second most customized

- **Fields (28):** VAT+excise+TDS totals ( `custom_total_vat_amount` / `_excise_amount` / `_tds_amount` / `_amount_including_excise` );  
`custom_tax_withholding_category_custom` (the custom TDS category);  
`numbering_custom_purchase_type` (→ Purchase Type) / `custom_p_type_code` /  
`custom_document_no` / `custom_name` / `custom_abbr` / `custom_fiscal_year` ;  
`Nepali dates` `custom_nepali_miti` / `custom_supplier_invoice_miti` ;  
`petroleum` `custom_pdo_no` / `custom_refinery` / `custom_ioc_challan_no/date` /  
`custom_ico_challan_miti` / `custom_store_receipt_no/date/miti` /  
`custom_voucher_receipt_no` / `custom_name_of_transportor` /  
`custom_vehicle_no` ; `custom_memo` ; audit block.
- **Class:** `PurchaseInvoice` — restore warehouse from PR/PO item.
- **Hooks:** `before_validate` / `before_save` / `validate` → purchase taxes  
( `purchase_taxes_handler.py` ); `validate` → **vehicle-mandatory**;  
`before_submit` → **excise\_ledger** (party-tag TDS-payable GL);  
`on_submit` → **stock\_revaluation** (backdated notice); audit + approval +  
fiscal-year.
- **JS:** `pi.js` (vehicle picker for `custom_subtype` ); `purchase_taxes_common.js`  
(global, VAT/TDS UI + buying-warehouse injection).
- **Reports:** Purchase Register, Advance Tax TDS, Gas Purchase, One Lakh Above,  
Vehicle Expense (JE+PI), Party Ledger.
- → chapters 4, 8, 12.

### Purchase Invoice Item

- **Fields (9):** VAT trio + TDS trio ( `custom_tds_apply_on` Percentage/Amount,  
`custom_tds_rate` , `custom_tds_amount` ) + `custom_excise_value` +  
`custom_total` + **custom\_subtype (→ Vehicle)** — the only item table with  
the vehicle link.

### Purchase Order

- **Fields (14):** VAT/excise/TDS totals; **approval** `custom_approver` (PO  
Reviewer), `custom_po_request_approver` (→ Purchase Order Request Approver),  
`custom_reason` , `custom_remarks` , `workflow_state` ; `custom_miti` ,  
`custom_abbr` ; audit block.
- **Class:** `PurchaseOrder` — restore from Supplier Quotation Item + Administrator  
workflow bypass on "Purchase Order Workflow".
- **Hooks:** `before_save` + `validate` → purchase taxes handler.
- **JS:** `purchase_taxes_common.js` .

### Purchase Receipt

- **Fields (24):** VAT/excise/TDS totals; `numbering_custom_receipt_type`  
(→ Receipt type) / `custom_p_type_code` / `custom_document_no` /  
`custom_name` / `custom_abbr` ; `custom_posting_miti` ; the same petroleum/  
IOC/store-receipt/transporter/vehicle fields as PI; `custom_remark` ; audit.
- **Class:** `PurchaseReceipt` — restore from PO Item.
- **Hooks:** `before_save` + `validate` → purchase taxes handler.

### Supplier Quotation

- **Fields (10):** `custom_preferred_quotation` (Check — drives the comparison  
report), `custom_miti` / `custom_valid_miti` , VAT/excise/TDS totals, audit.
- **Class:** `SupplierQuotation` — restore from RFQ Item.
- **Hooks:** `before_save` + `validate` → purchase taxes handler.
- **Reports:** Custom Supplier Quotation Comparison. Portal: `/rfq/<name>` .

### Purchase Order Item / Purchase Receipt Item / Supplier Quotation Item

- **Fields (8 each):** VAT trio + TDS trio + `custom_excise_value` +  
`custom_total` .

### Material Request

- **Fields (7):** `custom_transaction_miti` , `custom_required_miti` ,  
`custom_fiscal_year` , `custom_abbr` , audit.
- **Class:** `MaterialRequest` — Administrator workflow bypass on "Material  
Request One-Line Approver" + soften warehouse-mandatory.
- **Hooks:** `validate` → purchase taxes handler (warehouse defaults).
- **JS:** `material_request.js` (branch warehouse + approval reject dialog).

### Request for Quotation

- **Fields (5):** `custom_miti` , `custom_required_miti` , audit.

- **Class:** RequestforQuotation — restore from MR Item + soften mandatory.
- **Hooks:** validate → purchase taxes handler. Whitelisted  
create\_supplier\_quotation overridden by the RFQ portal (chapter 4 §9).

### 3. Accounting

#### Payment Entry

- **Fields (13):** numbering custom\_p\_type (→ Payment - Receipt Type) / custom\_p\_type\_code / custom\_document\_no / custom\_document\_word / custom\_name / custom\_abbr / custom\_fiscal\_year; custom\_posting\_miti / custom\_chequereference\_miti; custom\_remark; audit.
- **Hooks:** audit (numbering) + approval + fiscal-year. **No** class override.
- **JS:** payment\_entry.js — **Cheque Bounce** button (posts reversing GL, chapter 4 §5) + auto\_update\_document\_no.js (auto document number) + party scoping.
- **Reports:** Receipt Register, Party Ledger.

#### Journal Entry

- **Fields (12):** numbering custom\_p\_type (→ JV Type) / custom\_p\_type\_code / custom\_document\_no / custom\_document\_word / custom\_name / custom\_abbr / custom\_fiscal\_year; custom\_posting\_miti / custom\_reference\_miti; audit.
- **Hooks:** validate → **vehicle-mandatory** (accounts rows); audit (numbering) + approval + fiscal-year.
- **JS:** journal\_entry.js (vehicle picker for accounts.custom\_subtype) + auto\_update\_document\_no.js.
- **Reports:** Vehicle Expense (JE lines), Party Ledger (JE split per account), Loan Summary / Net Position / financial statements (via GL).

#### GL Entry / Journal Entry Account

- GL Entry: audit block only, plus it's the source for most financial reports; patch add\_gl\_entry\_fin\_stmt\_agg\_index adds the consolidated-report index. excise\_ledger.modify\_gl\_entries party-tags TDS-payable lines at PI submit.

#### Bank Account / Cost Center / Tax Withholding Category / Warehouse / Fiscal Year / Account

- **Account (5 fields):** custom\_sub\_type\_list (→ **Vehicle List** — the per-account vehicle whitelist the vehicle pickers read), custom\_department, audit. → chapters 4 §7, 11 Part C.
- The rest: audit block only (Cost Center, Tax Withholding Category, Warehouse, Fiscal Year, Bank Account + naming series).

### 4. HR / Payroll / Attendance

#### Attendance

- **Fields (8):** custom\_worked\_on\_holiday (Check), custom\_late\_entry / custom\_early\_entry / custom\_early\_exit / custom\_late\_exit (Duration — shift deviations), audit.
- **Hooks (all validate):** set\_holiday\_flag (payroll), set\_shift\_deviation\_fields, enforce\_late\_arrival\_half\_day (biometric). Uses validate not before\_save because device attendance inserts already-submitted (chapter 3 §3).
- **JS:** attendance.js — read-only Checkin Log table.
- **Reports:** Monthly Attendance BS, Monthly Attendance Summary BS, Work On Holiday BS. → chapter 3.

#### Employee Checkin

- **Fields (3):** custom\_company (fetch from employee), audit.
- **Hooks:** after\_insert → reconcile\_with\_existing\_attendance (chapter 3).
- Patch add\_company\_to\_employee\_checkin; company\_scoped\_attendance\_device\_id.

#### Employee — heavily extended (28 fields)

- **App fields:** custom\_document\_user (signatory User for Document Generator), custom\_signature\_image, custom\_ot\_eligibility (attendance allowances), custom\_attendance\_allowances (→ Employee Attendance Allowance), custom\_employee\_cost\_center\_manager (→ Employee Cost Center Manager), custom\_pan, custom\_ssf\_id\_no, custom\_citizenship\_number, custom\_districtplace\_of\_issue, custom\_date\_of\_citizenship\_issue, custom\_abbr, audit.
- **Hooks:** validate → validate\_unique\_device\_id (device id unique per company).
- (Note: many other Employee custom fields — grade, default\_shift, leave\_approver, employment\_type, health\_insurance\_, ifsc/micr, pan\_number, payroll\_cost\_center — are **HRMS-provided**, not this app's.)\*
- → chapters 3, 7.

#### Payroll Entry

- **Fields (3):** audit only.
- **JS:** `payroll_entry.js` — **Calculate Attendance Allowances** button (chapter 3 §6).

### Salary Component (10 fields)

- **App fields:** `custom_is_attendance_driven`, `custom_condition_type`, `custom_threshold_hours`, `custom_time_offset_hours`, `custom_unit`, `custom_default_rate`, `custom_summary_group`, `component_type`, `audit` — the attendance-allowance rule definition (chapter 3 §6).

### Additional Salary

- **Fields (3):** `custom_source` (tags allowance drafts as "Nepal HRMS Attendance Allowance"), `audit`.

### Shift Type

- **Fields (4):** `custom_company` (patch), `custom_late_arrival_cutoff_time` (drives the auto Half-Day rule), `audit`. Shift Types are per-company.

### Salary Slip / Leave Application / Leave Allocation / Expense Claim / Department

- **Salary Slip:** audit only. **Report:** Avinas Salary Statement.
- **Leave Application:** audit; patch `leave_application_dynamic_approval` brings it under Dynamic Approval (native `leave_approver` fields hidden). **Report:** Yearly Leave Details BS.
- **Leave Allocation:** audit. **Report:** Yearly Leave Details BS (allocations).
- **Department (7):** HRMS approver tables + `payroll_cost_center` (mostly HRMS-provided) + `audit`.

## 5. Stock

### Stock Adjustment (custom doctype — chapter 9/11)

Quantity-only correction; `engine_patch.py` preserves stock value through reposts. Not to be confused with core Stock Entry.

### Stock Entry / Stock Reconciliation / Batch / Serial No / Landed Cost Voucher

- **Stock Entry (3):** audit only.
- **Stock Reconciliation (5):** `custom_abbr`, `custom_posting_miti`, `audit`.
- **Batch (5):** company + naming series + `audit`.
- **Landed Cost Voucher (4):** `custom_remarks` + `audit`.
- **Reports:** Sales Stock Ledger (via Sales Invoice + SLE).

## 6. Masters

### Customer (8 fields)

- **Fields:** `custom_company`, `custom_abbr`, **credit control** `custom_amount_limit` / `custom_bill_count` / `custom_days_limit`, `audit`.
- **JS (doctype.js):** `party_duplicate_check.js` (same-company duplicate name/tax\_id warning), `party_default_account.js` (default accounts row).
- **Hooks:** audit + company-matching. Company scoping via Company Filter.
- → chapter 4 §10.

### Supplier (7 fields)

- **Fields:** `custom_company`, `custom_abbr`, `custom_territory`, `custom_payment_term_days` (auto PI due date), `audit`.
- **JS:** `party_duplicate_check.js`, `party_default_account.js`.

### Item (12 fields)

- **Fields:** `custom_company`, `custom_abbr`... actually `custom_code`, `custom_hs_code`, `custom_item_type`, `custom_excise_duty`, `custom_is_depend_on_branch`, `custom_buying_warehouse` / `custom_selling_warehouse` / `custom_branch_wise_warehouse` (→ Branch Wise Warehouse — the per-branch warehouse map), `audit`.
- **JS:** `item_default_account.js` (default item-defaults row).
- **Overrides:** Item Price auto-company stamping ( `auto_insert_item_price.py` ); `get_item_details` wrapped for branch warehouse.
- → chapter 4 §3.

### Item Price (5 fields)

- **Fields:** `custom_company` (mandatory — auto-stamped), `custom_total_vat_inclusive`, `audit`. In `master_doctypes` for naming series.

### Company (10 fields — mostly HRMS/payroll)

- **App field:** `custom_document_stamp` (Attach Image — used by Document

Generator letters). Others (payroll payable/advance/expense accounts, basic/hra/arrear components) are HR-payroll config. Per-company: CBMS Config, Company Filter, numbering abbr, `tax_id` (CBMS seller PAN).

## Address / Contact / Branch / Item Group / Customer Group / Supplier Group / Asset Category

- Company + naming series + audit (some add domain fields: Address `tax_category / is_your_company_address`; Contact `is_billing_contact`; Branch is used by branch-wise warehouse). Company scoping applies.

---

## 7. Custom & DB-only doctypes

The 28 file-based custom doctypes are in [chapter 9](#) + [chapter 11 Part B](#);  
the 13 DB-only ones (Purchase Type, Receipt type, Payment - Receipt Type, JV Type, Vehicle List, the approver/calc tables, Sub-Ledger Category...) in [chapter 11 Part C](#).

---

## 8. Everything else (audit block only)

The remaining ~250 doctypes with custom fields carry **only** the audit block (`custom_created_by / custom_created_on` + often `custom_modified_by`, plus `custom_company / custom_naming_series` where applicable) and, if audited, the audit `doc_events`. No other app behavior. This covers most CRM (Lead, Opportunity, Prospect, Campaign...), HR (the many Employee-\* and Leave-\* masters, Interview, Appraisal...), manufacturing (BOM, Work Order, Job Card, Operation, Routing, Workstation...), quality, projects, and setup doctypes. They inherit the cross-cutting behavior in \$0 but have no dedicated fields, hooks, JS, or reports of their own.

# avinas1 Site Customization Inventory — Code + Database

Chapter 14 of the technical documentation. A full audit of **every customization present on the avinas1 site**, from both sources: **(A) app code** (hooks, class overrides, JS files, shipped fields — version controlled) and **(B) the database** (Custom Fields, Property Setters, Client Scripts, custom Print Formats, Notifications, Custom DocPerm — created via the Customize/UI, **not in the repo**). Data pulled live from avinas1.

This chapter answers "go doctype by doctype: is it customized, and how." For the mechanism behind each feature see chapters 2–8; for the by-doctype field detail see chapter 13.

## 1. Totals

| Customization type               | Count                                                                   | In repo?                                                                                                   |
|----------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Custom Fields                    | 1,357 (across 379 doctypes)                                             | partly — only 13 bundles exported to <code>custom/*.json</code> ; the rest are DB-only or added by patches |
| Property Setters                 | 421 (across ~60 doctypes)                                               | × DB-only                                                                                                  |
| Client Scripts                   | 7                                                                       | × DB-only                                                                                                  |
| Custom Print Formats             | 8 (Jinja)                                                               | × DB-only                                                                                                  |
| Notifications                    | 6                                                                       | × DB-only                                                                                                  |
| Custom DocPerm (permission rows) | 490                                                                     | × DB-only                                                                                                  |
| Server Scripts                   | 0                                                                       | —                                                                                                          |
| Workflows                        | 0 stored (Dynamic Approval creates them <b>on demand</b> at setup time) | code (chapter 2)                                                                                           |

**396 doctypes** carry at least one customization: **121** have meaningful customization (property setters / client scripts / print formats / notifications / >3 fields); **275** carry only the audit block ( `custom_created_by/on` , `custom_modified_by` ) plus standard permission rows.

⚠ **Reproducibility gap.** The Property Setters, Client Scripts, Print Formats, Notifications and most Custom Fields live only in the avinas1 database. A fresh `bench get-app avinashgroup_app + install-app` will **not** recreate them — they must be exported as fixtures or migrated with the site. Only the code-side customizations (hooks, class overrides, JS, the 13 exported field bundles, the 2 balance-confirmation templates) travel with the repo.

## 2. Database-only customizations (not in the app code)

These are the ones the earlier by-feature/by-file chapters did **not** cover because they aren't in the repo.

### 2.1 Client Scripts (7)

DB-stored form scripts. Several **duplicate or predate** the app's file-based JS; note the ON/OFF state — disabled ones are leftovers, enabled ones are live and can **coexist/conflict** with the app scripts.

| Doctype / View          | Name                                            | State | What it does                                                                                                                                   | Note                                                                                                                        |
|-------------------------|-------------------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Journal Entry / Form    | <i>Journal Entry</i>                            | ON    | vehicle picker: caches account→vehicle subtypes, filters <code>custom_subtype</code> per account row                                           | <b>Duplicates</b> the app's <code>journal_entry.js</code> (chapter 4 §1). Two copies of the same behavior run — consolidate |
| Material Request / Form | <i>Fetch Approver of Request Creator</i>        | OFF   | auto-fills <code>custom_creator_employee</code> from the session user and builds an approver chain by walking Employee <code>reports_to</code> | disabled; an alternate approval approach superseded by Dynamic Approval                                                     |
| Payment Entry / Form    | <i>Cheque/Reference date as of Posting Date</i> | ON    | sets <code>reference_date = posting_date</code> on load and on posting-date change                                                             | <b>live, app-code has no equivalent</b> — a genuine DB-only behavior                                                        |
| Purchase Invoice / Form | <i>Purchase Invoice</i>                         | OFF   | old VAT/TDS field toggles using the stale <b>Percentage (%)</b> default                                                                        | disabled; superseded by <code>purchase_taxes_common.js</code> . Leftover of the pre-migration VAT scheme                    |
| Purchase Order / Form   | <i>Test</i>                                     | OFF   | <code>custom_approver_change</code> → calls <code>get_reviewer_managers</code> to fill <code>custom_po_request_approver</code>                 | disabled; builds the PO approver chain (references a <code>get_reviewer_managers</code> method)                             |
| Purchase Order / Form   | <i>Take me To Supplier Quotation</i>            | ON    | adds a "View Supplier Quotation Comparison" button routing to that report with <code>purchase_order</code> prefilled                           | live, useful — not in app code                                                                                              |
| Sales Invoice / Form    | <i>Sales Invoice</i>                            | OFF   | <code>update_fiscal_year</code> sets <code>custom_fiscal_year</code> from posting date; commented-out return naming-series switch              | disabled; the app now handles fiscal year server-side                                                                       |

**Action items surfaced:** the JE script duplicates app JS (dedupe); the PE reference-date and PO comparison-button scripts are live-only-in-DB (export as fixtures or port to the app if they must survive a re-install); the OFF scripts are stale leftovers (safe to delete).

### 2.2 Custom Print Formats (8, all Jinja)

Not in the repo — the actual printed layouts for the business documents.

| Doctype          | Print Format                 | Note                                                                                                   |
|------------------|------------------------------|--------------------------------------------------------------------------------------------------------|
| Sales Invoice    | Avinash Sales Invoice        | main tax-invoice layout (reads <code>custom_print_count</code> for the IRD copy title — chapter 4 §5a) |
| Sales Invoice    | Avinash Sales/Return Invoice | combined sale/return layout                                                                            |
| Sales Invoice    | Avinash Sales Return         | credit-note layout                                                                                     |
| Purchase Invoice | Avinash Purchase Invoice     |                                                                                                        |
| Purchase Invoice | A5 Avinash Purchase Invoice  | A5-size variant                                                                                        |
| Purchase Order   | Avinash Purchase Order       |                                                                                                        |
| Journal Entry    | Avinash Journal Entry        |                                                                                                        |
| Supplier         | IRS 1099 Form                | ERPNext-standard (not Avinash-specific)                                                                |

These should be exported as fixtures ( `Print Format` filtered to the `Avinash *` names) so they ship with the app — today they exist only on `avinash1`.

2.3 Notifications (6)

All email; mostly ERPNext/HRMS-standard, none Avinash-specific business logic: Exit Interview Scheduled (Exit Interview), Training Scheduled (Training Event), **Material Request Receipt Notification** (Material Request, on value change — "`{{ doc.name }}` has been received"), Retention Bonus alert, New-Fiscal-Year notification, Training Feedback. Only the Material Request one is workflow-relevant to this business.

2.4 Property Setters (421) — form-layout & behavior overrides

These modify **existing ERPNext field properties** (not new fields). The patterns across the customized transaction doctypes:

- **Naming series patterns** set here (complement the `naming_series.py` engine): e.g. Purchase Invoice `naming_series.options = .{custom_abbr}.PUR-#####{custom_fiscal_year}`, Journal Entry `ACC-JV-.YYYY.-, Item autoname = naming_series:`
- **Hiding standard fields/sections** to declutter the Nepali forms — e.g. Purchase Invoice hides `taxes_section`, `tax_category`, `shipping_rule`, `incoterm`, `is_paid`, `supplier_invoice_details`, `set posting time`, `named_place` ... (17+ hides); Material Request Item hides 8 standard sections; Payment Entry hides `taxes_and_charges_section`, `accounting_dimensions`.
- **List-view columns** ( `in_list_view` ) retargeted to the custom fields — e.g. Payment Entry/Journal Entry show `custom_name`, `custom_document_no`, `custom_p_type`; Purchase/Sales Invoice show `title + posting_date`.
- **Mandatory/optional flips** — e.g. Item `item_code.reqd = 0`, `item_name.reqd = 1`, `naming_series.reqd = 0`; the two 0-field bundles (Packed Item `rate` read-only, Purchase Taxes and Charges `add_deduct_tax` default = Deduct) are pure property setters.
- **field\_order** rewritten on the heavily-customized doctypes to interleave the custom fields.

Highest property-setter counts: Purchase Invoice (45), Sales Invoice (40), Purchase Receipt (32), Supplier Quotation (21), Payment Entry (19), Item (17), Supplier Quotation Item (15), Sales Invoice Item / Purchase Invoice Item (13), Sales Order (13), Delivery Note (11).

2.5 Custom DocPerm (490)

Permission-row overrides across ~150 doctypes (e.g. Fiscal Year 15, Company 13, Contact 13, Supplier 13, Item 12, Address 11, Customer 11, Material Request 11). These adjust which roles can read/write/create/submit each doctype relative to the ERPNext defaults. They are DB-only and should be exported if the role model must be reproduced.

3. Complete per-doctype matrix (meaningful customization)

Counts: **CF** custom fields · **PS** property setters · **CS** client scripts · **PF** print formats · **N** notifications · **DP** custom docperm. Doctypes with only the audit block are in §4.

| Doctype               | CF | PS | CS | PF | N | DP |
|-----------------------|----|----|----|----|---|----|
| Purchase Invoice      | 40 | 45 | 1  | 2  | 0 | 0  |
| Sales Invoice         | 33 | 40 | 1  | 3  | 0 | 6  |
| Purchase Receipt      | 32 | 32 | 0  | 0  | 0 | 0  |
| Supplier Quotation    | 15 | 21 | 0  | 0  | 0 | 8  |
| Payment Entry         | 18 | 19 | 1  | 0  | 0 | 3  |
| Purchase Order        | 16 | 12 | 2  | 1  | 0 | 9  |
| Journal Entry         | 14 | 9  | 1  | 1  | 0 | 5  |
| Employee              | 38 | 12 | 0  | 0  | 0 | 6  |
| Item                  | 13 | 17 | 0  | 0  | 0 | 12 |
| Sales Order           | 10 | 13 | 0  | 0  | 0 | 0  |
| Quotation             | 8  | 9  | 0  | 0  | 0 | 0  |
| Delivery Note         | 10 | 11 | 0  | 0  | 0 | 0  |
| Request for Quotation | 8  | 8  | 0  | 0  | 0 | 0  |

|                                         |    |    |   |   |   |    |
|-----------------------------------------|----|----|---|---|---|----|
| Material Request                        | 11 | 6  | 1 | 0 | 1 | 11 |
| Company                                 | 14 | 4  | 0 | 0 | 0 | 13 |
| Customer                                | 11 | 9  | 0 | 0 | 0 | 11 |
| Supplier                                | 8  | 5  | 0 | 1 | 0 | 13 |
| Supplier Quotation Item                 | 12 | 15 | 0 | 0 | 0 | 0  |
| Sales Invoice Item                      | 8  | 13 | 0 | 0 | 0 | 0  |
| Purchase Invoice Item                   | 11 | 13 | 0 | 0 | 0 | 0  |
| Purchase Receipt Item                   | 10 | 3  | 0 | 0 | 0 | 0  |
| Purchase Order Item                     | 10 | 1  | 0 | 0 | 0 | 0  |
| Quotation Item                          | 8  | 1  | 0 | 0 | 0 | 0  |
| Sales Order Item                        | 8  | 1  | 0 | 0 | 0 | 0  |
| Delivery Note Item                      | 8  | 3  | 0 | 0 | 0 | 0  |
| Request for Quotation Item              | 1  | 10 | 0 | 0 | 0 | 0  |
| Material Request Item                   | 0  | 9  | 0 | 0 | 0 | 0  |
| Supplier Group                          | 6  | 9  | 0 | 0 | 0 | 0  |
| Customer Group                          | 6  | 7  | 0 | 0 | 0 | 9  |
| Item Group                              | 6  | 6  | 0 | 0 | 0 | 0  |
| Price List                              | 6  | 6  | 0 | 0 | 0 | 6  |
| Asset Category                          | 6  | 7  | 0 | 0 | 0 | 0  |
| Attendance                              | 11 | 0  | 0 | 0 | 0 | 5  |
| Salary Component                        | 12 | 0  | 0 | 0 | 0 | 3  |
| Fiscal Year                             | 2  | 4  | 0 | 0 | 1 | 15 |
| Vehicle                                 | 6  | 5  | 0 | 0 | 0 | 4  |
| Account                                 | 6  | 1  | 0 | 0 | 0 | 7  |
| Stock Reconciliation                    | 6  | 2  | 0 | 0 | 0 | 1  |
| Leave Application                       | 4  | 2  | 0 | 0 | 0 | 9  |
| Salary Slip                             | 4  | 2  | 0 | 0 | 0 | 4  |
| Item Price                              | 7  | 1  | 0 | 0 | 0 | 3  |
| Contact                                 | 7  | 3  | 0 | 0 | 0 | 13 |
| Address                                 | 8  | 3  | 0 | 0 | 0 | 11 |
| Shift Type                              | 4  | 0  | 0 | 0 | 0 | 4  |
| Stock Entry                             | 4  | 1  | 0 | 0 | 0 | 0  |
| Landed Cost Voucher                     | 5  | 1  | 0 | 0 | 0 | 0  |
| Job Card                                | 4  | 1  | 0 | 0 | 0 | 0  |
| Pick List                               | 4  | 1  | 0 | 0 | 0 | 0  |
| POS Invoice                             | 4  | 1  | 0 | 0 | 0 | 0  |
| Purchase Taxes and Charges              | 0  | 5  | 0 | 0 | 0 | 0  |
| Packed Item                             | 0  | 1  | 0 | 0 | 0 | 0  |
| Journal Entry Account                   | 1  | 1  | 0 | 0 | 0 | 0  |
| Communication                           | 6  | 3  | 0 | 0 | 0 | 0  |
| Department                              | 10 | 0  | 0 | 0 | 0 | 5  |
| Designation                             | 9  | 0  | 0 | 0 | 0 | 3  |
| Expense Claim                           | 4  | 0  | 0 | 0 | 0 | 9  |
| Expense Claim Type                      | 6  | 0  | 0 | 0 | 0 | 3  |
| Branch                                  | 6  | 0  | 0 | 0 | 0 | 3  |
| Bank Account                            | 5  | 0  | 0 | 0 | 0 | 3  |
| Batch                                   | 5  | 0  | 0 | 0 | 0 | —  |
| Holiday List                            | 6  | 0  | 0 | 0 | 0 | 2  |
| Leave Type                              | 6  | 0  | 0 | 0 | 0 | 4  |
| Mode of Payment                         | 6  | 0  | 0 | 0 | 0 | 4  |
| Timesheet                               | 5  | 0  | 0 | 0 | 0 | 7  |
| Vehicle Log                             | 5  | 0  | 0 | 0 | 0 | 2  |
| Employee Tax Exemption Proof Submission | 11 | 0  | 0 | 0 | 0 | 5  |
| Employee Tax Exemption Declaration      | 9  | 0  | 0 | 0 | 0 | 5  |
| Terms and Conditions                    | 3  | 0  | 0 | 0 | 0 | 7  |
| Exit Interview                          | 2  | 0  | 0 | 0 | 1 | 0  |
| Retention Bonus                         | 2  | 0  | 0 | 0 | 1 | 0  |
| Training Event                          | 4  | 0  | 0 | 0 | 1 | 2  |
| Training Result                         | 2  | 0  | 0 | 0 | 1 | 0  |

(Plus the 28 file-based + 13 DB-only custom doctypes from chapters 9 & 11, and Employee Checkin CF3/DP9, Payroll Entry CF4, Salary Structure CF4, Warehouse CF2/DP9, Cost Center CF2/DP7, Territory DP8, Currency DP8, Employee Referral DP8 — all carrying audit fields + permission tweaks.)

#### 4. The 275 audit-only doctypes

These carry **only** the audit block (created/modified tracking) plus, for some, standard permission rows — no dedicated fields, property setters, scripts,

formats, or notifications. They inherit the cross-cutting behavior (audit stamping, dynamic-approval short-circuit, and — for transaction types — fiscal-year filtering) but the app does nothing else specific to them. They span CRM (Lead, Opportunity, Prospect, Campaign, Competitor...), the many HR masters (Appraisal, Interview, *Employee Onboarding/Separation/Promotion/Transfer, Leave Policy/Period/Encashment, Gratuity, Job ...*), manufacturing (BOM, *Work Order, Operation, Routing, Workstation*, Production Plan, Job Card details), quality (Quality ), *projects (Project Template/Type, Task Type, Milestone)*, accounting masters (Tax Category/Rule, Payment Terms Template, Finance Book, Cost Center Allocation, Period Closing Voucher, Repost ), *stock masters (Bin, Serial and Batch Bundle, Putaway Rule, Stock Entry Type, Stock Ledger Entry)*, and *platform doctypes (Email Account, Webhook, OAuth, Print Settings, Role, Workflow\*)*. The full list is 275 names; representative coverage above.

---

## 5. Reproducing the site elsewhere (fixtures to add)

---

Only the code side ships today. To make another site match avinas1, export these as fixtures in `hooks.py` (or migrate the DB):

```
fixtures = [
    "Custom Field",          # or filter to name like "%custom_" per doctype
    "Property Setter",
    {"dt": "Client Script", "filters": {"dt": ["in", [
        "Payment Entry", "Purchase Order", "Journal Entry"]]}}, # the live ones
    {"dt": "Print Format", "filters": {"name": ["like", "Avinash %"]}},
    {"dt": "Custom DocPerm"}, # if the role model must match
    # (Document Template balance-confirmation already exported)
]
```

Then the DB-only customizations become version-controlled. Until then, treat this chapter as the source of truth for what exists on avinas1 beyond the repo.

# Avinash Group ERP — User Guide Overview

---

Welcome! This guide explains, in plain language, the custom features added on top of ERPNext for the Avinash Group companies (Nepal Gas group and allied businesses). It is written for the people who use the system every day — accountants, HR staff, purchase/sales officers, and administrators.

For developers, the matching technical reference lives in [../technical/](#).

## What's in this guide

---

| Chapter                                      | Who it's for                               | What it covers                                                                                      |
|----------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <a href="#">2. Approvals</a>                 | everyone who submits or approves documents | how the multi-level approval workflow works: submitting, approving, rejecting                       |
| <a href="#">3. Attendance &amp; HR</a>       | HR staff                                   | biometric devices, check-ins, fixing attendance, allowances, HR reports                             |
| <a href="#">4. Buying, Selling &amp; VAT</a> | accounts, purchase & sales staff           | the VAT 13% / 0% / Amount scheme, excise, TDS, warehouses, voucher numbers, vehicles, cheque bounce |
| <a href="#">5. CBMS / IRD e-Billing</a>      | accounts staff                             | how invoices are reported to IRD automatically and what to do when something fails                  |
| <a href="#">6. Document Generator</a>        | accounts & admin staff                     | producing balance-confirmation letters and other formal documents                                   |
| <a href="#">7. Reports</a>                   | accounts, sales, HR, management            | every custom report: what it shows and how to use it                                                |
| <a href="#">8. Administrator Setup</a>       | System Managers                            | fiscal-year access, company filters, audit trail, devices, templates — all setup checklists         |

## The big ideas (read this once)

---

**Everything is company-scoped.** The group runs several companies in one system. Master records (customers, suppliers, items) carry a Company, and the system automatically limits dropdowns to the document's company and blocks cross-company mistakes. If a value you expect is missing from a dropdown, check the Company field first.

**Nepali dates everywhere.** Transactions store the English (AD) date, but show and filter by the Nepali (BS) "Miti" too. Reports have BS month pickers; attendance and payroll run on BS months; letters print BS dates.

**Approvals are configurable.** Any document type can be routed through a multi-level approval chain without programming — see chapter 2.

**Invoices go to IRD automatically.** When you submit a Sales Invoice it is reported to the IRD CBMS system in the background. You never wait for it, and a synced invoice can no longer be cancelled — corrections go through a Sales Return.

**Attendance comes from fingerprint devices.** Punches flow from the devices (via the K40 Bridge or direct push) into Employee Checkins and then daily Attendance, automatically flagged for lateness and holidays.

## Signing in

---

- Desk (staff): `http://<your-server>/app`
- Customer portal: `/customer_statement`, `/product_wise_invoice_details`, `/place_order` (requires a portal login with the Customer role)
- Supplier RFQ portal: link received by email

## Getting help

---

If a document refuses to save, read the error message carefully — most messages in this system are specific ("Row #2: Vehicle is mandatory...", "Company Mismatch...", "Only may make changes...") and this guide's chapters explain each one. When in doubt, contact your System Manager.

# Approvals — User Guide

---

The Dynamic Approval System routes documents (Purchase Orders, Material Requests, Leave Applications, ...) through a chain of approvers before they are submitted. This chapter explains what you see and do; setup is in [chapter 8](#).

## How an approval flows

---

1. **You create the document** as usual. If it is under approval control, you will see an **Approval Hierarchy** table on the form.
2. **Add your approvers** (if your document type lets you choose them): each row is one level — Level 1 is approved first, then Level 2, and so on. Management may have configured additional *fixed approvers* that are automatically appended after your levels; you don't need to add those.
3. **Submit for Approval.** The document moves to *Pending Approval* and the Level-1 approver gets an email.
4. **Each approver approves in turn.** The document stays in *Pending Approval* until the last level approves — then it becomes *Approved* and is submitted.
5. If any approver **rejects**, the document becomes *Rejected* and is editable again. Fix it and submit again from the start.

## What you see on the form

---

- A **progress banner** at the top: ✓ Level 1 → ✕ Level 2 (current) → ○ Level 3 , plus a line telling you whose turn it is — "Your approval is required at Level 2" or "Waiting for at Level 2".
- While the document is pending and it is **not your turn**, the form is read-only and the Approve/Reject buttons are hidden. Only the current approver can act or edit.
- Hierarchy rows that are already approved are **locked** — you cannot edit or delete them.
- An **Approval History** table records every step (created, submitted, approved at each level, rejected) with user and timestamp.

## Approving and rejecting

---

- **Approve:** open the document, use the workflow action button (top right) → *Approve*. If you are an intermediate level, the document stays pending and moves to the next approver; if you are the last level, it is submitted.
- **Reject:** choosing *Reject* opens a mandatory **Reason for Rejection** box. The reason is saved on the document (or as a comment) so there is always an audit trail. You cannot reject without a reason.

## Emails

---

The next approver is emailed at every step. Rejection sends a distinct email to the requester.

## Common questions

---

### Why did my document get approved instantly?

No approval rule matched it (for example, its company or category has no active rule). It is auto-approved with a history entry "Auto-approved (no matching approval rule)". If that's wrong, ask your admin to check the Dynamic Approval Setting.

### Why can't I edit a pending document?

Only the current approver may change a pending document. You'll see "Only may make changes". Wait for the flow to finish, or ask the current approver to reject it back to you.

### Can I skip a level?

No — even the Administrator advances one level at a time.

### The document was rejected — now what?

It is back in an editable state. Make the correction and submit for approval again; the chain restarts from Level 1.

### Who chooses the approvers?

Depends on the rule: your own hierarchy rows (Levels 1..N) are yours to fill; fixed approvers configured by management are appended automatically after them.

# Attendance & HR — User Guide

---

For HR Managers and HR Users. Covers how fingerprint punches become attendance, how to fix gaps, allowances, and the HR reports.

## 1. How attendance works (the happy path)

---

1. An employee punches on the fingerprint device (ZKTeco K40, Hikvision, or an HTMS-connected controller).
2. The **K40 Bridge** (a small Windows program at each site) — or the device itself, if it pushes directly — sends the punches to ERPNext.
3. Each punch becomes an **Employee Checkin**. Punches alternate IN → OUT → IN for the day, sorted by time. Re-sent punches are ignored (no duplicates), and a punch that arrives late slots into the right place automatically.
4. The Shift Type's auto-attendance job turns the day's checkins into a submitted **Attendance** record.
5. On the Attendance, the system automatically fills:
  - **Late entry / early exit minutes** (compared to the shift times),
  - **Half Day** if the first punch is after the shift's late-arrival cutoff

(with leave type "Leave Without Pay"),

  - **Worked on Holiday** if the date is in the employee's holiday list.

On the Attendance form you can see the day's **Checkin Log** (IN in green, OUT in orange) right below the status.

### Employee setup that must be right

- Every employee needs their **Attendance Device ID** set to the user ID they have *on the device* (for HTMS: the Emp No, with leading zeros exactly as stored).
- The same device ID may be reused **across companies** but must be unique **within** a company — the form blocks duplicates.
- The employee's Company must match the device's Company (a punch on Company A's device only matches Company A's employees).

## 2. Fixing attendance (Attendance Fix)

---

When a bridge PC was off, a device was offline, or attendance is otherwise wrong for a period, use **Attendance Fix**:

1. New Attendance Fix → choose the **Shift Type** and the **From/To dates**. Optionally narrow to one **Employee**, one **Company**, or specific **Devices**.
2. **Submit**. The job runs in the background; the form shows a live progress bar and refreshes itself.
3. When it finishes you get counters (attendance created/updated, checkins relinked, stale Absent rows deleted) and a day-by-day log.

What it does per employee/day: creates attendance from checkins where missing; replaces a wrong "Absent" with the real attendance when checkins exist; links orphan checkins to existing attendance; marks Absent where there are no checkins (skipping holidays); and never touches days that already look right.

Tip: when a bridge comes back online after an outage, the recovery email you receive tells you the date range to run Attendance Fix for.

## 3. Devices, syncing and alerts

---

- Each physical device is a **Biometric Device** record: name, serial number (must match the real hardware serial), Company, and enabled flag. Punches from unknown or disabled serials are rejected.
- **Force Bridge Sync**: on the Biometric Device form, this button asks the bridge to sync that device now; the result appears within a minute or two.
- **Alerts**: add emails to the device's *Alert Recipients* table and set the threshold (default 120 minutes). If the bridge stops reporting for longer than the threshold, those people get one "down" email, and one "recovered" email when it returns.
- The bridge itself is installed from `K40BridgeSetup.exe` on a Windows PC on the device's network — see the install runbook ( `k40_bridge/SETUP.md` ) or ask your administrator.

## 4. Attendance allowances (payroll)

---

Allowances that depend on attendance (meal allowance, tea/conveyance, holiday work, overtime-style rules) are computed from Attendance records:

- Each allowance is a **Salary Component** marked *attendance driven*, with a condition (Present / Half Day / Worked on Holiday / Late Stay After / Early Entry Before / Late Arrival After / Working Hours threshold), a unit (per day / per hour / flat per period), and a default rate.
- Per-employee overrides live on the Employee form in the **Attendance**

**Allowances** table (set a different rate, or untick *Eligible* to exclude the employee). Overtime-type rules also require the employee's **OT Eligibility** checkbox.

- On a **Payroll Entry**, click **Calculate Attendance Allowances** (under the Nepal HRMS menu). This creates/updates draft **Additional Salary** records — one per employee per component — and reports what was created or skipped. You can run it again safely; it replaces its own previous drafts.

## 5. HR reports (all in Nepali BS months)

| Report                               | Use it for                                                                                                                                                                |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Monthly Attendance BS</b>         | the day-by-day register for a BS month: in/out times, hours, late minutes, status, holiday work, and a column per attendance allowance                                    |
| <b>Monthly Attendance Summary BS</b> | one line per employee for the month — present days, OT hours, late time, meal/tea groups, leave this month / last month / year-to-date. Mirrors the physical muster sheet |
| <b>Work On Holiday BS</b>            | who worked on holidays, month by month across the fiscal year                                                                                                             |
| <b>Yearly Leave Details BS</b>       | leave taken per BS month, total allocation, and remaining balance per employee                                                                                            |
| <b>Avinas Salary Statement</b>       | the salary sheet for a BS month: all earnings and deductions in columns, grouped by designation with sub-totals, plus last month's net for comparison                     |

Pick the fiscal year / BS month in the filters; you can also switch to an AD date range where offered.

# Buying, Selling, VAT & Vouchers — User Guide

For accounts, purchase and sales staff. Covers the custom tax fields on every buying/selling document, warehouses, voucher numbers, vehicles, credit terms, and cheque bounce.

## 1. VAT on every line — the "Apply On" selector

Every item row on Purchase Invoice / Order / Receipt / Supplier Quotation and Sales Invoice / Quotation / Sales Order / Delivery Note has:

| Field               | What to do                                                                                                          |
|---------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>VAT Apply On</b> | VAT 13% (default — VAT is calculated for you), VAT 0% (tax-free line), or Amount (you type the VAT amount yourself) |
| <b>Excise Value</b> | type the excise duty for the line if any — it is never auto-calculated                                              |
| <b>Total</b>        | shown automatically = line amount + excise. VAT is charged on this                                                  |
| <b>VAT Amount</b>   | calculated (read-only) except in Amount mode                                                                        |

The system automatically writes the matching **Excise Duty** and **VAT** rows into the taxes table and recalculates the grand total — don't add those tax rows by hand.

**Returns:** on a Purchase/Sales Return, quantities and VAT are automatically made negative — just enter positive numbers as usual.

## 2. TDS on Purchase Invoices

1. Pick the **Tax Withholding Category** in the invoice's custom TDS field (this is separate from ERPNext's standard TDS field — use the custom one).
2. Tick **Apply TDS** on the item rows it applies to.
3. Per row choose Percentage (%) (rate comes from the category) or Amount (type it). The TDS total is deducted in the taxes table automatically and, on submit, the TDS ledger entry is tagged with the supplier so the TDS ledger reconciles per party.

## 3. Warehouses pick themselves

Items can have different warehouses per **branch** (and buying vs selling). The mapping lives on the Item (*Branch Wise Warehouse* table, with item-level defaults as fallback). When you pick an item on a document, the correct branch warehouse is filled in automatically. If you choose a warehouse manually and no mapping exists, your choice is kept.

## 4. Voucher numbers

- Document names are generated as {company}-{prefix}-{fiscal year}-{number} (e.g. a Sales Invoice NGK-SB-82/83-0000123 ; returns get their own prefix).
- Some documents also carry a human **Voucher No** ( custom\_name , e.g. SGU-RC-000006-82/83 ) built from the company, type code, document number and fiscal year.
- For certain types (Other Purchase Receipts, Purchase Returns, Bank/Journal voucher types, bank receipts), the **Document No** field fills itself with the next number when you open a new form. If you type a number yourself, it is kept — the letter suffix ("65" vs "65A") distinguishes duplicates.
- Duplicate voucher numbers are blocked (cancelled documents excluded, so an amended document reuses its number).

## 5. Vehicles on expense lines

Any Purchase Invoice item or Journal Entry line booked to a vehicle expense account (**Fuel Expenses, R & M - Vehicles, Other Vehicle Expenses**) must have a **Vehicle** selected. The dropdown only offers the vehicles configured on that account (Account → Vehicle List). If you get "Row #N: Vehicle is mandatory...", pick the vehicle; if the dropdown is empty, ask your admin to configure the account's vehicle list. These expenses feed the Vehicle Expense report.

## 6. Master-data helpers

- **Duplicate warning:** saving a Customer/Supplier whose name or VAT/PAN already exists in the same company asks "Do you want to create it again?" — choose No and use the existing record (its ID is a clickable link).
- **Default accounts row:** Customers, Suppliers and Items automatically get one default accounting row with the right Company filled in.
- **Company is locked after saving** — you cannot change a record's company once created (the number series embeds the company). Create a new record instead.
- **Company Mismatch errors** mean you picked a record belonging to a different company — the dropdowns filter for you, but pasted values are checked on save.

## 7. Customer credit & due dates

---

- The invoice **due date** fills automatically from the customer's payment-day terms (Customer → days limit); supplier invoices likewise from the supplier's payment term days.
- Credit-limit blocking (bill count / overdue days / amount) exists in the system but is **currently not active** — ask your administrator about its status before relying on it.

## 8. Cheque bounce

---

If a customer's cheque bounces after you submitted the Payment Entry: open the submitted Payment Entry → **Actions** → **Cheque Bounce** → confirm. The system posts a reversing ledger entry (the invoice becomes outstanding again) and marks the Payment Entry "Cheque Bounced" in red. The original entry is *not* cancelled — the reversal sits alongside it, which keeps the audit trail intact.

## 9. Stock Adjustment (quantity-only correction)

---

For pure quantity corrections (found/lost cylinders etc.) use **Stock Adjustment**: date, company, Loss/Gain, reason, and item rows with warehouse and quantity. **Rate must stay 0** — if value must move, use a Stock Entry instead. On submit the stock quantity changes but the stock *value* is preserved, and this survives later revaluations.

Backdated purchase invoices that update stock show an orange notice that valuation will be reposted overnight — that is informational, not an error.

## 10. Invoice print copies (IRD rule)

---

The system counts how many times each Sales Invoice is actually printed or downloaded as PDF and labels the output accordingly, as IRD requires:

- **1st print** → *Tax Invoice*
- **2nd print** → *Copy of Original*
- **3rd print** → *Copy of Original 2*, and so on.

Only a real print or PDF download counts — opening the Print **preview** does not use up a copy number (the preview shows the title the next print will carry). You don't do anything for this; it is automatic.

## 11. Portals

---

- **/place\_order** — customers order LP Gas online; orders arrive as submitted Sales Orders.
- **/rfq/** — suppliers answer a Request for Quotation online, including VAT, discounts and attachments; a Supplier Quotation is created automatically.
- Customer statement and product-wise invoice portals: see [chapter 7](#).

# CBMS / IRD e-Billing — User Guide

For accounts staff. CBMS is the IRD's Central Billing Monitoring System — every sales invoice must be reported to it. In this system that happens **automatically**; this chapter tells you what to watch and what to do when something needs attention.

## What happens automatically

- When you **submit a Sales Invoice**, a **CBMS Bill** record is created and sent to IRD in the background. You never wait for it — submission is never blocked by CBMS.
- When you submit a **Sales Return** (credit note), a **CBMS Bill Return** is created and sent — after the original invoice's bill has reached IRD (the system enforces the order for you).
- Every 5 minutes the system automatically **retries failures** and **catches any invoice that was missed**. You normally don't need to do anything.

## The one field you fill in

On a **Sales Return**, fill **Reason for Return** — it is reported to IRD. If you leave it blank, "Goods Returned" is sent.

## Monitoring

Open the **CBMS Bill** (and **CBMS Bill Return**) list and filter by **Sync Status**:

| Status  | Meaning                                                         |
|---------|-----------------------------------------------------------------|
| Synced  | reported to IRD — done                                          |
| Pending | queued; will be sent within minutes                             |
| Failed  | IRD rejected it — open the record and read <b>Sync Response</b> |

Each record shows the attempt count, last attempt time, and IRD's last response message. All fields are read-only — you never edit CBMS records.

## Fixing failures

| Sync Response                             | What to do                                                                                                                          |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 100: API credentials do not match         | the IRD username/password in <b>CBMS Config</b> is wrong — fix it there; the retry job re-sends automatically                       |
| Return stuck <b>Pending</b>               | its original invoice's CBMS Bill isn't Synced yet — resolve the original first; the return recovers by itself                       |
| 104: Model invalid                        | something in the invoice data was rejected (e.g. bad PAN) — check the customer's tax id and the invoice values, then ask your admin |
| An invoice has <b>no CBMS Bill at all</b> | the catch-up job creates it within 5 minutes; if it still doesn't appear, ask your admin to check the Error Log                     |

There is no button for an instant retry — the system retries every 5 minutes on its own. An administrator can trigger an immediate retry from the bench console if ever needed.

## Cancelling invoices

**A synced invoice cannot be cancelled** — you'll get an error saying it has already been reported to IRD. This is by design: corrections must be made with a **Sales Return** (credit note), which is itself reported to IRD. Invoices that are still Pending/Failed (not yet at IRD) can be cancelled normally.

## Setup (admin, once per company)

Create one **CBMS Config** per company: tick *Enable CBMS Sync*, set the go-live **Enable From Date** (invoices dated before it are never sent), and enter the company's IRD **Username/Password**. There is no test mode — the config talks to the real IRD system, so only enable it when the credentials are the production ones.

# Document Generator — User Guide

---

For accounts and admin staff. The Document Generator produces formal letters — today mainly the **Customer / Vendor Balance Confirmation** letters — from ready-made templates, filled with live data, editable before printing or emailing.

You need one of these roles: *Document Template User* (generate documents), *Document Template Manager* (also author templates), or *System Manager*.

## Generating a letter

---

1. Open the **Document Generator** page (search "Document Generator" in the awesomebar).
2. **Choose a template** — e.g. *Customer Balance Confirmation*.
3. **Choose the company** the letter is from.
4. **Fill the inputs**. For the balance-confirmation letters:
  - the **Customer** (or **Vendor**) — the dropdown only shows parties of the chosen company;
  - **either** a **Fiscal Year** or a **From/To date range** — they are mutually exclusive; filling one clears the other. Every date field has a paired Nepali (BS) picker that stays in sync.
5. The letter **builds itself** as soon as the inputs are complete — there is no **Generate** button. The rendered letter appears in an editable page.
6. **Edit if needed** — the letter is directly editable (bold, lists, alignment, even inserting an image from the toolbar).
7. From the **Actions** menu choose:
  - **Print (letterhead paper)** — the PDF leaves the letterhead area blank, for printing on pre-printed company paper;
  - **Print (with letterhead)** — the PDF includes the drawn header/footer (logo, company name, VAT no, contacts) — use this for digital copies;
  - **Email** — enter recipients (or leave blank to use the party's email on file); the letter is emailed as a PDF.

Every print or email automatically saves a **Generated Document** record — your archive copy, with the exact inputs used, the final text, and the email status. Open one later and click **Edit in Generator** to revise and re-issue it.

## What the balance-confirmation letters contain

---

A dated (BS) letter addressed to the party, showing for the period: opening balance, taxable sales/purchases, VAT @13%, (TDS for vendors), total, and closing balance — followed by a request to confirm within 7 days and a signature block with your signature image, name, designation and the company stamp.

For your signature and the stamp to appear, the admin must have set up: your Employee record's *signature image* and *Document User* link, and the Company's *document stamp* image ([chapter 8](#)).

## Authoring or changing templates (managers)

---

Templates are **Document Template** records (name, target companies, inputs, data sources, letter body in HTML, header/footer, email subject). Saving validates the template; the **Preview** button renders it with sample data.

Authoring is a semi-technical task (HTML + query writing) — the full authoring guide is in the technical documentation:

[../technical/07-document-generator.md](#)

\$7.

# Reports — User Guide

Every custom report, grouped by who uses it. Open any report from the awesomebar by name. General tips:

- **Company** filters accept multiple companies where shown as a multi-select.
- **Dates:** reports with date filters usually show a Nepali (BS) "Miti" twin and a **Select Month** button — pick a BS month and both dates fill themselves.
- **Printing:** most reports have their own **Download PDF** / Print buttons that produce a properly paginated PDF (with page numbers); some ask Portrait/Landscape first. "Fit Columns" is a screen-view toggle and doesn't affect the print.

## For accountants — compliance (IRD)

| Report                      | What it gives you                                                                                                                                                    |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sales Register              | the VAT sales register: one row per invoice, values split into tax-free / export / taxable / VAT — ready for the VAT return. Returns mode shows credit notes         |
| Purchase Register           | the VAT purchase register: tax-free / taxable / import / capitalized buckets with their VAT amounts and quantity                                                     |
| Advance Tax TDS Details     | the TDS (अग्रिम कर) withholding statement per supplier and category, in IRD format with Nepali headers; print includes the Prepared/Checked/Verified signature block |
| One Lakh Above Transactions | all customers & suppliers whose taxable or exempt turnover in the period is NPR 100,000 or more — the IRD annexure                                                   |

## For accountants — ledgers & cash

| Report                        | What it gives you                                                                                                                                                                                                                  |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Party Ledger                  | a full statement for one party (or grouped statements for many): opening balance, every voucher with running balance, closing. BS Miti dates, remarks, optional item-level detail. This is also what customers see on their portal |
| Party Ledger Summary          | one line per party: opening / debit / credit / closing. "Super Summary" (flat) or "Group Wise" (per customer/supplier group). Excel export shows Dr/Cr columns                                                                     |
| Receipt Register              | customer receipts in three layouts: by date, by customer, or one line per customer                                                                                                                                                 |
| Net Position of Cash and Bank | every cash/bank account's opening, receipts, payments and closing for the period                                                                                                                                                   |
| Loan Summary                  | short- and long-term borrowings per loan type across companies, with a ratio row; "Show Details" expands sub-accounts. Balances are cumulative up to the To Date                                                                   |

## For management

| Report                                        | What it gives you                                                                                                                                         |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consolidated Financial Statement Hierarchy    | Balance Sheet / P&L / Cash Flow consolidated across any set of group companies, with a Total column and a depth control to collapse the account tree      |
| Profit and Loss Hierarchy                     | a single company P&L collapsed to the account level you choose (1-6)                                                                                      |
| Sales Analysis — Customer wise Summary        | per-customer quantity and value, with optional returns and net columns                                                                                    |
| Sales Analysis — Customer wise Details        | customer → product drill-down with VAT and totals                                                                                                         |
| Sales Analysis — Product wise Invoice Details | product → customer → individual invoices and returns, with product and grand totals                                                                       |
| Avinas Vehicle Expense                        | per-vehicle Fuel / Repair / Others expense totals (from purchase invoices and journal entries) for a fiscal year or date range                            |
| Custom Supplier Quotation Comparison          | items as rows, suppliers as columns — compare quoted prices, discounts, VAT and invoice totals side by side and pick a default supplier                   |
| Gas Purchase Report                           | the LPG procurement register per store receipt: refinery, tanker, IOC challan, quantity, rate, VAT, ICP/NA service charges, totals — filtered by BS month |
| Sales Stock Ledger                            | stock movement driven by sales invoices, detail or summarized, with BS dates and an optional sales/returns merge                                          |

## For HR

See [chapter 3 §5](#):  
Monthly Attendance BS, Monthly Attendance Summary BS, Work On Holiday BS, Yearly Leave Details BS, Avinas Salary Statement.

## For auditors / System Managers

| Report                   | What it gives you                                                                                                                                                      |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User Audit Trail         | everything one user created or changed in a date range — per field, old value → new value, including child-table rows. Pick the user, dates, optionally document types |
| User Daily Entry Summary | how many documents a user created on a given day, by type and status (draft/submitted/cancelled); every count is a click-through to the underlying list                |

## Customer portals

Customers with a portal login (Customer role) can self-serve:

| Page                          | What the customer sees                                                                                                                                      |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /customer_statement           | their own account statement (same data as Party Ledger), with AD + BS date pickers and a PDF download. Deposit/security accounts are excluded automatically |
| /product_wise_invoice_details | their own product-wise invoice details, including returns                                                                                                   |
| /place_order                  | place an LP Gas order online (cylinder sizes, live rates, VAT); it arrives as a Sales Order                                                                 |

Customers only ever see their own companies and accounts — every request is re-checked on the server.

# Administrator Setup — User Guide

---

For System Managers. Every feature's setup checklist in one place. Console commands run via `bench --site avinas1 console` (or `bench --site avinas1 execute <dotted.path>`).

## 1. Dynamic Approvals

---

1. **Dynamic Approval Setting** → New: pick the **Document Type** and **Company**; keep the default fieldnames.
2. In the **rule cards** area, *Add Section*: give it a name, add **criteria** (field = value pairs — ALL must match; the rule with the most matching criteria wins) and the ordered **fixed approvers**.  
△ A section with *no* criteria matches nothing — for a catch-all rule add one always-true criterion (e.g. `company = that company`).
3. Click **Setup Workflow** (button, System Manager only). This injects the approval fields onto the document type and creates/updates the "Approval Workflow" (Draft → Pending Approval → Approved / Rejected).
4. Documents with **no matching rule auto-approve** — verify your criteria cover what you intend.
5. After changing the engine or Setting fields, rebuild all workflows:  

```
from avinashgroup_app.custom_code import update_workflows; update_workflows.run()
```
6. Rejection reasons are stored in the document's `custom_reason` field if it exists (e.g. Purchase Order), otherwise as a comment.
7. Only the literal **Administrator** account bypasses approver checks (one level at a time); System Managers do not.

## 2. Biometric devices & the bridge

---

1. Per device: **Biometric Device** → New — name, **hardware serial** (must match exactly; unknown serials are rejected), **Company**, enabled.
2. Per employee: set **Attendance Device ID** (unique within the company), correct Company, and a Shift Assignment. Shift Types need their Company set and auto-attendance enabled.
3. Bridge PC (per site): install `K40BridgeSetup.exe` on an always-on Windows machine on the device LAN. In the wizard: ERPNext URL + an **API key/secret** for a bridge user (needs Employee Checkin create, Biometric Device read/write, Employee read), then one row per device (name, type, IP/port or HTMS folder, serial, company). Test Connection → Save & Start.
4. Verify: tap a finger, check the Employee Checkin list.
5. Alerts: fill **Alert Recipients** + threshold on each device.
6. Optional Shift Type extras: `custom_late_arrival_cutoff_time` enables the automatic Half-Day rule.

## 3. Attendance allowances

---

1. Mark each allowance **Salary Component**: *attendance driven*, condition type, unit, default rate, optional summary group (groups columns in the summary report).
2. Employee overrides: Employee → **Attendance Allowances** table; OT-type rules also need the employee's **OT Eligibility** checkbox.
3. HR runs **Calculate Attendance Allowances** on each Payroll Entry.

## 4. CBMS (IRD e-billing)

---

One **CBMS Config** per company: Enable + **Enable From Date** + IRD username/password. No test mode — production endpoints only. Monitor the CBMS Bill list; errors are in each record's Sync Response and in the Error Log. Immediate manual retry (rarely needed):  

```
frappe.get_doc("CBMS Config", "<company>").sync_failed_now("<company>").
```

## 5. Fiscal-year access control

---

- **Default is full access.** A user is restricted only once a **Fiscal Year Access Control** record exists for them: either *Full Access*, or rows of (Document Type → specific Fiscal Year / all years).
- Administrator and anyone with **System Manager** always bypass.
- It restricts *seeing* (lists, reports, direct opens, API) — it does **not** block saving a document dated outside the range.
- Changes take effect on the user's next request.

## 6. Company filtering

---

- **Company Filter Config** (one per doctype) + **Company Filter Field** rows define which Link fields are filtered to the document's company (plain, child-table, or dynamic-link fields).
- Saving clears the server cache; users must **refresh their browser** to pick up config changes.
- Server-side "Company Mismatch" validation runs on save for all audited doctypes regardless of the JS.

## 7. Audit fields, numbering & company lock

- Audit fields ( created by/on , modified by , company, naming series) are stamped automatically on ~85 doctypes. Manage per doctype from the console — **always pass an explicit list** (no argument = all ~85 doctypes):

```
from avinashgroup_app.utils.audit_file_manager import AuditFieldsManager, update_custom_company_reqd
AuditFieldsManager(["Item Price"]).create_fields()
AuditFieldsManager(["Item Price"]).verify_fields()
AuditFieldsManager(["Item Price"]).remove_fields()
update_custom_company_reqd()
```

- Voucher zero-pad width per doctype: **Voucher Number Settings** (child rows: doctype → digits, default 6).
- Company-field lock (read-only after first save) — runtime block is always on; to also grey the field in the UI:

```
from avinashgroup_app.custom_code.override import setup_company_lock
setup_company_lock.quick_setup(); setup_company_lock.verify_setup()
```

- Track-changes for the audit trail and the (owner, creation) / GL-aggregation indexes are installed by `bench --site avinas1 migrate`.

## 8. User Daily Entry Summary

**User Daily Entry Summary Settings** (single) → add the doctypes to track (keep the list short — one query each per report run), then `migrate` to build the indexes.

## 9. Document Generator

1. Roles come from migrate: *Document Template Manager* (authors), *Document Template User* (generates own documents only).
2. One-time assets: Company → **Document Stamp** image; Employee → **Signature Image** and **Document User** (link to the User who signs — deliberately separate from the employee's login user).
3. The two balance-confirmation templates ship as fixtures; new templates are authored per the technical guide ([../technical/07-document-generator.md](#) §7).

## 10. Housekeeping & diagnostics

| Task                                                                    | Command                                                                                       |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Stock health check (negative stock, zero-valuation sales, repost queue) | <code>bench --site avinas1 execute avinashgroup_app.custom_code.stock_health_check.run</code> |
| Restart failed valuation reposts                                        | <code>console → avinashgroup_app.custom_code.stock_revaluation.reprocess_all_pending()</code> |
| Export all custom fields to Excel                                       | <code>console → avinashgroup_app.utils.export_field.export_custom_fields_to_excel()</code>    |
| Pad voucher numbers to 6 digits (dry run)                               | <code>console → avinashgroup_app.scripts.pad_custom_name.run(dry_run=1)</code>                |
| Remove audit/naming fields from a doctype                               | <code>avinashgroup_app.scripts.cleanup_audit_fields.run(["&lt;DocType&gt;"])</code>           |
| DB replication runbook                                                  | <code>docs/db-master-slave-replication.md</code>                                              |

## 11. Deployment reminders

- After editing client JS: bump its `?v=` in `hooks.py`, `bench build`, restart, and have users hard-refresh.
- After Python changes on live: `bench restart`.
- `bench --site avinas1 migrate` after pulling — patches are idempotent.
- The dev server can mask real errors behind a 500 BrokenPipeError page — check the console/logs, not the HTTP response.